Silent Distributed Cryptography for DNFs and Threshold Policies from Lattices*

Abtin Afshar
UW-Madison†

Rishab Goyal
UW-Madison‡

Saikumar Yadugiri
UW-Madison§

Abstract

We study two central problems in threshold cryptography from lattices: (1) threshold encryption with silent setup for general thresholds $t \geq 2$, where no post-quantum constructions were previously known, and (2) threshold fully homomorphic encryption (TFHE) with sublinear parameters, an open problem since the work of [BGG+18].

We introduce (α, β)-*Scaled Linear Secret Sharing Schemes* (LSSS), a relaxation of standard LSSS in which each authorized set S reconstructs a scaled version of the secret, $\gamma_S \cdot k$, where both the reconstruction coefficients and the set-dependent scaling factor γ_S are bounded over the integers. Unlike prior approaches based on bit decomposition, this preserves the uniform distribution of unauthorized shares. Building on this, we obtain:

- **Silent Threshold and Distributed monotone-policy encryption.** We provide the first post-quantum construction supporting: (i) threshold policies with ciphertexts growing as $\tau^{16} \cdot \mathsf{poly}(\lambda, \log N)$, where $\tau = \min(t^2, N-t)$, and (ii) DNF formulas with fully compact parameters. Our constructions are proven secure under the decomposed LWE assumption in the random oracle model. If we additionally rely on a common reference string, then the ciphertext size for our threshold policy scheme can be reduced to $\tau^4 \cdot \mathsf{poly}(\lambda, \log N)$ under the Succinct LWE assumption.
- **Decentralized TFHE with silent setup.** We provide the first *decentralized* TFHE, for threshold policies and DNFs, from the decomposed LWE assumption. Our construction supports homomorphic evaluation of arbitrary circuits, one-round distributed decryption, and silent setup. This was left as an open problem by [BGG+18] and, prior to this work, we did not have any non-trivial construction for decentralized TFHE from any assumption.
- **Sublinear centralized TFHE from LWE.** We also extend our techniques to *centralized* TFHE. We provide a TFHE scheme under the standard LWE assumption, where all parameters are simultaneously sublinear in N for any threshold t as long as $\min(t^2, N-t) = o(N^{1/6})$. This breaks the $\omega(N)$ barrier that has persisted in the TFHE literature since 2018.

*This paper subsumes an earlier version that contained a subset of current results [AGY26].

†Email: abtin@cs.wisc.edu.

‡Email: rishab@cs.wisc.edu. Research supported by a gift from the Stellar Development Foundation.

§Email: saikumar@cs.wisc.edu.

Contents

1 Introduction **1**
- 1.1 Our Contributions . . . 2

2 Technical Overview **4**
- 2.1 (α, β)-Scaled Linear Secret Sharing . . . 4
- 2.2 Silent Threshold and Distributed Monotone Policy Encryption from Lattices . . . 8
- 2.3 Distributed Threshold FHE with Silent Setup . . . 10
- 2.4 Sublinear Centralized Threshold FHE from LWE . . . 12
- 2.5 Applications and Related Work . . . 12

3 Preliminaries **15**
- 3.1 Lattice Preliminaries . . . 15
- 3.2 Assumptions: LWE and ℓ-Decomposed LWE . . . 16
- 3.3 Matrix Commitment Scheme . . . 17
- 3.4 Explainable Re-randomizer . . . 17
- 3.5 Secret Sharing Schemes . . . 18

4 (α, β)-Scaled Linear Secret Sharing **19**
- 4.1 Defining (α, β)-Scaled LSS . . . 20
- 4.2 Programmability: Capturing Randomness of Unauthorized Shares . . . 21

5 Constructing Programmable (α, β)-Scaled LSS Schemes **22**
- 5.1 Threshold Access Structures: Scaling and Magnifying Shamir's scheme . . . 22
- 5.2 DNF Formula Access Structures . . . 27
- 5.3 Composing Scaled LSS: OR-Threshold-AND Policies . . . 30

6 Monotone Policy Encryption with Silent Setup **33**
- 6.1 Definition . . . 33
- 6.2 Construction . . . 34
- 6.3 Security Analysis . . . 37

7 Monotone Policy Homomorphic Encryption with Silent Setup **42**
- 7.1 Definition . . . 42
- 7.2 Abstracting Homomorphic Evaluation Procedure from [GSW13] . . . 44
- 7.3 Construction . . . 44
- 7.4 Security Analysis . . . 47

8 Threshold Homomorphic Encryption from LWE **52**
- 8.1 Definition . . . 52
- 8.2 Construction . . . 53

9 Compact Fuzzy Identity Based Encryption **59**
- 9.1 Definition . . . 60
- 9.2 Construction . . . 60

A Additional Preliminaries and Proofs **69**
- A.1 Gadget Trapdoor Preliminaries . . . 69
- A.2 Additional Proofs . . . 69

1 Introduction

Threshold cryptography [Des87, DF90, Fra89, DSDFY94] is a fundamental paradigm for distributing trust in cryptographic systems. The central goal is to split a cryptographic secret among N parties such that any authorized coalition, most commonly, any subset of size at least t, can collectively perform a cryptographic operation such as decryption or signing, while any smaller coalition learns nothing. This elegant framework underlies a rich spectrum of applications, from secure multiparty computation [AJL+12, BGG+18] to blockchain-based distributed protocols [MRV+21, DCX+23, GJM+24], and its growing practical importance has recently spurred standardization efforts by the National Institute of Standards and Technology (NIST) for multi-party threshold schemes [BP23].

In the classical formulation, threshold cryptography requires either a trusted dealer who generates and distributes secret key shares, or an interactive *distributed key generation* (DKG) protocol in which the N parties jointly establish a shared public key and distribute secret key shares. While DKG eliminates the trusted dealer, it is expensive in communication and round complexity and must be re-run each time the committee membership or the access structure changes. These practical limitations are especially pronounced in large-scale distributed systems, where parties may be offline, asynchronous, or adversarial.

The Silent Setup Model. A compelling alternative is the *silent setup* model, also called the *ad hoc* [DHMR07, DHMR08] or *distributed* model [WQZD10]. In this model, each party independently and non-interactively generates a public/secret key pair. There is no DKG and no coordination during the setup. The encryptor deterministically aggregates public keys into an encryption key, and decryption proceeds by collecting partial decryptions from any authorized subset of parties. This model immediately inherits a powerful set of features: asynchronous setup, dynamic threshold (any threshold can be chosen at encryption time), multiverse support (any subset of registered users can form a committee without further interaction), and easy addition or removal of users.

A key efficiency desideratum is *compactness*: the parameter sizes do not grow with N. Without compactness, threshold encryption is essentially trivial: one can simply encrypt independently under each user's public key and include all N ciphertexts, but the resulting system scales linearly in the committee size and is impractical for large-scale deployments. A second desideratum, equally important in practice, is *non-interactive share combination*: given partial decryptions asynchronously from any authorized set, reconstruction should proceed without any further communication among the parties. Together, compactness and non-interactive reconstruction are what make threshold cryptography a powerful paradigm rather than a straightforward composition of single-user schemes.

Despite its appeal, realizing *general* threshold encryption in the silent setup model, supporting thresholds $t \geq 2$, has been remarkably difficult, especially so from post-quantum assumptions.

The State of the Art for Silent Threshold Encryption. The special case $t = 1$, *distributed broadcast encryption* (DBE) [WQZD10, BZ14] and its flexible variant [FWW23], has received considerable recent attention. Following early constructions from indistinguishability obfuscation and witness encryption [BZ14, GVW19, FWW23], a series of breakthroughs achieved DBE from pairings [KMW23, GKPW24] and, most recently, from lattice-based assumptions [WW25, AMR25, GY26c, GY26a]. These post-quantum constructions rely on the *decomposed LWE assumption* [AMR25, Wee25], a new falsifiable lattice assumption that is comparable to q-type assumptions popular in pairing-based cryptography.

However, for general thresholds $t \geq 2$, the picture is far less developed. The only known constructions of threshold encryption with silent setup rely on obfuscation [RSY21] or use SNARK machinery in idealized algebraic models [GKPW24, GJM+24]. Very recently, Waters and Wu [WW26] achieved silent threshold cryptography supporting monotone Boolean formula policies from q-type pairing-based assumptions, and [DJWW26] gave distributed monotone-policy encryption from obfuscation for even richer policy classes. However, none of these constructions provide *post-quantum security*, leaving the following question completely open:

Q1. Can we realize threshold encryption with silent setup from lattice assumptions for any $t \geq 2$?

The State of the Art for Threshold FHE. The notion of threshold encryption has also been popularized in the context of fully homomorphic encryption [LTV11, BGG+18]. Starting with the work of Boneh et al. [BGG+18], *threshold fully homomorphic encryption* (TFHE) has been designed under standard lattice assumptions. The Boneh et al. construction comes in two flavors: one based on $\{0,1\}$-linear secret sharing schemes [BL90, LW11], resulting in $O(N^{4.3})$ share size and ciphertexts independent of the circuit size, and another based on Shamir secret sharing [Sha79]. The latter clears the denominators of the Shamir reconstruction coefficients and therefore requires $q > (N!)^{O(1)}$. Although its stated parameter analysis treats the resulting $O(N \log N)$ bit-length of the modulus as the relevant dependence on N, a tighter accounting must also increase the LWE dimension with $\log q$. Under sub-exponential modulus-to-noise ratio, its GSW public key and fresh ciphertext have size $\widetilde{O}(N^6)$, while its secret-key shares and evaluated ciphertext have size $\widetilde{O}(N^3)$. Using the *Universal Thresholdizer* framework [BGG+18], their second construction can be improved to obtain constant-size ciphertexts with all other parameters growing as $O(N \log N)$. Although a rich line of research [BGG+18, ASY22, BS23, CCK23, STH+23, EY24, CKL24, DPKM+24, PS24, DRSNS24, BFM+25, CL25, HLW+25, dVPS26] improved the efficiency and security of TFHE and many related primitives, the following fundamental question has remained open:

Q2. Can all parameters of TFHE *be simultaneously sublinear in N?*

Moreover, all of these TFHE schemes rely on a (centralized) trusted setup. To remedy this, Boneh et al. [BGG+18] also introduced the notion of *decentralized/distributed* TFHE, and left it as an important open problem to construct it. Despite significant interest, the following is still open:

Q3. Does there exist any non-trivial *decentralized* TFHE *scheme from any assumption?*

Here 'non-trivial' means the parameter sizes are sublinear in N, number of distributed parties.

The Reconstruction Coefficient Barrier. A major obstacle to lattice-based threshold cryptography is that the *reconstruction coefficients in LSSS are incompatible with the noise management requirements of lattice-based cryptography.* In a standard LSSS, any authorized set S reconstructs the secret k as $k = \sum_{i \in S} c_{S,i} \cdot \mathsf{sh}_i$. In lattice-based systems, the reconstruction coefficients $c_{S,i}$ multiply with the LWE noise, and if these are large, the correctness fails. For Shamir secret sharing over N parties, the Lagrange coefficients modulo q can be as large as $q-1$, forcing the modulus to scale as $\Omega(N \log N)$ to absorb the resulting noise growth [BGG+18]. A common workaround of encoding each share via bit decomposition destroys the *uniform distribution of unauthorized shares*, a property that is essential for lattice security proofs: without it, the reduction cannot program the LWE challenge into the shares of corrupt users. As demonstrated by Ball, Çakan, and Malkin [BÇM21], constructing an LSSS with short reconstruction coefficients and short shares for expressive policies is notoriously tricky and faces strong theoretical barriers.

1.1 Our Contributions

In this work, we make fundamental progress on all three questions. Throughout this section, $\tau = \min(t^2, N-t)$ for a threshold t over N users, and $\epsilon \in (0, 1/2)$ is the constant from the lattice parameter convention of Section 6.2. $O^\dagger(\cdot)$ hides $\mathsf{poly}(\lambda, \log N)$ factors; for the homomorphic results, $O^\dagger_{\ell,d}(\cdot)$ and $O^\dagger_d(\cdot)$ additionally hide factors polynomial in the evaluated circuit's input length ℓ and/or depth d.

Silent Threshold and Distributed Monotone Policy Encryption. Our first set of results concerns *distributed monotone-policy encryption* (DPE), the silent-setup analogue of policy-based threshold encryption [DJWW26, WW26]. In a DPE scheme, each user independently generates a key pair; the encryptor aggregates public keys into a master encryption key; and any authorized set of users can jointly decrypt. Our constructions rely on the decomposed LWE assumption [AMR25, Wee25, WW25], the same falsifiable lattice assumption used in recent state-of-the-art work on distributed broadcast encryption (i.e., silent threshold encryption for $t = 1$) from lattices.[1] We construct DPE schemes for two natural policy classes:

[1] At present, we do not know how to construct even broadcast encryption from lattice assumptions such as LWE [Reg05].

Informal Theorem 1. *(Silent Threshold & DPE for DNF Formulas with Silent Setup.) Assuming the hardness of the decomposed LWE assumption, there exists a secure* DPE *scheme with silent setup in the random oracle model supporting:*

(a) threshold access structures where the public keys, secret keys, ciphertexts, and partial decryptions have size $O^\dagger(\tau^3)$, $O^\dagger(\tau^{15})$, $O^\dagger(\tau^{16})$, and $O^\dagger(\tau)$, respectively. In particular, all parameters are sublinear in N whenever $\tau = o(N^{1/16})$; and
(b) DNF formula access structures where the ciphertexts, and partial decryptions all have size $O^\dagger(1)$ with compact public-secret key pairs up to a reuse factor.

Theorem 1 yields the first post-quantum construction for threshold broadcast encryption with silent setup for $t \geq 2$, and the first silent threshold encryption with compact parameters for a large range of thresholds, both from falsifiable lattice assumptions. Concretely, by choosing ϵ sufficiently close to $1/2$, we obtain compact parameters for thresholds $t \in [2, N^{\varepsilon/2}] \cup [N - N^\varepsilon, N - 1]$ for every constant $\varepsilon < 1/16$.

Alternatively, if we rely on a common reference string, then we can improve the parameters of Theorem 1 for threshold policies as follows:

Informal Theorem 2. *(DPE for Threshold Policies with CRS.) Assuming the hardness of the succinct LWE assumption, there exists a secure* DPE *scheme in the random oracle model supporting threshold access structures where the public keys, secret keys, ciphertexts, and partial decryptions have size $O^\dagger(\tau^3)$, $O^\dagger(\tau^3)$, $O^\dagger(\tau^4)$, and $O^\dagger(\tau)$, respectively. In particular, all parameters are sublinear in N whenever $\tau = o(N^{1/4})$.*

Decentralized Threshold FHE with Silent Setup. Our next result resolves the problem of *distributed* (or *decentralized*) threshold FHE, that we generalize and call *silent monotone-policy FHE* (SFHE). In SFHE, the setup is fully non-interactive, and the system supports homomorphic evaluation of arbitrary Boolean circuits followed by threshold decryption. We construct SFHE schemes for the same policy classes as our DPE schemes. *This is the first non-trivial decentralized monotone-policy FHE scheme for any access structure from any assumption.*

Informal Theorem 3. *(Decentralized Monotone-Policy FHE with Silent Setup.) Assuming the hardness of the decomposed LWE assumption, there exists a secure* SFHE *scheme in the random oracle model supporting:*

(a) threshold access structures where the public keys, secret keys, evaluated ciphertexts, and partial decryptions have size $O^\dagger_{\ell,d}(\tau^6)$, $O^\dagger_{\ell,d}(\tau^{18})$, $O^\dagger_{\ell,d}(\tau^{16})$, and $O^\dagger_{\ell,d}(\tau^3)$, respectively. In particular, all parameters are sublinear in N whenever $\tau = o(N^{1/18})$; and
(b) DNF formula access structures with fully compact parameters $O^\dagger_{\ell,d}(1)$.

Additionally, assuming circular security [Gen09] we can remove the dependence on d. Again, if we assume a common reference string, then we can improve the parameters of Theorem 3 for threshold policies as follows:

Informal Theorem 4. *(SFHE for Threshold Policies with CRS.) Assuming the hardness of the succinct LWE assumption, there exists a secure* SFHE *scheme in the random oracle model with a structured common reference string, supporting threshold access structures. Using the bounded-integer representation of the evaluation matrix, the public keys, secret keys, evaluated ciphertexts, and partial decryptions have size $O^\dagger_{\ell,d}(\tau^6)$, $O^\dagger_{\ell,d}(\tau^6)$, $O^\dagger_{\ell,d}(\tau^6)$, and $O^\dagger_{\ell,d}(\tau^3)$, respectively. In particular, all parameters are sublinear in N whenever $\tau = o(N^{1/6})$.*

As we discuss in Section 5.3 our DPE and SFHE schemes extend to a more expressive class of policies, namely *OR-of-Thresholds-of-ANDs* formulas, which subsume both threshold and DNF access structures. All our DPE and SFHE constructions satisfy selective security. The problem of designing even adaptively-secure DPE with compactness, even for $t = 1$, is an important open problem in the area. It requires overcoming well-established proof barriers in lattice-based cryptography [BM24].

Sublinear Centralized Threshold FHE from LWE. In the centralized setting, we partly resolve the sublinear parameter question for TFHE under the hardness of standard LWE assumption [Reg05].

Informal Theorem 5. *(Sublinear Threshold FHE from LWE.) Assuming the hardness of the* standard *LWE assumption, there exists a* TFHE *scheme for threshold access structures where the public key and fresh ciphertext have size $O^{\dagger}_d(\tau^6)$, each secret-key share and evaluated ciphertext have size $O^{\dagger}_d(\tau^3)$, and each partial decryption has size $O^{\dagger}_d(\tau)$. In particular, all parameters are sublinear in N whenever $\tau = o(N^{1/6})$.*

In particular, all parameters are simultaneously sublinear for thresholds $t \in [2, N^{\varepsilon/2}] \cup [N - N^{\varepsilon}, N - 1]$ for every constant $\varepsilon < 1/6$. Applying a bootstrapping compiler similar to [BGG+18], we further obtain evaluated ciphertexts whose size is independent of the evaluated circuit size. The secret-key shares grow with $O^{\dagger}_d(\tau^3)$. Furthermore, assuming circular security, we can remove the dependence on d. Our construction works nearly in a black-box manner over existing lattice-based FHE schemes [BV11, BGV12, GSW13]. This is the first TFHE scheme with all parameters simultaneously sublinear in N for any non-trivial class of threshold access structures, breaking the linear-parameter barrier that has persisted since [BGG+18].

Applications. By plugging our TFHE (Informal Theorem 5) into the Universal Thresholdizer framework of [BGG+18], we immediately obtain post-quantum constructions with *sublinear parameters* for threshold signatures, threshold PRFs, threshold IBE, and any other primitive that can be thresholdized by TFHE. Furthermore, our SFHE (Informal Theorem 3) can be used to design a Universal Thresholdizer in the *silent setup* model, and would enable new approaches to ad-hoc / silent-setup versions for many threshold primitives from lattice assumptions. For instance, our SFHE directly gives the first CCA-secure threshold PKE with fully silent setup from post-quantum assumptions, where the ciphertext size does not scale with the number of servers. We discuss this further in Section 2.5.

New Abstraction: (α, β)-Scaled Linear Secret Sharing. An important driving force for our results is a new secret sharing notion, that we call (α, β)-*Scaled Linear Secret Sharing Schemes* (Scaled LSSS). This new primitive is designed precisely to bypass the Reconstruction Coefficient Barrier described earlier, and we believe it will find applications broadly in lattice-based cryptography beyond this work. Some examples include threshold MPC with sublinear communication complexity, proactive and verifiable secret sharing, and threshold signatures. As a concrete example, we obtain compact Fuzzy Identity-Based Encryption [SW05] from lattices by using Scaled LSSS, significantly improving the parameters obtained by [ABV+12].

The key relaxation. In a standard LSSS, authorized sets reconstruct the secret exactly: $\sum_{i \in S} c_{S,i} \cdot \mathsf{sh}_i = k$. Our key insight is that, for lattice applications, it suffices to reconstruct a *scaled* version of the secret: $\sum_{i \in S} \delta_{S,i} \cdot \mathsf{sh}_i = \gamma_S \cdot k$, where $\delta_{S,i}$ are *scaled reconstruction coefficients* bounded by $|\delta_{S,i}| \leq \alpha$ and $\gamma_S \neq 0$ is a public *set-dependent scaling factor* bounded by $|\gamma_S| \leq \beta$. Crucially, this extra degree of freedom allows us to choose $\delta_{S,i}$ that are truly small over the integers, without any bit decomposition of shares, thus preserving the uniform distribution of unauthorized shares.

Beyond scaled reconstruction, we formalize a new notion of a *programmable* (α, β)-Scaled LSSS, that additionally requires the shares of any unauthorized set of parties to look jointly uniform and can be set independently of the remaining shares. This programmability property is precisely what enables the security reduction to embed its LWE challenge into the shares of corrupt users.

2 Technical Overview

The overview is split into four parts. First, we give a brief introduction to the (α, β)-Scaled LSSS. Next, we explain our ideas for designing silent distributed-policy encryption from lattices. In the third part, we describe how to add homomorphism functionality to silent distributed-policy encryption. Finally, we discuss our approach to designing a centralized threshold FHE with sublinear parameters from standard LWE. We conclude with a brief discussion about applications and comparison with concurrent and related works.

2.1 (α, β)-Scaled Linear Secret Sharing

Linear secret sharing: a quick review. A linear secret sharing scheme (LSSS) for a monotone access structure $\mathbb{A}$ over N parties encodes a secret $k \in \mathbb{Z}_q$ into a vector of shares $(\mathsf{sh}_1, \ldots, \mathsf{sh}_N) \leftarrow \mathsf{Share}(\mathbb{A}, k)$

such that any authorized set $S \in \mathbb{A}$ can recover k via a weighted linear combination of its shares: $k = \sum_{i \in S} c_{S,i} \cdot \mathsf{sh}_i$, where $c_{S,i} \in \mathbb{Z}_q$ are the *reconstruction coefficients* for set S. Any unauthorized set $S \notin \mathbb{A}$ learns nothing about k from its shares.

A useful additional property, which we formalize as *programmability* in this work, captures the intuition that any set of unauthorized shares look jointly random. Formally, this means one can efficiently sample shares for an unauthorized set first, at random, and then consistently complete the remaining shares so that the full share vector is distributed exactly as a standard sharing of any desired secret which might not be random. This property is crucial in lattice-based security proofs: the reduction must embed its LWE challenge into the shares held by corrupt users, and programmability is precisely what enables this embedding without affecting the distribution seen by the adversary.

The reconstruction coefficient barrier. Short reconstruction coefficients and programmability together form a powerful combination for thresholdizing lattice-based cryptography. In a lattice scheme, partial decryption contributions are computed by multiplying the reconstruction coefficients into noisy LWE terms; thus the noise in the final decryption grows proportionally to $|S| \cdot \max_i |c_{S,i}|$. For the scheme to be correct, this noise must remain below $q/4$, which forces the modulus q to be at least as large as the coefficient magnitude. Since all parameter sizes grow with $\log q$, having small reconstruction coefficients is critical for compact schemes.

This obstacle has been recognized explicitly in numerous prior works [ABV+12, BLMR13, BGG+18, BÇM21, ASY22, BS23]. The natural candidate for threshold structures is Shamir secret sharing [Sha79], but its Lagrange reconstruction coefficients modulo q can be as large as $q-1$, making them useless as-is. A standard workaround is to decompose each share into its binary representation. This yields bounded coefficients in $\{0,1\}$, but completely destroys the programmability property as the resulting bit-decomposed share vectors are no longer uniformly distributed for unauthorized sets, which breaks the LWE embedding at the core of the security proof. As Ball, Çakan, and Malkin [BÇM21] demonstrate, constructing an LSSS with simultaneously short coefficients, short shares, and programmability for expressive policies faces strong theoretical barriers.

(α, β)-Scaled LSSS. We introduce the notion of (α, β)-*Scaled Linear Secret Sharing*. In a standard LSSS, reconstruction recovers k exactly. We relax this: for each authorized set S, there exist *scaled reconstruction coefficients* $\delta_{S,i}$ and a nonzero *set-dependent scaling factor* γ_S such that

$$\sum_{i \in S} \delta_{S,i} \cdot \mathsf{sh}_i \;=\; \gamma_S \cdot k,$$

where $|\delta_{S,i}| \le \alpha$ and $|\gamma_S| \le \beta$ as integers. Crucially, we want programmability to be preserved: shares for any unauthorized set should remain jointly uniform, since the relaxation affects only the reconstruction identity and not the distribution of the shares themselves. This is the key distinction from bit decomposition: by keeping shares in their natural form and instead absorbing the arithmetic difficulty into a public scaling factor, we maintain the uniformity that the security proof requires.

An important distinction from prior works in threshold lattice cryptography [ABV+12, BLMR13, BGG+18] is that the scaling factor γ_S depends on the *specific* reconstructing set S, not merely on the threshold t or any fixed public parameter. Prior works either require $\gamma = 1$ exactly (which hits the [BÇM21] barrier) or use a globally-fixed γ (that is, γ is the same for all authorized sets, thus is typically very large; e.g. $\gamma \propto N!$ [ABV+12, BLMR13, BGG+18]).

This reconstruction set-dependent scaling gives us much needed flexibility. As we show, this lets us realize scaled LSSS with small coefficients for a broader class of access structures. However, this added flexibility comes at a cost: because γ_S is not available at encryption or partial-decryption time, using these schemes inside lattice-based constructions is no longer black-box. Making such applications go through requires a careful security analysis, especially in how the underlying noise is smudged. We discuss the application of scaled LSSS, and the challenges it raises, in the following toy example.

Toy example: thresholdizing Regev encryption. We illustrate the utility of (α, β)-Scaled LSSS via a toy threshold version of Regev encryption [Reg05]. Recall that a Regev public key has the form $(\mathbf{A}, \mathbf{s}^\top \mathbf{A} + \mathbf{e}^\top)$

and a ciphertext encrypting $\mu \in \{0,1\}$ has the form

$$(\mathbf{c}_1, c_2) = (\mathbf{Ar},\ \mathbf{s}^\top \mathbf{Ar} + \mathbf{e}^\top \mathbf{r} + \mu \cdot \lfloor q/2\beta \rfloor)$$

for a random vector $\mathbf{r}$.[2] The secret is the LWE vector $\mathbf{s} \in \mathbb{Z}_q^n$.

The natural approach to thresholdize this scheme is to secret-share $\mathbf{s}$ between N parties: $(\mathsf{sh}_1, \ldots, \mathsf{sh}_N) \leftarrow \mathsf{Share}(\mathbb{A}, \mathbf{s})$, where each $\mathsf{sh}_i \in \mathbb{Z}_q^n$. For partial decryption, party i computes

$$\mathsf{dsh}_i \ = \ \mathsf{sh}_i^\top \cdot \mathbf{c}_1 \ = \ \mathsf{sh}_i^\top \cdot (\mathbf{Ar})$$

using only the first component of the ciphertext and its own share. For reconstruction, given the partial decryptions $\{\mathsf{dsh}_i\}_{i \in S'}$ for an authorized set $S' \in \mathbb{A}$, one computes the scaled coefficients by running the following set-dependent coefficient generation algorithm $((\delta_{S',i})_{i \in S'}, \gamma_{S'}) \leftarrow \mathsf{GetCoeffs}(\mathbb{A}, S')$ and forms

$$\sum_{i \in S'} \delta_{S',i} \cdot \mathsf{dsh}_i \ = \ \sum_{i \in S'} \delta_{S',i} \cdot \mathsf{sh}_i^\top \cdot (\mathbf{Ar}) \ = \ \left(\sum_{i \in S'} \delta_{S',i} \cdot \mathsf{sh}_i \right)^\top (\mathbf{Ar}) \ = \ (\gamma_{S'} \cdot \mathbf{s})^\top (\mathbf{Ar}).$$

The decryptor then computes $y = \gamma_{S'} \cdot c_2 - \sum_{i \in S'} \delta_{S',i} \cdot \mathsf{dsh}_i$ where $c_2 = \mathbf{s}^\top \mathbf{Ar} + \mathbf{e}^\top \mathbf{r} + \mu \lfloor q/2\beta \rfloor$:

$$y \ = \ \gamma_{S'} \cdot (\mathbf{e}^\top \mathbf{r} + \mu \lfloor q/2\beta \rfloor) \ \approx \ \gamma_{S'} \mu \cdot \lfloor q/2\beta \rfloor \pmod q,$$

from which μ is recovered as long as $\gamma_{S'}$ is nonzero and known to the decryptor. Correctness holds as long as $\gamma_{S'} m \cdot \|\mathbf{e}\| \cdot \|\mathbf{r}\|$ remains small relative to $q/4$, which is guaranteed when $|\gamma_{S'}| \le \beta$ and β is small.

The same strategy, however, fails for a general LSSS with unbounded coefficients. Indeed, in the above decryption procedure, the final noise is multiplied by $\gamma_{S'}$, so the correctness requires $|\gamma_{S'}| \cdot |\mathbf{e}^\top \mathbf{r}| \ll q$ for every authorized set S'. Since $|\gamma_{S'}| \le \beta$, the modulus must dominate $\beta \cdot \|\mathbf{e}\| \cdot \|\mathbf{r}\|$. If β is itself as large as q, as in raw Shamir-type reconstruction, this requirement becomes circular and gives no meaningful parameter regime. Thus, the role of (α, β)-Scaled LSSS is to achieve scaled reconstruction with *polynomially bounded* α and β, so that correctness remains feasible while programmability is preserved.

We stress that $\gamma_{S'}$ depends on the actual set S' that performs reconstruction, not just on the access structure $\mathbb{A}$. This is a fundamental departure from [Sho00, ABV+12, BLMR13, BGG+18], where the analogous scaling factor is fixed for all authorized sets of a given size. In our construction, the encryptor does not know in advance which set S' will decrypt, and $\gamma_{S'}$ is not available to the individual parties at partial-decryption time either. Consequently, $\gamma_{S'}$ cannot be hardwired into the ciphertext or used in the decryption procedure. Rather, the set S' is available only at the reconstruction stage, where the combiner uses it together with the corresponding partial decryptions to recover the message. This set-dependence propagates throughout the simulation argument for partial decryptions and requires careful handling.

Toy example: simulating Regev partial decryptions. To see concretely how this set-dependence complicates simulation, fix a maximally invalid corrupted set C and consider an honest party $i \notin C$. Let $S = C \cup \{i\}$. By scaled reconstruction, for $((\delta_{S,j})_{j \in S}, \gamma_S) \leftarrow \mathsf{GetCoeffs}(\mathbb{A}, S)$, we have

$$\delta_{S,i} \mathsf{sh}_i = \gamma_S \mathbf{s} - \sum_{j \in C} \delta_{S,j} \mathsf{sh}_j.$$

Multiplying by $\mathbf{Ar}$ and using $\mathbf{s}^\top \mathbf{Ar} = c_2 - \mathbf{e}^\top \mathbf{r} - \mu \lfloor q/2\beta \rfloor$, we obtain

$$\mathsf{dsh}_i = \delta_{S,i}^{-1} \left(\gamma_S \left(c_2 - \mu \lfloor q/2\beta \rfloor \right) - \sum_{j \in C} \delta_{S,j} \mathsf{dsh}_j \right) - \delta_{S,i}^{-1} \gamma_S \mathbf{e}^\top \mathbf{r}.$$

[2] We make a slight modification to classic Regev encryption, wherein the message is encoded by multiplying $\lfloor q/2\beta \rfloor$ instead of $\lfloor q/2 \rfloor$. This is to ensure that the message will be held in higher-order bits after decryption since it is multiplied by $\gamma_{S'}$.

Given this, a natural simulation strategy is for the simulator to compute the known terms on the right-hand side (i.e., those depending on $c_2, \mu, \{\mathsf{dsh}_j\}_{j\in C}$) and smudge the residual error term $\delta_{S,i}^{-1}\gamma_S \mathbf{e}^\top \mathbf{r}$. The difficulty is that, while γ_S is bounded over the integers, the inverse $\delta_{S,i}^{-1} \in \mathbb{Z}_q$ need not be; in particular, it may be arbitrarily large as a field element. Thus, the usual smudging argument can no longer be applied, since the effective error term that is leaked through a partial decryption share is no longer a bounded value.

We overcome this by a *double-smudging* argument; we add a smudging noise to, *both*, partial decryption shares as well as public keys. Concretely, we perturb the public key to $(\mathbf{A}, \mathbf{s}^\top\mathbf{A} + \mathbf{e}^\top + \hat{\mathbf{e}}^\top)$, where $\hat{\mathbf{e}}$ is an additional wide error term. In the simulation, this auxiliary noise $\hat{\mathbf{e}}$ enables us to statistically replace the original error term $\mathbf{e}^\top\mathbf{r}$ by a "scaled" error term of the form $\delta_{S,i}\widetilde{\mathbf{e}}^\top\mathbf{r}$. Namely, we obtain

$$\mathsf{dsh}_i = \delta_{S,i}^{-1}\left(\gamma_S\left(c_2 - \mu\lfloor q/2\beta\rfloor\right) - \sum_{j\in C}\delta_{S,j}\mathsf{dsh}_j\right) - \cancel{\delta_{S,i}^{-1}}\,\gamma_S(\cancel{\delta_{S,i}}\,\widetilde{\mathbf{e}}^\top\mathbf{r}) - \delta_{S,i}^{-1}\gamma_S\hat{\mathbf{e}}^\top\mathbf{r}.$$

Now, we can use a fresh smudging noise (added to dsh_i) to hide $\delta_{S,i}^{-1}\gamma_S(\delta_{S,i}\widetilde{\mathbf{e}}^\top\mathbf{r}) = \gamma_S\widetilde{\mathbf{e}}^\top\mathbf{r}$, which is smudgeable since $|\gamma_S|$ is bounded. The price for this repair is that the decryption share now also contains an additional contribution coming from the extra smudging term that we added, namely $\delta_{S,i}^{-1}\gamma_S\hat{\mathbf{e}}^\top\mathbf{r}$. Clearly, this term is not small, thus can no longer be smudged. However, we prove that it is okay for the simulator to know $\hat{\mathbf{e}}$. Accordingly, dsh_i is simulated as

$$\mathsf{dsh\text{-}sim}_i \approx_s \delta_{S,i}^{-1}\left(\gamma_S\left(c_2 - \mu\lfloor q/2\beta\rfloor\right) - \sum_{j\in C}\delta_{S,j}\mathsf{dsh}_j\right) + \delta_{S,i}^{-1}\gamma_S\hat{\mathbf{e}}^\top\mathbf{r} + e_{\mathsf{sm}}$$

where e_{sm} is the smudging error. Consequently, the simulator carries leakage of the form $(\hat{\mathbf{e}}, \hat{\mathbf{e}}^\top\mathbf{r})$, and semantic security must therefore be re-established using a more careful application of the generalized leftover hash lemma and a scaled-error version of LWE (which follows from standard LWE).

Constructing Programmable (α, β)-Scaled LSSS. We construct programmable (α, β)-Scaled LSSS for threshold and DNF-formula access policies.

Threshold Policies: Sublinear t. For Shamir secret sharing at evaluation points $\alpha_i = i$ over $\mathbb{Z}_q$ with $q > N$, the Lagrange reconstruction coefficients modulo q are arbitrarily large, but their rational forms over $\mathbb{Z}$ are well-controlled for small t: each coefficient $\lambda_i = n_i/d_i$ has numerator and denominator both bounded by N^{t-1} in absolute value. Our key observation is that the Least Common Multiple (LCM) of all denominators $\{d_i\}_{i\in S_0}$ for any authorized set S_0 of size t is bounded by $D = \mathrm{LCM}(|d_i| : i \in S_0) \le N^{t(t-1)/2}$.

Setting $\gamma_S = D$ and $\delta_{S,i} = D\cdot(n_i/d_i) \in \mathbb{Z}$ clears all denominators without any modular inversion, yielding reconstruction coefficients that are integers bounded by $N^{t^2/2}$ in absolute value. Programmability follows from the interpolation structure: any invalid set fixes fewer than t evaluations of the degree-$(t-1)$ sharing polynomial, leaving the remaining coefficients uniformly random, while a maximally invalid set plus one more prescribed value determines the unique consistent polynomial. Therefore, the programmed shares are distributed exactly as in an honest Shamir's secret sharing. Thus, Shamir's LSSS is already a programmable $(N^{t^2/2}, N^{t^2/2})$-Scaled LSSS. This translates to parameter sizes of $O(t^2 \log N)$ bits.

Threshold Policies: Sublinear $N-t$. When t is large, the aforementioned scaling argument no longer gives useful bounds, as the denominators of the Lagrange coefficients for large t each involve products of $t-1$ terms. Instead, we propose a *magnified Shamir* LSSS: multiply each party i's share by the publicly known Lagrange coefficient $\Lambda_i = \lambda_{i,0}^{[N]}$ for reconstructing from the full set $[N]$. After this magnification, the reconstruction identity for any authorized set S can be rewritten so that only the *missing* parties $[N]\setminus S$ appear:

$$\sum_{i\in S}\delta_{S,i}\cdot\mathsf{sh}_i \;=\; \gamma_S\cdot k, \quad \text{where } \delta_{S,i} = \prod_{j\in[N]\setminus S}(\alpha_i - \alpha_j), \quad \gamma_S = \prod_{j\in[N]\setminus S}(-\alpha_j).$$

Since $|[N]\setminus S| = N - t$, both $|\delta_{S,i}|$ and $|\gamma_S|$ are products of at most $N-t$ terms, each bounded by N, giving $\alpha, \beta \le N^{N-t}$. The programmability of magnified Shamir LSSS also follows from that of ordinary Shamir

LSSS, since the magnification factors Λ_i are fixed public nonzero constants. This gives a programmable (N^{N-t}, N^{N-t})-Scaled LSSS with parameter sizes $O((N-t)\log N)$.

DNF formulas. The Lewko–Waters LSSS for DNF formulas [LW11], together with prior linear-independence observations [DKW21, ABN+22], directly gives a programmable $(1,1)$-Scaled LSSS. We provide the formal statement and proof in Section 5.2.

We highlight that these LSSS constructions compose. In Section 5.3, we show that correctness, privacy, and bounded scaled reconstruction are preserved under *arbitrary* composition trees over our base schemes, whereas *programmability* is preserved only for three-layer OR-of-Thresholds-of-ANDs policies (which subsume DNFs). This still yields an expressive class of policies with near-optimal parameters. At the same time, it points to a clean frontier: if one could obtain threshold scaled-LSSS with $\alpha, \beta \leq N^{\log N}$, or prove that suitable compositions of our current components preserve such bounds, then our framework would immediately give a fully-optimal compact silent TFHE.

2.2 Silent Threshold and Distributed Monotone Policy Encryption from Lattices

We now describe how to use (α, β)-Scaled LSSS to build *Distributed Monotone Policy Encryption* (DPE), the silent-setup analogue of policy-based threshold encryption. In a DPE scheme, each user i independently runs KGen to generate $(\mathsf{pk}_i, \mathsf{sk}_i)$. The encryptor aggregates a set $\{\mathsf{pk}_i\}_{i\in S}$ and an access policy $\mathbb{A}$ into a master public key mpk via a public-coin $\mathsf{PreProcess}$ algorithm. This mpk can be used to encrypt a message μ and generate ciphertext ct. Any authorized subset $S' \in \mathbb{A}$ collectively decrypt ct using one round of partial decryptions.

Starting point: DBE from lattices. Our construction builds on the recent Wee-Wu selectively-secure *Distributed Broadcast Encryption* (DBE) scheme [WW25], which is a special case of DPE where the access structure is a disjunction of all users which can be thought of as a threshold access structure with $t = 1$. Their construction relies on the hardness of the decomposed LWE assumption [AMR25, Wee25] and matrix commitment scheme (MCOM). We briefly explain these below.

Matrix commitments and decomposed LWE. The MCOM scheme takes as implicit public parameters $(\mathbf{W}, \mathbf{D}, \mathbf{B})$ sampled from the decomposed LWE assumption and contains three deterministic algorithms $(\mathsf{Com}, \mathsf{Verify}, \mathsf{Open})$. For any matrix $\mathbf{M} \in \mathbb{Z}_q^{n\times N}$, these satisfy $\mathbf{C}\mathbf{V}_N = \mathbf{M} - \mathbf{B}\mathbf{Z}$, where $\mathbf{C} = \mathsf{Com}(\mathbf{M}) \in \mathbb{Z}_q^{n\times m}$, $\mathbf{V}_N = \mathsf{Verify}(1^N)$, and $\mathbf{Z} = \mathsf{Open}(\mathbf{M})$. $\mathbf{V}_N, \mathbf{Z}$ have bounded norms. Decomposed LWE assumption [AMR25] itself states that given $(\mathbf{W}, \mathbf{D}, \mathbf{B})$, $\mathbf{s}^\top\mathbf{B} + \mathsf{err}$ is computationally indistinguishable from a uniformly random vector.

In the Wee-Wu DBE scheme, crs contains $(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{A}, \mathbf{p})$ where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n\times m}$. Each user's key pair is $(\mathsf{pk}_i, \mathsf{sk}_i) = (\mathbf{B}\mathbf{r}_i - \mathbf{A}\mathbf{v}_i, \mathbf{r}_i)$ for $\mathbf{r}_i \leftarrow \{\pm 1\}^{4m^5}$ and $\mathbf{v}_i$ is the i-th column of $\mathbf{V}_N = \mathsf{Verify}(1^N)$. Encryption for a set S constructs a commitment to the matrix $\mathbf{M} = [\mathbf{p} + \mathsf{pk}_i \text{ for } i \in S]$ (with $\mathbf{0}$ outside S) and produces a ciphertext $\mathsf{ct} = (\mathbf{c}_1^\top := \mathbf{s}^\top\mathbf{B} + \mathsf{err}, \mathbf{c}_2^\top := \mathbf{s}^\top(\mathbf{A}+\mathbf{C}) + \mathsf{err}, c_3 := \mathbf{s}^\top\mathbf{p} + \mathsf{err} + \mu\lfloor q/2\rfloor)$ for a fresh $\mathbf{s} \leftarrow \mathbb{Z}_q^n$. Any user $j \in S$ can then compute $\mathbf{c}_2^\top\mathbf{v}_i + \mathbf{c}_1^\top\mathbf{z}_i \approx \mathbf{s}^\top(\mathbf{p} + \mathsf{pk}_i + \mathbf{A}\mathbf{v}_i)$ where $\mathbf{z}_i$ is the i-th column of $\mathbf{Z} = \mathsf{Open}(\mathbf{M})$. The decryptor can compute $\mathbf{M}$ exactly as in encryption as it knows the set S. Then using $\mathbf{r}_j$, recovers μ with high probability.

From DBE to DPE: secret sharing the one-time pad. To generalize from $t = 1$ to a general access structure $\mathbb{A}$, our idea is that an encryptor will sample fresh $\mathbf{p}$ each time and secret share $\mathbf{p}$ using a programmable (α, β)-Scaled LSSS. Rather than embedding the same $\mathbf{p}$ in each column of $\mathbf{M}$, we share $\mathbf{p}$ as $(\mathsf{sh}_{i_1}, \ldots, \mathsf{sh}_{i_{|S|}}) \leftarrow \mathsf{Share}(\mathbb{A}, \mathbf{p}; r)$ and embed one share per user, i.e,. $\mathbf{M} = [\mathsf{pk}_i + \mathsf{sh}_i \text{ for } i \in S]$. The encryptor then commits to this matrix and encrypts as in DBE. A user j performing partial decryption recovers $\mathbf{s}^\top\mathsf{sh}_j + \mathsf{err}$, and an authorized set $S' \in \mathbb{A}$ combines the partial decryptions:

$$\sum_{j\in S'} \delta_{S',j} \cdot (\mathbf{s}^\top\mathsf{sh}_j) \;=\; \mathbf{s}^\top\left(\sum_{j\in S'} \delta_{S',j}\mathsf{sh}_j\right) \;=\; \mathbf{s}^\top(\gamma_{S'} \cdot \mathbf{p}),$$

from which μ is recovered by computing $\gamma_{S'} c_3 - \sum \delta_{S',j} \mathsf{dsh}_j$. Since $|\delta_{S',j}| \le \alpha$ and $|\gamma_{S'}| \le \beta$ over $\mathbb{Z}$, the accumulated noise remains below $q/4$ as long as q is chosen appropriately, and correctness holds.

One subtlety in the above construction is that the sharing randomness r must be included in the ciphertext so that each party can compute $\mathbf{M}$ exactly as in encryption during decryption. Otherwise, correctness will not hold. Compactness is maintained in the Random Oracle Model (ROM) by having $\mathsf{PreProcess}$ derive r from a short seed $\mathsf{sd} \leftarrow \{0,1\}^\lambda$ via a hash function $\mathcal{H}$; the ciphertext carries only sd. We leave it as an interesting open problem to prove security without relying on ROM.

Why releasing the sharing randomness does not hurt security. Giving out the sharing randomness r might seem catastrophic, as it reveals all shares explicitly. However, we observe that we are not trying to hide $\mathbf{p}$ itself. Rather, what the ciphertext hides is $\mathbf{s}^\top \mathbf{p}$, the LWE-masked version of $\mathbf{p}$. Our strategy uses the LSSS to create a *dual-mode LWE instance*. In a bit more detail, for every user $j \in S$, the j-th column of $\mathbf{M}$ is $\mathsf{pk}_j + \mathsf{sh}_j$, and during partial decryption one recovers approximately

$$j\text{-th user learns } \approx \mathbf{s}^\top(\mathsf{pk}_j + \mathsf{sh}_j) = \underbrace{\mathbf{s}^\top \mathsf{pk}_j}_{\text{Key Term}} + \underbrace{\mathbf{s}^\top \mathsf{sh}_j}_{\text{Shared Mask Term}}.$$

Our insight is that for *any* user, only one of these two components needs to be truly random to mask the other. In the security proof:

- For **honest users** $j \in S \setminus C$, the public key $\mathsf{pk}_j = \mathbf{B}\mathbf{r}_j$ is uniformly random (by the leftover hash lemma), so the Key Term effectively acts as the LWE challenge to completely hide the Shared Mask Term $\mathbf{s}^\top \mathsf{sh}_j$.
- For **corrupt users** $j \in C$, the adversary may control pk_j maliciously, but the access policy $\mathbb{A}$ forces the corrupted set to be unauthorized. Thus, programmability lets the reduction set $\mathsf{sh}_j \leftarrow \mathbb{Z}_q^n$ uniformly at random. In this case, the Shared Mask Term takes over as the LWE challenge to hide the Key Term.

This dual-mode argument where honest users hide their shares via their public keys and corrupt users' shares are uniform by programmability is the central technique. Formalizing it requires three steps: (1) using a programmable LSSS to reprogram shares of corrupt users without affecting the global distribution, (2) working in the ROM to retain compactness and (3) using the explainable rerandomizer (ER) framework [CHW25] to "explain" the commitment matrix $\mathbf{C}$. Step (3) is required as some users in the encryption set S maybe corrupted. Thus, we cannot exactly program $\mathbf{A} + \mathbf{C}$ in this case as some elements of $\mathbf{C}$ depend on $\mathbf{A}$. ER helps us circumvent this subtlety by letting the reduction sample $\mathbf{C}$ in a 'random' fashion.

Compactness and parameter analysis. We instantiate decomposed LWE with a sub-exponential modulus-to-noise ratio: for a fixed constant $\epsilon \in (0, 1/2)$, we assume $q/B_{\mathsf{LWE}} = 2^{\widetilde{O}(n^\epsilon)}$. Let $\rho = \log\alpha + \log\beta + \mathsf{poly}(\lambda)$. Correctness requires $\log q = \widetilde{O}(\rho)$, and hence we choose $n = \widetilde{O}(\rho^{1/\epsilon})$ and $m = \widetilde{O}(\rho^{1+1/\epsilon})$. The public key, secret key, ciphertext, and partial decryption in our DPE construction consequently have size $\widetilde{O}(\rho^{1+1/\epsilon})$, $\widetilde{O}(\rho^{5+5/\epsilon})$, $\widetilde{O}(\rho^{6+5/\epsilon})$, and $\widetilde{O}(\rho)$, respectively.

DNF formulas. For DNF formulas, our programmable LSSS has $\alpha = \beta = 1$, and hence $\rho = \mathsf{poly}(\lambda)$ is independent of $|S|$. Consequently, the ciphertext and each single-share key component are fully compact. As discussed in the formal construction, if a party receives multiple shares, its key size additionally grows with the corresponding policy fanout.

Threshold access structures. For sublinear thresholds, our scaled LSSS has $\log\alpha + \log\beta = O(t^2 \log|S|)$, and hence $\rho = O^\dagger(t^2)$. For linear-minus-sublinear thresholds, it has $\log\alpha + \log\beta = O((|S| - t)\log|S|)$, and hence $\rho = O^\dagger(|S| - t)$. Thus, writing $\tau = \min(t^2, |S| - t)$, the public key, secret key, ciphertext, and partial decryption have size $O^\dagger(\tau^{1+1/\epsilon})$, $O^\dagger(\tau^{5+5/\epsilon})$, $O^\dagger(\tau^{6+5/\epsilon})$, and $O^\dagger(\tau)$, respectively. All parameters are sublinear whenever $\tau = o(|S|^{\epsilon/(6\epsilon+5)})$. As ϵ approaches $1/2$ from below, this condition approaches $\tau = o(|S|^{1/16})$, corresponding to $t = o(|S|^{1/32})$ in the sublinear-threshold regime and $|S| - t = o(|S|^{1/16})$ in the linear-minus-sublinear regime.

With a structured common reference string and the matrix commitment from succinct LWE [Wee24, Wee25], the public and secret keys have size $\widetilde{O}(\rho^{1+1/\epsilon})$, the ciphertext has size $\widetilde{O}(\rho^{2+1/\epsilon})$, and a partial decryption has size $\widetilde{O}(\rho)$. For threshold policies, all parameters are therefore sublinear whenever $\tau = o(|S|^{\epsilon/(2\epsilon+1)})$, which approaches $\tau = o(|S|^{1/4})$ as ϵ approaches $1/2$.

On the $|S|^{1/16}$ bottleneck. There are two useful ways to understand this bottleneck. First, it is not inherent to our proof strategy or to scaled LSSS. It arises from the decomposed-LWE matrix commitment whose matrix $\mathbf{B}$ has $4m^5$ columns, and the ciphertext inherits this width. Substituting $m = \widetilde{O}(\rho^{1+1/\epsilon})$ gives the $\widetilde{O}(\rho^{6+5/\epsilon})$ ciphertext bound. The improvement from succinct LWE, whose commitment matrix has only $O(n \log q)$ columns, confirms that this loss comes from the matrix commitment rather than from the scaled-reconstruction argument.

Second, a related parameter loss is already present in the direct Shamir-based threshold-FHE paradigm of Boneh et al. [BGG+18]. Clearing the denominators of the Shamir reconstruction coefficients requires $q > (N!)^{O(1)}$, and hence $\log q = \widetilde{\Omega}(N)$. Once the LWE dimension is correctly chosen as a function of this modulus, the GSW public key and fresh ciphertext have size $\widetilde{O}(N^{2+2/\epsilon})$, which approaches an N^6 blowup as ϵ approaches $1/2$. Thus, a substantial part of the parameter growth comes from using Shamir sharing together with denominator-clearing, rather than being particular to our construction. Our scaled LSSS avoids this direct N-dependent blowup by replacing it with dependence on $\min(t^2, |S| - t)$. Nevertheless, the silent setting additionally requires a matrix commitment, whose decomposed-LWE instantiation leaves us with the $|S|^{1/16}$ barrier above.

2.3 Distributed Threshold FHE with Silent Setup

Having built DPE, we now describe how to obtain Silent Monotone Policy FHE (SFHE): a scheme that additionally supports homomorphic evaluation of arbitrary Boolean circuits $C : \{0,1\}^\ell \to \{0,1\}$ over ciphertexts followed by threshold decryption, all under silent setup.

A faulty yet natural starting point. A natural first attempt is to use hybrid encryption: the encryptor samples an FHE key pair $(\mathsf{fhe.pk}, \mathsf{fhe.sk})$, encrypts the message μ under FHE, and encrypts $\mathsf{fhe.sk}$ under the DPE scheme. This fails to achieve simulation security. Once an adversary obtains partial decryptions and reconstructs $\mathsf{fhe.sk}$, it can use this key to decrypt any FHE ciphertext, including those encrypted for other messages. The FHE ciphertext is thus completely transparent to an adversary that has obtained enough partial decryptions, breaking all meaningful security for any future ciphertext. Simulation-based security requires that the simulator can produce partial decryptions without knowing μ, which is impossible if $\mathsf{fhe.sk}$ is directly recoverable.

Our approach: algebraic fusion of GSW and DPE. Rather than using FHE and DPE as black boxes, we fuse them algebraically. Consider first the case of a single-bit message $\mu \in \{0,1\}$. The encryptor produces a GSW-style [GSW13] ciphertext matrix $\Psi \in \mathbb{Z}_q^{(n+1)\times \ell m}$, whose top n rows we denote $\overline{\Psi}$ and whose last row we denote $\underline{\Psi}$. The key structural property of GSW is that $\underline{\Psi} \approx \mathbf{s}^\top \overline{\Psi} + \mu \cdot \lfloor q/2\beta \rfloor$ for an LWE secret $\mathbf{s} \in \mathbb{Z}_q^{n}$[3]. We embed this into the DPE framework by secret-sharing the top rows $\overline{\Psi}$ across parties: compute $(\overline{\Psi}, \mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) \leftarrow \mathsf{Share}(\mathcal{A}, \overline{\Psi})$ and form the commitment matrix $\mathbf{M} = [\mathsf{pk}_i + \mathsf{sh}_i \text{ for } i \in S]$ exactly as in DPE. Each party j's partial decryption then recovers $\mathbf{s}^\top \mathsf{sh}_j + \mathsf{err}$ using the DPE mechanism. An authorized set $S' \in \mathcal{A}$ combines these to obtain $\mathbf{s}^\top(\gamma_{S'} \cdot \overline{\Psi}) + \mathsf{err}$. Since the bottom row satisfies $\underline{\Psi} \approx \mathbf{s}^\top \overline{\Psi} + \mu \cdot \lfloor q/2\beta \rfloor$, the decryptor computes $\gamma_{S'} \underline{\Psi} - \mathbf{s}^\top(\gamma_{S'} \overline{\Psi}) \approx \gamma_{S'} \mu \lfloor q/2\beta \rfloor$ and recovers μ.

Supporting homomorphic evaluation. The above handles unevaluated ciphertexts. To support evaluation of a circuit C on ℓ-bit inputs, we encode all ℓ input bits into a single GSW ciphertext matrix $\Psi = [\Psi_1 \mid \cdots \mid \Psi_\ell]$. A naive approach would be to evaluate the circuit directly on Ψ and then share the result via DPE. However, this breaks correctness. The DPE partial decryption requires the *exact* matrix $\mathbf{M}$ as computed during encryption to decrypt. So, we cannot pursue this idea as circuit evaluation on Ψ is not invertible to recover the original ciphertext back.

[3] As in the toy example (Regev), we scale the message by $\lfloor q/2\beta \rfloor$.

Our solution exploits the algebraic structure of GSW evaluation. We observe that the GSW evaluation $\mathsf{GSW.Eval}(C, \Psi_1, \ldots, \Psi_\ell)$ can be factored as $\Psi \cdot \mathbf{H}_{C,\Psi}$, where $\mathbf{H}_{C,\Psi} = \mathsf{EvalF}(C, \Psi)$ is a deterministic low-norm matrix satisfying $\|\mathbf{H}_{C,\Psi}\| \leq m^{O(d)}$ for depth-d circuits. For instance, for two GSW ciphertexts Ψ_1, Ψ_2, the multiplication evaluation matrix is $\mathbf{H}_{\times,[\Psi_1|\Psi_2]} = \begin{bmatrix} \mathbf{G}^{-1}(\Psi_2) \\ \mathbf{0} \end{bmatrix}$, and $[\Psi_1 \mid \Psi_2] \cdot \mathbf{H}_{\times,[\Psi_1|\Psi_2]}$ equals the GSW multiplication output. This factored form is the key: rather than evaluating the circuit on the ciphertext and then sharing the result, we simply forward the *original* input ciphertext Ψ (which is already shared inside DPE) together with $\mathbf{H}_{C,\Psi}$. The evaluated ciphertext is $\widetilde{\mathsf{ct}} = (\mathsf{ct}, \mathbf{H}_{C,\Psi})$, and its size depends on ℓ and d but is sub-linear in $|S|$, which is meaningful for circuits of large size.

Partial decryption of evaluated ciphertexts. During partial decryption, party j first recovers $\mathbf{s}^\top \mathsf{sh}_j + \mathsf{err}$ from DPE as before. Note that sh_j is a share of the *original* top rows $\overline{\Psi}$, not the evaluated ciphertext. Party j then multiplies this by $\mathbf{H}_{C,\Psi}$) to obtain a partial decryption of the evaluated ciphertext. An authorized set S' combines these shares to recover

$$\mathbf{s}^\top(\gamma_{S'} \cdot \overline{\Psi} \cdot H_{C,\Psi}) + \mathsf{err} = \mathbf{s}^\top(\gamma_{S'} \cdot \overline{\mathsf{GSW.Eval}(C, \Psi)}) + \mathsf{err}.$$

Since the evaluated bottom row satisfies $\underline{\mathsf{GSW.Eval}(C, \Psi)} \approx \mathbf{s}^\top \overline{\mathsf{GSW.Eval}(C, \Psi)} + C(\mu) \cdot \lfloor q/2\beta \rfloor$, the decryptor subtracts to recover $C(\mu)$.

Simulation overview. Our goal is to simulate the partial decryption share for user j given secret keys for users $i \in \mathcal{C}$, a ciphertext ct, and an evaluation circuit C. Simulating partial decryptions using Lagrange coefficients $(\lambda_k^{\mathcal{C}})$ is straightforward, as we can follow the same methodology present in previous works such as [BGG+18]. The idea is that

$$\mathbf{s}^\top \mathsf{sh}_j \cdot \mathbf{H}_{C,\Psi} = \lambda_0^{\mathcal{C}} \cdot \mathbf{s}^\top \overline{\mathsf{GSW.Eval}(C, \Psi)} + \sum_{i \in \mathcal{C}} \lambda_i^{\mathcal{C}} \cdot \mathbf{s}^\top \mathsf{sh}_i \cdot \mathbf{H}_{C,\Psi}.$$

The second term on the right-hand side is computable from the corrupt keys $\{\mathsf{sk}_i\}_{i \in \mathcal{C}}$. For the first term, we do not have $\mathbf{s}^\top \overline{\mathsf{GSW.Eval}(C, \Psi)}$ directly, but we can use $\underline{\mathsf{GSW.Eval}(C, \Psi)} - \lceil q/2\beta \rfloor C(\mu) - \widehat{e}$ for some error $\widehat{e}$. As long as we can smudge this error, the distributions of honest partial decryptions and simulated ones remain statistically close.

Using (α, β)-scaled LSSS reconstruction, this equation becomes

$$\delta_{\mathcal{C}\cup\{0\},j} \cdot \mathbf{s}^\top \mathsf{sh}_j \cdot \mathbf{H}_{C,\Psi} = \gamma_{\mathcal{C}\cup\{0\}} \cdot \mathbf{s}^\top \overline{\mathsf{GSW.Eval}(C, \Psi)} + \sum_{i \in \mathcal{C}} \delta_{\mathcal{C}\cup\{0\},i} \cdot \mathbf{s}^\top \mathsf{sh}_i \cdot \mathbf{H}_{C,\Psi}.$$

Now, each δ_i value is small (bounded by α), but δ_j^{-1} need not be. So, we will use the double smudging argument similar to the toy example presented earlier. In a bit more detail, to facilitate smudging in this regime, we introduce an additional error $\widetilde{e}$ added to the GSW ciphertext component. Specifically, our Ψ takes the form

$$\Psi = \begin{bmatrix} \mathbf{AR} \\ \mathbf{s}^\top \mathbf{AR} + \mathbf{e}^\top \mathbf{R} + \widetilde{\mathbf{e}}^\top \end{bmatrix} + \mu^\top \otimes \mathbf{G}.$$

During partial decryption simulation, we can change $\widetilde{\mathbf{e}}^\top + \mathbf{e}^\top \mathbf{R}$ to $\widetilde{\mathbf{e}}^\top + \delta_{\mathcal{C}\cup\{0\},j} \cdot \mathbf{e}^\top \mathbf{R}$, and as long as $\widetilde{\mathbf{e}}^\top$ is sampled from a sufficiently wide Gaussian distribution, these two are statistically indistinguishable by the smudging lemma. Consequently, the quantity we need to smudge becomes $\delta_j^{-1} \cdot \gamma_S \cdot (\delta_j \cdot \widehat{e}') = \gamma_S \cdot \widehat{e}'$, which is bounded by $\beta \cdot \widehat{e}'$, a quantity that the smudging noise χ' can dominate. In the actual construction, we need a few additional $\widetilde{\mathbf{e}}$ terms in other components of the ciphertext (specifically $\widetilde{\mathbf{e}}_1$ in $\mathbf{c}_1^\top$ and $\widetilde{\mathbf{e}}_2$ in $\mathbf{c}_2^\top$) to ensure that the same smudging argument applies uniformly across all ciphertext components. However, essentially the same idea is used there as well.

Finally, we need to fix one last issue. We do not know δ_j during encryption, since the honest party j who will perform partial decryption is not determined at encryption time. To resolve this, we rely on the *targeted* variant of (α, β)-Scaled LSSS. At a high level, consider the low-threshold regime where γ_S is the

LCM of the denominators of the Lagrange coefficients for the set $S = \mathcal{C} \cup \{0\}$. The crucial property is that the *same* γ_S can be used to retrieve sh_j from $\{\mathsf{sh}_0 = \text{secret}, \{\mathsf{sh}_i\}_{i\in\mathcal{C}}\}$ for *any* target $j \notin \mathcal{C}$. Therefore, we can augment the GSW ciphertext's noise by γ_S as long as we know the corrupted user set $\mathcal{C}$ selectively. This is sufficient because γ_S depends only on $\mathcal{C}$, not on j, ensuring that the simulator's ciphertext is consistent across all honest parties' partial decryptions.

2.4 Sublinear Centralized Threshold FHE from LWE

Our construction follows the threshold-FHE paradigm of [BGG+18]. A trusted dealer first runs FHE setup to obtain $(\mathsf{fhe.pk}, \mathsf{fhe.sk})$, secret-shares $\mathsf{fhe.sk}$ as $(\mathsf{sk}_1, \ldots, \mathsf{sk}_N) \leftarrow \mathsf{sh}(\mathbb{A}, \mathsf{fhe.sk})$, and distributes sk_i to party i. Encryption and homomorphic evaluation are those of the underlying FHE scheme, while partial decryption by party i applies the Decode_0 procedure to $(\mathsf{sk}_i, \widetilde{\mathsf{ct}})$. Recall that FHE decryption can be decomposed into two steps: Decode_0, which applies a linear operation in the secret key to produce a decrypted element, and Decode_1, which rounds this decrypted element to recover the message. Since Decode_0 is linear in the secret key, the decryptors can combine their partial decryptions using the scaled reconstruction coefficients of our LSSS to obtain $\gamma_{S'} \cdot \mathrm{FHE.Decode}_0(\mathsf{fhe.sk}, \widetilde{\mathsf{ct}})$, from which the message can then be recovered.

Why the scaled-LSSS plug-in is not immediate. At first sight, it seems that one can simply plug our (α, β)-Scaled LSSS into the [BGG+18] framework. The difficulty is the same one that already appeared in the silent-setting construction: the simulation of partial decryptions introduces inverse reconstruction coefficients such as δ_j^{-1}, which is large, and this destroys the usual smudging argument. As in the silent setting, we again need a double-smudging strategy: one layer modifies the initial error terms so that they remain controllable after evaluation, and the second layer smudges the resulting evaluated error terms inside the partial decryptions. The centralized setting introduces an added complication. Here the relevant LWE error is created during *setup*, when the FHE public key is generated, rather than during encryption. Consequently, the simulator cannot postpone the smudging step to encryption time, and simply storing the underlying LWE error in the simulator state would be incompatible with semantic security.

Moving the smudging noise into the public key. Our solution is to place the extra smudging directly in the public key. This lets us arrange that the public-key error is already multiplied by the appropriate reconstruction factor, so that the problematic inverse term is canceled during reconstruction. At the same time, the simulator must still keep track of the contribution of this smudging term after homomorphic evaluation, since that contribution is not a-priori bounded and must be subtracted when simulating partial decryptions. Exactly as in the silent-setting construction, the linearized GSW evaluation lets us express the accumulated smudging error after evaluation as $[\hat{\mathbf{e}}^\top \mathbf{R}_1 \mid \cdots \mid \hat{\mathbf{e}}^\top \mathbf{R}_\ell]\mathbf{H}_{C,\mathsf{ct}}$, where the matrices $\mathbf{R}_i$ are the encryption-time randomness. Thus the simulator keeps $\hat{\mathbf{e}}$ together with $(\hat{\mathbf{e}}^\top \mathbf{R}_i)_{i\in[\ell]}$ in its state. This does leak the smudging term and a bounded amount of encryption randomness, but this leakage is harmless for our proof: the state is hidden in the simulation-security game, which is the only place where the smudging argument is needed, and the residual leakage relevant to semantic security can still be handled using the same bounded-leakage ideas with the generalized leftover hash lemma.

Bootstrapping to constant ciphertext size. Applying the bootstrapping compiler of [BGG+18] reduces the evaluated ciphertext size from $\mathsf{poly}(\lambda, d)$ to $O(1) \cdot \mathsf{poly}(\lambda)$, namely, constant in the circuit depth. The compiler homomorphically evaluates the FHE decryption procedure recursively. Since it relies only on the standard correctness and security guarantees of the underlying FHE scheme, it applies to our construction without modification. The resulting scheme has constant-size ciphertexts and secret key/share size $\min(t^6, (N-t)^3) \cdot \mathsf{poly}(\log N, \lambda)$.

2.5 Applications and Related Work

As we briefly discussed in the introduction, our results and techniques pave the way for many interesting applications for both threshold and non-threshold lattice-based cryptography. As a quick illustration of applications beyond threshold cryptosystems, we briefly explain how we can design an improved Fuzzy Identity-Based Encryption (FIBE) from lattices by using our Scaled LSSS.

Compact FIBE from Lattices. In our FIBE scheme, the master public key contains LWE matrices $(\mathbf{A}_{i,b})_{i\in[\ell],b\in\{0,1\}}$ with trapdoors, and a target vector $\mathbf{p} \in \mathbb{Z}_q^n$. A secret key for identity $\mathsf{id} \in \{0,1\}^\ell$ is generated by sharing $\mathbf{p}$ across the ℓ attribute positions via a programmable (α, β)-Scaled LSSS for threshold-τ access structures, and sampling short pre-images $\mathbf{r}_i \leftarrow \mathsf{SamplePre}(\mathbf{A}_{i,\mathsf{id}_i}, \mathbf{T}_{i,\mathsf{id}_i}, \mathsf{sh}_i, \sigma)$ for each $i \in [\ell]$.

A ciphertext for identity $\mathsf{id}' \in \{0,1\}^\ell$ is simply $(\mathbf{c}_i^\top = \mathbf{s}^\top \mathbf{A}_{i,\mathsf{id}'_i} + p\mathbf{e}_i^\top)_{i\in[\ell]}$, $c' = \mathbf{s}^\top \mathbf{p} + pe' + \mu$, consisting of $O(\ell)$ LWE samples. Decryption for a key with identity id checks $S = \mathsf{id} \cap \mathsf{id}'$ and runs scaled reconstruction:

$$y = \gamma_S \cdot c' - \sum_{j\in S} \delta_{S,j} \cdot c_j^\top \mathbf{r}_j \;=\; \gamma_S \mu \pmod p,$$

recovering μ as $\gamma_S^{-1} y \bmod p$. Writing $\rho = \min(\tau^2, \ell - \tau)$, the public parameters, secret keys, and ciphertexts have size $O^\dagger(\ell\rho^6)$, $O^\dagger(\ell\rho^3)$, and $O^\dagger(\ell\rho^4)$, respectively. Thus ciphertexts and secret keys are sub-quadratic whenever $\rho = o(\ell^{1/4})$. The prior construction of [ABV+12] required parameters polynomial in ℓ^2. Attribute-based encryption for circuits [GVW13, BGG+14] generically yields FIBE with optimal parameters, but our direct algebraic construction is more efficient. Next, we explain applications to other threshold cryptosystems.

Universal Thresholdizer and more. A universal thresholdizer (UT) takes a cryptographic key and produces a number of key shares that can be used to individually evaluate a cryptographic function. Each of these individual evaluation shares can then be combined to result in the final evaluation of the function. [BGG+18] introduced UT and designed it generically from any threshold FHE system. They used UT to further improve the efficiency of TFHE as well as thresholdize many other primitves such as CCA-secure PKE, PRFs, IBE, signatures. Using our TFHE scheme, we can improve the efficiency of all these downstream applications. In particular, we can obtain: (1) TFHE with constant-size ciphertexts with $O(\min(t^6, (N-t)^3)\mathsf{poly}(\lambda))$-size keys, (2) threshold PRFs where the length of each distributed PRF key grows as $O(\min(t^6, (N-t)^3)\mathsf{poly}(\lambda))$, thus is sublinear in N, (3) threshold CCA-secure PKE where even the secret key size is now sublinear. These objects have many downstream applications [BLMR13, BGG+18], such as distributed credential protection, low-communication multiparty computation, and federated learning, whose efficiency can also be improved by plugging in our results.

In addition, our SFHE immediately gives an approach to designing Universal Thresholdizers in the *silent setup* model. Although we consider silent-setup UT as an object outside the scope of this work, we believe it could enable new approaches to ad-hoc/ silent-setup versions for many threshold primitives from lattice assumptions. We leave this as an interesting direction for future research. We remark that our SFHE directly gives the first CCA-secure threshold PKE with fully silent setup from post-quantum assumptions, where the ciphertext size does not scale with the number of servers. This follows immediately by adapting the Naor-Yung [NY90] or Fujisaki-Okamoto [FO99] transforms to our SFHE schemes as discussed in [BKW25], and does not need a silent-setup UT.

Related work. Waters and Wu [WW26] achieve silent threshold cryptography from pairing-based q-type assumptions in the plain model, supporting monotone Boolean formula policies with compact parameters. Their construction avoids SNARK machinery and achieves shorter signatures and ciphertexts than prior pairing-based work [GKPW24, GJM+24], but is inherently non-post-quantum. [DJWW26] construct distributed monotone-policy encryption for S-space read-once Turing machines using iO and somewhere-statistically-binding hash functions, giving a very expressive policy class from non-standard assumptions. Additionally, [HASW25] achieves plausibly post-quantum silent threshold encryption with a weaker computational correctness property. [HASW25] is not a standard threshold (broadcast) encryption scheme, namely, unlike the usual notion where every threshold set of parties can decrypt, non-decrypting threshold sets may exist, although such sets are computationally hard to identify given secret keys for unauthorized set of users.

[BGG+18] also described a *decentralized* TFHE construction in which parties generate keys independently and a joint public key is derived without a trusted dealer. However, their decentralized construction did not achieve compact parameters (the ciphertext and key sizes grow polynomially with the number of parties), and to the best of our knowledge, it remains the only prior construction of decentralized threshold FHE. Our silent TFHE is the first decentralized threshold FHE with short ciphertexts from any assumption.

A number of recent works have studied relaxed variants of TFHE that weaken the functionality in various ways: requiring the decryptors to know the authorized set S before partial decryption [MBH22, BFM+25], introducing a private or interactive pre-processing step on the evaluated ciphertext before partial decryption can begin [PS24], assuming synchronized or non-broadcast communication among decryptors [dVPS26, STH+23], or relying on interactive multi-round setup or decryption protocols [HLW+25, CL25]. While these works make interesting progress on efficiency and concrete security within their respective models, each restriction significantly limits the functionality and applicability of the resulting TFHE scheme compared to the original TFHE formulation of [BGG+18], which supports non-interactive one-round partial decryption, does not require knowledge of S, and works in the standard asynchronous broadcast model. Our centralized TFHE maintains all of these properties while achieving sublinear parameters for the first time.

Concurrent Work. In independent and concurrent works, Champion and Wu [CW26a] construct DPE for DNF formulas, while Champion, Wu, and Yamada [CWY26] construct silent threshold encryption for sublinear thresholds. We compare our results with these works below.
We first compare our work with [CW26a].

Main results. Like us, [CW26a] construct DPE for DNF formulas from decomposed LWE in the random oracle model. We additionally give DPE for threshold policies, SFHE for both DNF and threshold policies, sublinear centralized TFHE, and compact Fuzzy IBE, none of which appear in [CW26a].

Efficiency and security. Both works achieve similar efficiency. However, [CW26a] also shows how to achieve simulatable one round partial decryptions. Their work can also handle maliciously generated public keys during encryption. While we do not define or demonstrate our scheme with this property, it is easy to see that our construction can indeed achieve this as the secret keys for corrupted only need to be bounded and their distribution is not relevant.

Techniques. While the core lattice techniques are conceptually similar, we develop the abstraction of (α, β)-Scaled LSSS and use it as a common foundation for all our constructions.

We next compare our work with [CWY26].

Main result. [CWY26] constructs silent threshold encryption from decomposed LWE for t-out-of-N thresholds with sublinear t. Under the same assumption, our construction additionally achieves compactness for a non-trivial range of linear-minus-sublinear thresholds $t \in [N - N^{\varepsilon}, N - 1]$ for constant $\varepsilon < 1/16$, a regime not covered by [CWY26]. Moreover, we also construct other primitives such as monotone policy FHE with silent setup and threshold FHE.

Security. Their security notion is closely related to ours: the adversary declares the corrupted slots $C \subseteq [N]$ before seeing the CRS. Their definition also permits maliciously generated keys for the corrupted slots. Although we do not explicitly include this feature in our definition, our construction and proof extend to this setting. Moreover, their static-security game allows the adversary to obtain multiple honestly generated public keys and subsequently assign them to honest slots. Our construction could be extended to support the corresponding *slotted* notion of static security: the slot is fixed when a key is generated, but the adversary may obtain multiple honestly generated public keys for each fixed slot and later choose which key to use for a slot.

Techniques. We give a direct construction from (α, β)-Scaled LSSS, whereas [CWY26] proceeds by a conceptually different route. Their techniques do not appear to extend to our decentralized threshold FHE, whose security proof requires simulating partial decryptions of homomorphically evaluated ciphertexts.

Subsequent Work. In a subsequent work, [GY26b] constructs a DPE scheme for DNFs with optimal ciphertext size under a stronger notion of security than the one we consider. They further use our (α, β)-Scaled LSSS to construct silent threshold encryption with stronger security.

None of the concurrent or subsequent works seem to be relevant to partial-decryption simulation of decentralized threshold FHE and centralized TFHE constructions. Therefore, our work offers a distinct way of using (α, β)-Scaled LSSS, which we expect to be useful in broader contexts. We also show how to expand programmable access structures (Section 5.3), yielding a substantially different family of access structures for our constructions, beyond the monotone access structures considered in [CW26b, GY26b, CWY26].

3 Preliminaries

Notation. By PPT we denote probabilistic polynomial-time. We denote the security parameter by λ and the set of positive integers by $\mathbb{N}$. For any $a, b \in \{0\} \cup \mathbb{N}$, $a \leq b$, we denote by $[a, b]$, the set of all integers from a to b including a and b. In other words, $[a, b] = \{a, \ldots, b\}$. We denote by $[n] := [1, n]$. We denote by $x \leftarrow \mathcal{X}$, the process of sampling an element x from the set $\mathcal{X}$, with uniform probability. Similarly, for any PPT algorithm $\mathcal{A}$, $x \leftarrow \mathcal{A}(y)$ denotes the process of sampling x from the output distribution of $\mathcal{A}$ when run on y with randomly chosen coins. By $\mathsf{negl}(\cdot)$, we denote negligible functions. By $\mathsf{poly}(\cdot)$, we denote positive polynomials. All of our logarithms log are in base 2. We denote the power set A by 2^A. Consider two efficiently samplable probability distributions $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda\in\mathbb{N}}$ and $\mathcal{Y} = \{\mathcal{Y}_\lambda\}_{\lambda\in\mathbb{N}}$ over support Ω. We say $\mathcal{X}$ and $\mathcal{Y}$ are computationally indistinguishable if for any non-uniform PPT distinguisher $\mathcal{D} = \{\mathcal{D}_\lambda\}_{\lambda\in\mathbb{N}}$, and for large enough $\lambda \in \mathbb{N}$, $\left|\Pr\left[b \leftarrow \mathcal{D}(1^\lambda, \alpha_b) \ : \ \alpha_0 \leftarrow \mathcal{X}_\lambda, \alpha_1 \leftarrow \mathcal{Y}_\lambda, b \leftarrow \{0, 1\}\right] - 1/2\right| \leq \mathsf{negl}(\lambda)$. We write $\mathcal{X} \approx_c \mathcal{Y}$ to denote that distributions $\mathcal{X}$ and $\mathcal{Y}$ computationally indistinguishable. Similarly, by $\approx_s$, we denote statistical indistinguishability. Concretely, $\mathcal{X} \approx_s \mathcal{Y}$ if for large enough $\lambda \in \mathbb{N}$, $\mathsf{SD}(\mathcal{X}_\lambda, \mathcal{Y}_\lambda) = 1/2 \sum_{\alpha\in\Omega} \left|\Pr\left[\alpha \leftarrow \mathcal{X}_\lambda\right] - \Pr\left[\alpha \leftarrow \mathcal{Y}_\lambda\right]\right| \leq \mathsf{negl}(\lambda)$.

3.1 Lattice Preliminaries

For $q \in \mathbb{N}$, we denote the finite field with q elements as $\mathbb{Z}_q$. In this work, if q is prime, we view the field as integers from $[-\lfloor q/2 \rfloor, \lceil q/2 \rceil) \subset \mathbb{Z}$. Following popular convention, we denote column vectors by lowercase bold letters, e.g., $\mathbf{x} \in \mathbb{Z}_q^n$ and denote matrices by uppercase bold letters, e.g., $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$. We use x_i to denote the i-th element in $\mathbf{x}$ for $i \in [n]$ and $a_{i,j}$ to denote the element in the i-th row and j-th column of matrix $\mathbf{A}$ for $i \in [n], j \in [m]$. We sometimes alternately write $\mathbf{x}[i] := x_i$ and $\mathbf{A}[i, j] := a_{i,j}$. We define $\mathbf{1}_n$ to be a vector of all ones, $\mathbf{0}_n$ to be a vector of all zeros, and $\mathbb{1}_{n,j}$ to denote the n-dimensional canonical basis vector that has 1 in the j-th position, i.e., $\mathbf{1}_n = [1\ 1\ \ldots\ 1]^\top$ and $\mathbb{1}_{n,j} = [\ \ldots\ 1\ \ldots]^\top$. We frequently drop n from subscript in these vectors when dimension is clear. We denote the $n \times n$ identity matrix by $\mathbf{I}_n$ and drop the subscript when dimension is clear. For a matrix $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$, $\mathsf{vec}(\mathbf{A}) \in \mathbb{Z}_q^{nm}$ denoted the vector obtained by concatenating the columns of $\mathbf{A}$. For any vector $\mathbf{a} \in \mathbb{Z}_q^n$ or matrix $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$, we use $\|\mathbf{a}\|$ and $\|\mathbf{A}\|$ to denote the maximum of all the entries in $\mathbf{a}$ and $\mathbf{A}$ respectively. We use $\mathbf{A} \otimes \mathbf{B}$ to denote the Kronecker product between matrices $\mathbf{A}, \mathbf{B}$. The rounding function $\lceil\cdot\rfloor_p : \mathbb{Z}_q \to \mathbb{Z}_p$ is given by $\lceil x\rfloor_p = \lceil \frac{p}{q} x \rfloor \in \mathbb{Z}_p$. That is, if $x = y \cdot q/p + z$ where $z \in [-q/2p, q/2p)$, we have $\lceil x\rfloor_p = y$. The general rounding function $\lceil\cdot\rfloor$ is performed over $\mathbb{Z}_q$.

Discrete Gaussian. We write $\mathcal{D}_{\mathbb{Z},\sigma}$ to denote the discrete gaussian distribution with width parameter $\sigma > 0$. We state the following important lemma about the maximum norm of a value sampled from discrete gaussian distribution.

Lemma 3.1 ([Lyu12, MP12])**.** For any $k > 0$, $\Pr[|x| > k\sigma : x \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}] \leq 2\exp\left(\frac{-k^2}{2}\right)$.

Truncated Discrete Gaussians. The truncated discrete Gaussian distribution over $\mathbb{Z}$ with parameter σ, denoted by $\widetilde{\mathcal{D}}_{m,\mathbb{Z},\sigma}$, is same as the discrete Gaussian distribution $\mathcal{D}_\sigma$ except it outputs 0 whenever the ℓ_∞ norm exceeds $\sqrt{m}\cdot\sigma$. Note that, by definition, $\widetilde{\mathcal{D}}_{m,\mathbb{Z},\sigma}$ is $\sqrt{m}\cdot\sigma$-bounded. Also, note that $\widetilde{\mathcal{D}}_{m,\mathbb{Z},\sigma} \approx_s \mathcal{D}_{m,\mathbb{Z},\sigma}$.

Gadget Matrix. For positive integers n, m, q, and $m = \Omega(n \log q)$, we write $\mathbf{G} \in \mathbb{Z}_q^{n\times m}$ to denote the gadget matrix [MP12]. It is defined as $\mathbf{G} = (\mathbf{I}_n \otimes \mathbf{g}^\top)$, $\mathbf{g} = [1\ 2\ 2^2\ \ldots\ 2^{\lfloor \log q \rfloor}\ 0\ \ldots\ 0]^\top \in \mathbb{Z}_q^m$.

We write $\mathbf{G}^{-1}$ to denote the standard entry-wise bit decomposition. Note that $\mathbf{G}^{-1} : \mathbb{Z}_q^n \to \mathbb{Z}_q^m$ is an operation such that $\mathbf{G}\mathbf{G}^{-1}(\mathbf{a}) = \mathbf{a}$ and not inverse of $\mathbf{G}$. Output of $\mathbf{G}^{-1}$ is padded with 0s to match the dimensions. We often overload the notation and write $\mathbf{G}^{-1}(\mathbf{A})$ where $\mathbf{G}^{-1}$ is performed for each column.

Leftover Hash Lemma. We recall the (generalized) leftover hash lemma adapted from [ILL89, DRS04, GPV08, ABB10].

Lemma 3.2 (Generalized Leftover Hash Lemma)**.** Let $n, m, q \in \mathbb{N}$ and q is an odd prime.

1. $\{(\mathbf{B}, \mathbf{BR}, \mathbf{e}^\top\mathbf{R})\} \approx_s \{(\mathbf{B}, \mathbf{A}, \mathbf{e}^\top\mathbf{R})\}$ where $m = \Omega(n \log q)$, $\mathbf{B} \leftarrow \mathbb{Z}_q^{n\times m}, \mathbf{R} \leftarrow \{\pm 1\}^{m\times \ell}$(or $\mathbf{R} \leftarrow \{0,1\}^{m\times\ell}$), $\mathbf{A} \leftarrow \mathbb{Z}_q^{n\times\ell}$, $\ell = \mathsf{poly}(n)$ and $\mathbf{e}^\top \in \mathbb{Z}_q^m$ is any fixed vector.
2. $\{(\mathbf{B}, \mathbf{BR})\} \approx_s \{(\mathbf{B}, \mathbf{A})\}$ where $m = \Omega(n \log q)$, $\mathbf{B} \leftarrow \mathbb{Z}_q^{n\times m}, \mathbf{R} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times\ell}, \mathbf{A} \leftarrow \mathbb{Z}_q^{n\times\ell}$, $\ell = \mathsf{poly}(n)$ and $\sigma = \omega(\sqrt{\log m})$.

Gadget Trapdoors. In this work, we also use gadget trapdoor framework which provides a way to sample a random-looking matrix $\mathbf{A}$ along with a trapdoor $\mathbf{T_A}$ such that $\|\mathbf{T_A}\| = 1$ and $\mathbf{AT_A} = \mathbf{G}$. This framework also provides a useful property to sample short pre-images $\mathbf{x}$ for any $\mathbf{y}$ efficiently such that $\mathbf{Ax} = \mathbf{y}$. For completeness, we provide full definition in Appendix A.

Lemma 3.3 (Smudging Lemma, [AJL+12])**.** Let $B_1 = B_1(\lambda)$ and $B_2 = B_2(\lambda)$ be positive integers and let $e_1 \in [-B_1, B_1]$ be a fixed integer. Let $e_2 \leftarrow [-B_2, B_2]$ be chosen uniformly at random. Then the distribution of e_2 is statistically indistinguishable from that of $e_2 + e_1$ as long as $B_1/B_2 = \mathsf{negl}(\lambda)$.

3.2 Assumptions: LWE and ℓ-Decomposed LWE

The Learning With Errors (LWE) assumption is defined as follows.

Assumption 3.4 ([Reg05])**.** The LWE assumption with parameters (n, m, q, χ) states that

$$\left\{(\mathbf{A}, \mathbf{s}^\top\mathbf{A} + \mathbf{e}^\top)\right\} \approx_c \left\{(\mathbf{A}, \mathbf{c}^\top)\right\}$$

where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n\times m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^m, \mathbf{c} \leftarrow \mathbb{Z}_q^m$.

We provide the ℓ-decomposed LWE assumption which was originally proposed in [AMR25]. Our formulation is adapted from [Wee25, WW25].

Assumption 3.5 ([AMR25], adapted from [Wee25, WW25])**.** Let (n, m, q, χ) be LWE parameters. Then, the (ℓ, σ)-decomposed LWE assumption states that

$$\left\{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{s}^\top\mathbf{B} + \mathbf{e}^\top)\right\} \approx_c \left\{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{c}^\top)\right\}$$

where $\mathbf{W} \leftarrow \mathbb{Z}_q^{n\times\ell m}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times\ell m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{\ell^2 m}, \mathbf{c} \leftarrow \mathbb{Z}_q^{\ell^2 m}$ and $\mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G} \in \mathbb{Z}_q^{n\times\ell^2 m}$. When $\sigma = \mathsf{poly}(\lambda, \ell, m)$ we simply call this as ℓ-decomposed LWE assumption.

Leftover Hash Lemma for Structured B. We remark that $\mathbf{B}$ defined in above assumption is a structured matrix uniquely defined by $\mathbf{W}, \mathbf{D}$. Thus, it might not be immediately clear that Lemma 3.2 works with $\mathbf{B}$. We provide the following lemmas which show that leftover hash lemma holds even with respect to $\mathbf{B}$.

Lemma 3.6. Lemma 3.2, Item 1 holds with respect to $\mathbf{B}$ even given $(\mathbf{W}, \mathbf{D})$.

Proof. Full proof is provided in Appendix A. □

Trapdoor for B. We will show an alternate way to derive $(\mathbf{W}, \mathbf{D}, \mathbf{B})$ of the (ℓ, σ)-decomposed LWE assumption assumption such that there is a way to construct $\mathbf{T_B}$ such that $\mathbf{BT_B} = \mathbf{G}, \|\mathbf{T_B}\| = 1$. The procedure is described below.

1. Sample $(\mathbf{W}_1, \mathbf{T}_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q)$, sample $[\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_\ell] \leftarrow \mathbb{Z}_q^{n\times(\ell-1)m}$. Set $\mathbf{W} := [\mathbf{W}_1 \mid \mathbf{W}_2 \mid \ldots \mid \mathbf{W}_\ell]$.
2. Sample $\mathbf{D}_1 \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times m}, [\mathbf{D}_3 \mid \ldots \mid \mathbf{D}_\ell] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(\ell-2)m}$. Sample $(\mathbf{A}, \mathbf{T_A}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q)$ Sample $\mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, \mathbf{T}_{\mathbf{W}_1}, \mathbf{A}, \sigma)$. Set $\mathbf{D} = [\mathbf{D}_1 \mid \mathbf{D}_2 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_\ell]$.
3. Set $\mathbf{B} := \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G}$, $\mathbf{T_B} = \mathbb{1}_{\ell^2,2} \otimes \mathbf{T_A}$. Output $(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B})$.

It is straightforward to see that $\|\mathbf{T_B}\| = \|\mathbf{T_A}\| = 1$. Now, $\mathbf{BT_B} = [\mathbf{W}_1\mathbf{D}_1 + \mathbf{G} \mid \mathbf{W}_1\mathbf{D}_2 \mid \ldots \mid \mathbf{W}_\ell\mathbf{D}_\ell + \mathbf{G}][\mathbf{0} \mid \mathbf{T_A}^\top \mid \mathbf{0} \mid \ldots \mid \mathbf{0}]^\top = \mathbf{W}_1\mathbf{D}_2\mathbf{T_A} = \mathbf{AT_A} = \mathbf{G}$. We also remark that $(\mathbf{W}, \mathbf{D}, \mathbf{B})$ given out by the procedure is statistically close to uniformly random values. Proof is provided in Appendix A.

3.3 Matrix Commitment Scheme

We provide the definition of a matrix commitment scheme (MCOM) from decomposed LWE adapted from [Wee25, Appendix C].

Syntax. An MCOM scheme consists of the following polynomial time algorithms.

$\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma) \to (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B})$**.** The randomized alternative setup algorithm takes as input LWE parameters (n, m, q, σ) where $\sigma = \mathsf{poly}(\lambda, m)$ and outputs $\mathbf{W} \leftarrow \mathbb{Z}_q^{n \times 2m^3}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times 2m^3}$, and $\mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \mathbf{T_B}$ such that $\mathbf{B}\mathbf{T_B} = \mathbf{G}$. This is essentially the trapdoor sampling for $\mathbf{B}$ procedure from above.

$\mathsf{Setup}(1^n, 1^m, q, \sigma) \to (\mathbf{W}, \mathbf{D}, \mathbf{B})$**.** The randomized *public-coin* setup algorithm takes as input LWE parameters (n, m, q, σ) where $\sigma = \mathsf{poly}(\lambda, m)$ and outputs $\mathbf{W} \leftarrow \mathbb{Z}_q^{n \times 2m^3}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times 2m^3}$, and $\mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}$. The following algorithms take $(\mathbf{W}, \mathbf{D}, \mathbf{B})$ implicitly.

$\mathsf{Com}(\mathbf{M}) \to \mathbf{C}$**.** The deterministic commitment algorithm takes as input matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times N}$ and outputs commitment matrix $\mathbf{C} \in \mathbb{Z}_q^{n \times m}$.

$\mathsf{Verify}(1^N) \to \mathbf{V}_N$**.** The deterministic verification algorithm takes as input column length N and outputs verification matrix $\mathbf{V}_N \in \mathbb{Z}_q^{m \times N}$.

$\mathsf{Open}(\mathbf{M}) \to \mathbf{Z}$**.** The deterministic opening algorithm takes as input matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times N}$ and outputs opening matrix $\mathbf{Z} \in \mathbb{Z}_q^{4m^5 \times N}$.

Definition 3.7 (MCOM)**.** A MCOM scheme $(\mathsf{Setup}, \mathsf{Com}, \mathsf{Verify}, \mathsf{Open})$ satisfies the following properties.

Correctness. For any $n, q \in \mathbb{N}, m = \Omega(n \log q), \sigma = \mathsf{poly}(n, m), N = \mathsf{poly}(\lambda)$, any $(\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{Setup}(1^n, 1^m, q, \sigma)$, any matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times N}$, $\mathbf{C}\mathbf{V}_N = \mathbf{M} - \mathbf{B}\mathbf{Z}$ where $\mathbf{C} = \mathsf{Com}(\mathbf{M}), \mathbf{V}_N = \mathsf{Verify}(1^N), \mathbf{Z} = \mathsf{Open}(\mathbf{M})$.

Compactness. For any $n, q \in \mathbb{N}, m = \Omega(n \log q), \sigma = \mathsf{poly}(n, m), N = \mathsf{poly}(\lambda)$, any $(\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{Setup}(1^n, 1^m, q, \sigma)$, any matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times N}$, $\|\mathbf{V}_N\| \le O(\|\mathbf{D}\| m^4 \log q)$ and $\|\mathbf{Z}\| \le O(\|\mathbf{D}\| m^7 \log q \log N)$ where $\mathbf{V}_N = \mathsf{Verify}(1^N)$ and $\mathbf{Z} = \mathsf{Open}(\mathbf{M})$.

Mode Indistinguishability. For any $n, q \in \mathbb{N}, m = \Omega(n \log q), \sigma = \mathsf{poly}(n, m)$,

$$\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}) \ : \ \begin{array}{c} (\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \\ \mathsf{Setup}(1^n, 1^m, q, \sigma) \end{array} \right\} \approx_s \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}) \ : \ \begin{array}{c} (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \\ \widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma) \end{array} \right\}$$

This property follows from Lemma A.8, matrix distribution, and pre-image distribution properties of gadget trapdoors. Full proof is in Appendix A.

3.4 Explainable Re-randomizer

We provide the definition of an explainable re-randomizer (ER) scheme from [CHW25, WW25]. Essentially, an ER scheme lets us sample a *random* commitment to a $\mathbf{0} \in \mathbb{Z}_q^{n \times N}$ of the matrix commitment scheme from Section 3.3.

Syntax. An ER scheme consists of the following polynomial time algorithms.

$\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma) \to (\mathbf{W}, \mathbf{D}, \mathbf{B})$**.** Same as $\widetilde{\mathsf{Setup}}$ from Section 3.3.

$\mathsf{Setup}(1^n, 1^m, q, \sigma) \to (\mathbf{W}, \mathbf{D}, \mathbf{B})$**.** Same as Setup from Section 3.3. The following algorithms take $(\mathbf{W}, \mathbf{D}, \mathbf{B})$ implicitly.

$\mathsf{Sample}(1^\lambda, 1^N, \sigma) \to (\mathbf{C}, \mathbf{Z})$**.** The randomized sampling algorithm takes as input security parameter λ, column length N, width parameter $\sigma > 0$, and outputs matrices $\mathbf{C} \in \mathbb{Z}_q^{n \times m}, \mathbf{Z} \in \mathbb{Z}_q^{m \times N}$.

$\mathsf{Explain}(1^\lambda, 1^T, \mathbf{C}, \mathbf{Z}, \sigma) \to r'$**.** The randomized explanation algorithm takes as input $T = \mathsf{poly}(\lambda)$, matrices $\mathbf{C} \in \mathbb{Z}_q^{n \times m}, \mathbf{Z} \in \mathbb{Z}_q^{m \times N}$, width parameter $\sigma > 0$, and outputs $r' \in \{0,1\}^\rho$.

Definition 3.8 (ER)**.** An ER scheme $(\mathsf{Setup}, \mathsf{Sample}, \mathsf{Explain})$ is said to be an ER scheme if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}, N \in \mathbb{N}, (\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{Setup}(1^n, 1^m, q, \sigma)$, there exists a polynomial $\rho = \mathsf{poly}(\lambda, N)$ such that $\forall\ n \geq \lambda, m \geq 2n\lceil \log q \rceil, \sigma > 0$, $\mathbf{C}\mathbf{V}_N = -\mathbf{B}\mathbf{Z}$ where $(\mathbf{C}, \mathbf{Z}) \leftarrow \mathsf{Sample}(1^\lambda, 1^N, \sigma)$, $\mathbf{V}_N = \mathsf{MCOM.Verify}(1^N)$.

Randomness. For any $\lambda \in \mathbb{N}, N \in \mathbb{N}, (\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{Setup}(1^n, 1^m, q, \sigma)$, there exists a polynomial $\rho = \mathsf{poly}(\lambda, N)$ such that $\forall\ n \geq \lambda, m \geq 2n\lceil \log q \rceil, \|\mathbf{T}\|\, O(m^{12}N^3) < \sigma < 2^\lambda$,

$$\left\{ (\mathbf{C}, \mathbf{Z}) \ : \ \begin{array}{l} r \leftarrow \{0,1\}^\rho, \\ (\mathbf{C}, \mathbf{Z}) = \mathsf{Sample}(1^\lambda, 1^N, \sigma; r) \end{array} \right\} \approx_s \left\{ (\mathbf{C}, \mathbf{Z}) \ : \ \begin{array}{l} \mathbf{C} \leftarrow \mathbb{Z}_q^{n \times m}, \\ \mathbf{V}_N = \mathsf{MCOM.Verify}(1^N), \\ \mathbf{Z} \leftarrow \mathbf{B}_\sigma^{-1}(-\mathbf{C}\mathbf{V}_N) \end{array} \right\}$$

Explainability. There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $N \in \mathbb{N}, T = \mathsf{poly}(\lambda)$, $\|\mathbf{T}\|\, O(m^{12}N^3) < \sigma < 2^\lambda$, the following holds: $\mathsf{SD}(\mathcal{D}_0, \mathcal{D}_1) \leq \frac{1}{T} + \mathsf{negl}(\lambda)$, where $\mathcal{D}_0, \mathcal{D}_1$ are defined as:

$$\mathcal{D}_0 = \left\{ (\mathbf{C}, \mathbf{Z}, r) \ : \ \begin{array}{l} r \leftarrow \{0,1\}^\rho, \\ (\mathbf{C}, \mathbf{Z}) = \mathsf{Sample}(1^\lambda, 1^N, \sigma; r) \end{array} \right\},$$
$$\mathcal{D}_1 = \left\{ (\mathbf{C}, \mathbf{Z}, r') \ : \ \begin{array}{l} (\mathbf{C}, \mathbf{Z}) \leftarrow \mathsf{Sample}(1^\lambda, 1^N, \sigma), \\ r' \leftarrow \mathsf{Explain}(1^\lambda, 1^T, \mathbf{C}, \mathbf{Z}, \sigma) \end{array} \right\}$$

Mode Indistinguishability. Same as Definition 3.7.

3.5 Secret Sharing Schemes

Here, we recall basic notation and definitions for secret sharing. Some definitions are adapted from [LW11, BGG+18].

Access Structures. In this work, for simplicity, we identify each party P_i with its index $i \in [N]$. For a party set $S \subseteq [N]$, let $\mathbf{x}_S \in \{0,1\}^N$ be its characteristic vector, i.e., $(\mathbf{x}_S)_i = 1$ if $i \in S$ and $(\mathbf{x}_S)_i = 0$ otherwise.

Definition 3.9 (Monotone Access Structure)**.** Let $\{1, \ldots, N\}$ be a set of parties. A collection $\mathbb{A} \subseteq 2^{[N]}$ is *monotone* if for any sets B, C satisfying $B \in \mathbb{A}$ and $B \subseteq C \subseteq [N]$, we have $C \in \mathbb{A}$. A *(monotone) access structure* on $[N]$ is a monotone collection $\mathbb{A} \subseteq 2^{[N]} \setminus \emptyset$. Sets in $\mathbb{A}$ are called *valid* (or *authorized*) and sets in $2^{[N]} \setminus \mathbb{A}$ are called *invalid* (or *unauthorized*).

Definition 3.10 (Efficient Access Structure)**.** Let $[N]$ be a set of parties and let $\mathbb{A}$ be a monotone access structure on $[N]$. We say that $\mathbb{A}$ is *efficient* if there exists a polynomial-size *monotone* Boolean circuit $f_\mathbb{A} : \{0,1\}^N \to \{0,1\}$ such that for all $S \subseteq [N]$, $f_\mathbb{A}(\mathbf{x}_S) = 1$ if and only if $S \in \mathbb{A}$.

In this work, we only consider efficient access structures.

Definition 3.11 (Class of Monotone Access Structures)**.** Let $[N]$ be a set of parties. A *class* of monotone access structures is a collection $\mathbb{S} \subseteq 2^{2^{[N]}}$ of monotone access structures on $[N]$.

Definition 3.12 (Threshold Access Structures (TAS))**.** Let $[N]$ be a set of parties. For $t \in [N]$, define the threshold access structure

$$\mathsf{TAS}_t \ := \ \{S \subseteq [N] : \ |S| \geq t\}.$$

Let TAS denote the class $\{\mathsf{TAS}_t : t \in [N]\}$.

We now define secret sharing for access structures [CK93, BC02, BR07].

Syntax. A secret sharing scheme for parties $[N]$, secret space $\mathcal{K}$, and class of access structures $\mathbb{S}$ consists of polynomial-time algorithms:

$\mathsf{Share}(k, \mathbb{A}) \to (\mathbf{sh}_1, \ldots, \mathbf{sh}_N)$**.** On input a secret $k \in \mathcal{K}$ and an access structure $\mathbb{A} \in \mathbb{S}$, outputs shares $(\mathbf{sh}_1, \ldots, \mathbf{sh}_N)$.

$\mathsf{Combine}(\mathbb{A}, (\mathbf{sh}_i)_{i \in S}) \to k$**.** On input $\mathbb{A} \in \mathbb{S}$ and shares held by a set $S \subseteq [N]$, outputs a value $k \in \mathcal{K}$.

Definition 3.13 (Secret Sharing)**.** A secret sharing scheme $\mathsf{SS} = (\mathsf{Share}, \mathsf{Combine})$ for parties $[N]$, secret space $\mathcal{K}$, and class $\mathbb{S}$ satisfies:

Correctness. For every $\mathbb{A} \in \mathbb{S}$, every $S \in \mathbb{A}$, every $k \in \mathcal{K}$, and every $(\mathbf{sh}_1, \ldots, \mathbf{sh}_N) \leftarrow \mathsf{Share}(k, \mathbb{A})$, we have

$$\mathsf{Combine}\big(\mathbb{A}, (\mathbf{sh}_i)_{i \in S}\big) = k.$$

Privacy. For every $\mathbb{A} \in \mathbb{S}$, every $S \notin \mathbb{A}$, and every $k_0, k_1 \in \mathcal{K}$, if $(\mathbf{sh}_{b,1}, \ldots, \mathbf{sh}_{b,N}) \leftarrow \mathsf{Share}(k_b, \mathbb{A})$ for $b \in \{0, 1\}$, then the distributions $(\mathbf{sh}_{0,i})_{i \in S}$ and $(\mathbf{sh}_{1,i})_{i \in S}$ are identical.

4 (α, β)-Scaled Linear Secret Sharing

In this section, we introduce a new generalization that we call (α, β)-Scaled LSS schemes. A standard LSS scheme only guarantees that every authorized set can reconstruct the secret by linearly combining shares of authorized users.

Issue #1: Large reconstruction coefficients. Although LSS schemes have been extremely useful in many areas of cryptography [CGMA85, CCD88, BGW88, DF90, DSDFY94, CDM00, GPSW06, BSW07, Wat11, ABV+12, BLMR13, BGG+18, DKW21], their applicability in lattice-based cryptography has so far been limited. The core reason for this disparity is that lattice-based cryptosystems are typically "noisy", and standard LSS schemes interfere with the underlying noise management approach. In other words, standard LSS schemes have large 'reconstruction coefficients', and these coefficients when multiplied with noise terms in lattice-based systems lead to large unbounded noise, which essentially destroys the correctness (and sometimes compactness) of the resulting system.

Issue #2: Insufficiency of bit-decomposition. To overcome this fundamental issue, a standard folklore approach is to encode each party's share in the LSS after scaling by successive 'powers-of-2,' as it enables one to write multiplication of each (original) share by a (possibly) large reconstruction coefficient c as a binary sum of a subset of the powers-of-2 encoded shares. We refer the reader to [BÇM21] for further details. Unfortunately, such encodings compromise a rather consequential property of many well-known LSS schemes, namely the *uniform distribution of unauthorized shares*. Informally, this property states that the shares held by any group of unauthorized parties are essentially indistinguishable from random, not just hiding the secret.

Our proposal: relaxing reconstruction guarantee in LSS. In this work, we present a new methodology to bypass both these issues in the applications of LSS schemes in lattice-based cryptography. Our insight is that requiring reconstruction coefficients to be bounded (i.e., small values) is often stronger than necessary. Essentially, in a standard LSS scheme, reconstruction works as follows:

$$\mathsf{Combine}(\mathbb{A}, (\mathbf{sh}_i)_{i \in S}) = \sum_{i \in S} \langle \mathbf{c}_{S,i}, \mathbf{sh}_i \rangle = k,$$

where $\mathbf{c}_{S,i}$ is the reconstruction coefficient vector for the i-th share when S is the given authorized set. Our proposition is that requiring all such reconstruction coefficients, $\mathbf{c}_{S,i}$, to be "short" vectors is unnecessary. Instead, we propose a *relaxed* reconstruction property that gives a "scaled" secret. In particular,

$$\mathsf{ScaledCombine}(\mathbb{A}, (\mathbf{sh}_i)_{i\in S}) = \sum_{i\in S} \langle \boldsymbol{\delta}_{S,i}, \mathbf{sh}_i \rangle = \gamma_S k,$$

where we refer to $\boldsymbol{\delta}_{S,i}$ as the 'scaled' reconstruction coefficients and γ_S as the secret 'scaling' factor. With this relaxation, we require all scaled coefficients, $\boldsymbol{\delta}_{S,i}$, to be *short*, as well as the scaling factor, γ_S, to be a small value. This scaling relaxation provides us with more flexibility in designing LSS as we show in the next section, while still sufficing for a variety of lattice-based cryptographic applications, as we show throughout the paper.

4.1 Defining (α, β)-Scaled LSS

Syntactically, a Scaled LSS is defined just as a standard LSS, except that it consists of an additional algorithm, $\mathsf{GetCoeffs}$. This generates the scaled reconstruction coefficients and secret scaling factor. In particular, in addition to $(\mathsf{Share}, \mathsf{Combine})$, Scaled LSS has the following algorithm:

$\mathsf{GetCoeffs}(\mathbb{A}, S) \to ((\boldsymbol{\delta}_{S,i})_{i\in S}, \gamma_S)$**.** This is a deterministic algorithm that takes as input an access structure $\mathbb{A}$, and an authorized set of parties S, and outputs the reconstruction coefficients and secret scaling factor $((\boldsymbol{\delta}_{S,i})_{i\in S}, \gamma_S)$.

Next, we define the desired set of properties for a Scaled LSS scheme.

Definition 4.1 ((α, β)-Scaled LSS)**.** Let $\mathbb{A}$ denote the access structure, N denote the number of parties, and q be the modulus. We consider $\alpha(\cdot)$ and $\beta(\cdot)$ to be functions in $\mathbb{A}$, N, and q.

A Scaled LSS scheme $(\mathsf{Share}, \mathsf{Combine}, \mathsf{GetCoeffs})$ must satisfy correctness and privacy (as in Theorem 3.13), as well as the following two additional properties.

Linear Sharing. There exist integers $d = d(\mathbb{A}, N, q)$ and $L^* = L^*(\mathbb{A}, N, q)$, and an efficiently computable matrix $\mathbf{M} \in \mathbb{Z}_q^{L^* \times d}$, and a partition $T_1, \ldots, T_N$ of $[L^*]$ (where T_i is associated with party i), such that:

- The sharing algorithm outputs $\mathbf{sh}_i = (w_j)_{j\in T_i}$ where $\mathbf{w} = \mathbf{M} \cdot \mathbf{k} \in \mathbb{Z}_q^{L^*}$ and $\mathbf{k} = [k, r_2, \ldots, r_d]^\top \in \mathbb{Z}_q^d$ for uniformly random $r_2, \ldots, r_d \leftarrow \mathbb{Z}_q$.
- For every valid set $S \in \mathbb{A}$, we have $[1, 0, \ldots, 0] \in \mathrm{span}\big((\mathbf{M}[j])_{j\in T_S}\big) \subseteq \mathbb{Z}_q^d$, where $T_S = \bigcup_{i\in S} T_i$ and $\mathbf{M}[j]$ denotes the j-th row of $\mathbf{M}$.

Scaled Reconstruction. Let $\alpha = \alpha(\mathbb{A}, N, q)$ and $\beta = \beta(\mathbb{A}, N, q)$. For every $S \in \mathbb{A}$, every $k \in \mathcal{K}$, and every $(\mathbf{sh}_1, \ldots, \mathbf{sh}_N) \leftarrow \mathsf{Share}(k, \mathbb{A})$, and $((\boldsymbol{\delta}_{S,i})_{i\in S}, \gamma_S) \leftarrow \mathsf{GetCoeffs}(\mathbb{A}, S)$, the following must hold:

- *Reconstruction correctness*: $\sum_{i\in S} \langle \boldsymbol{\delta}_{S,i}, \mathbf{sh}_i \rangle = \gamma_S \cdot k$.
- (α, β)-*Bounded Coefficients*: $\forall i \in S$, $\|\boldsymbol{\delta}_{S,i}\|_\infty \le \alpha$, and $|\gamma_S| \le \beta$ s.t. $\gamma_S \neq 0$.

We note that whenever $|T_i| = 1$ (i.e., when every party gets a single share in the secret sharing scheme), we denote the shares and reconstruction coefficients with sh_i and δ_i instead of $\mathbf{sh}_i$ and $\boldsymbol{\delta}_i$ (single elements instead of vectors). Next, we recall the notation for maximally invalid and minimally valid sets, and then use it to formalize the rest of our definitions.

Definition 4.2 (Maximally Invalid and Minimally Valid Sets For LSSS)**.** Let $[N]$ be a set of parties and let $\mathbb{A}$ be a monotone access structure on $[N]$.

- A set of parties $S \subseteq [N]$ is *maximally invalid* if $S \notin \mathbb{A}$ but for every $i \in [N] \setminus S$, we have $S \cup \{i\} \in \mathbb{A}$.
- A set of parties $S \subseteq [N]$ is *minimally valid* if $S \in \mathbb{A}$ and for every $S' \subsetneq S$, we have $S' \notin \mathbb{A}$.

Fix an LSSS for $\mathbb{A}$ with the share matrix $\mathbf{M} \in \mathbb{Z}_q^{L^* \times d}$. For an index set $T \subseteq [L^*]$, we say that T is a *valid share set* if $[1, 0, \ldots, 0] \in \text{span}((\mathbf{M}[j])_{j \in T})$, and *invalid* otherwise.

- A set $T \subseteq [L^*]$ is a *maximally invalid share set* if T is invalid, but for every $i \in [L^*] \setminus T$, $T \cup \{i\}$ is valid.
- A set $T \subseteq [L^*]$ is a *minimally valid share set* if T is valid, but for every $T' \subset T$, T' is invalid.

In addition to scaled reconstruction, we define a targeted scaled reconstruction as follows:

Definition 4.3 ((α, β)-Targeted Reconstruction)**.** Let $\mathbb{A}$ denote the access structure, N denote the number of parties, and q be the modulus. We consider $\alpha(\cdot)$ and $\beta(\cdot)$ to be functions of $\mathbb{A}$, N, and q.

We say a Scaled LSS scheme satisfies (α, β)-targeted reconstruction if there exists a deterministic algorithm:

$\mathsf{GetTargetedCoeffs}(\mathbb{A}, C, j) \to ((\delta_{S,i})_{i \in C \cup \{0\}}, \gamma_S)$**.** This is a deterministic algorithm that takes as input an access structure $\mathbb{A}$, a maximally invalid set of parties C, and a target party $j \in [N] \setminus C$, and outputs the reconstruction coefficients and secret scaling factor associated with the target set $S = C \cup \{j\}$, namely $((\delta_{S,i})_{i \in C \cup \{0\}}, \gamma_S)$, where the subscript S indicates the target reconstruction instance, while the δ coefficients themselves are indexed by the values in $C \cup \{0\}$.

such that the following additional properties hold:

Targeted Scaled Reconstruction. Let $\alpha = \alpha(\mathbb{A}, N, q)$ and $\beta = \beta(\mathbb{A}, N, q)$. For every maximally invalid set C, every $j \in [N] \setminus C$, every $k \in \mathcal{K}$, every $(\mathsf{sh}_1, \ldots, \mathsf{sh}_N) \leftarrow \mathsf{Share}(k, \mathbb{A})$, and every $((\delta_{S,i})_{i \in C \cup \{0\}}, \gamma_S) \leftarrow \mathsf{GetTargetedCoeffs}(\mathbb{A}, C, j)$, the following must hold:

- *Reconstruction correctness*: $\sum_{i \in C \cup \{0\}} \delta_{S,i} \cdot \mathsf{sh}_i = \gamma_S \cdot \mathsf{sh}_j$ where we denote $\mathsf{sh}_0 = k$.
- (α, β)*-Bounded Targeted Coefficients*: $\forall i \in C \cup \{0\}$, $|\delta_{S,i}| \leq \alpha$, and $|\gamma_S| \leq \beta$ such that $\gamma_S \neq 0$.
- *Unique Target Scaling*: for any $j_0, j_1 \in [N] \setminus C$, letting $S_b = C \cup \{j_b\}$ for each $b \in \{0, 1\}$, and

$$((\delta_{S_b,i})_{i \in C \cup \{0\}}, \gamma_{S_b}) \leftarrow \mathsf{GetTargetedCoeffs}(\mathbb{A}, C, j_b),$$

it holds that $\gamma_{S_0} = \gamma_{S_1}$. Hence, in targeted reconstruction we denote the scaling coefficient by γ_C.

4.2 Programmability: Capturing Randomness of Unauthorized Shares

Additionally, we introduce a new notion of 'share programmability' in the context of LSS schemes. Briefly, it states that the following two distributions are identical: (i) a group of N shares sampled by uniformly running the Share algorithm for an arbitrary secret k, and (ii) a collection of random shares (sampled for any unauthorized set) together with all remaining shares conditioned on the fact that they jointly reconstruct k. In other words, rather than computing shares by running Share, one can indistinguishably 'program' random values as the shares for any unauthorized set and then equivocate the remaining shares for a desired secret value. As we demonstrate in this work, the notion of programmability is very useful in a variety of applications.

Defining Programmability. Let $\mathbb{A}$ denote the access structure, N denote the number of parties, and q be the modulus. For an access structure $\mathbb{A}$, let $\mathbf{M} \in \mathbb{Z}_q^{L^* \times d}$ be the associated share-generating matrix, and $T_1, \ldots, T_N$ be the partition of $[L^*]$ as per the LSSS representation for $\mathbb{A}$.
A *programmable* LSS scheme has the following additional polynomial-time algorithms:

$\mathsf{ProgShare}_1(\mathbb{A}, T, (\tilde{w}_j)_{j \in T}, j^*, \tilde{w}_{j^*}) \to (k, \mathbf{sh}_1, \ldots, \mathbf{sh}_N)$**.** The deterministic progr-ammed sharing algorithm in Mode 1 takes as input an access structure $\mathbb{A}$, a maximally invalid share set T (with respect to the LSSS for $\mathbb{A}$), a collection of programmed shares $(\tilde{w}_j)_{j \in T}$, an index $j^* \notin T$ and a value $\tilde{w}_{j^*}$. It outputs a secret $k \in \mathcal{K}$ and party shares $(\mathbf{sh}_1, \ldots, \mathbf{sh}_N)$.

$\mathsf{ProgShare}_2(\mathbb{A}, C, (\tilde{\mathbf{sh}}_i)_{i\in C}, k) \to (k, \mathbf{sh}_1, \ldots, \mathbf{sh}_N)$**.** The possibly randomized programmed sharing algorithm in Mode 2 takes as input an access structure $\mathbb{A}$, an invalid set of parties C, a collection of programmed shares $(\tilde{\mathbf{sh}}_i)_{i\in C}$, and a secret $k \in \mathcal{K}$. It outputs the same secret k and party shares $(\mathbf{sh}_1, \ldots, \mathbf{sh}_N)$.

Next, we define the desired set of properties for a *programmable* LSS scheme.

Definition 4.4 (Programmability)**.** A *programmable* LSS scheme must satisfy the following properties:

Consistency. For every valid set of parties $A \in \mathbb{A}$, every maximally invalid share set $T^* \subseteq [L^*]$ (with respect to $\mathbb{A}$), every invalid set of parties $C^* \subseteq [N]$, every $j^* \in [L^*] \setminus T^*$, every choice of inputs $\tilde{w}_j \in \mathbb{Z}_q$ for $j \in T^*$, every $\tilde{\mathbf{sh}}_i \in \mathbb{Z}_q^{|T_i|}$ for $i \in C^*$, every $\tilde{w}_{j^*} \in \mathbb{Z}_q$, and every $k \in \mathbb{Z}_q$, let

$$(k_0, \mathbf{sh}_{0,1}, \ldots, \mathbf{sh}_{0,N}) = \mathsf{ProgShare}_1(\mathbb{A}, T^*, (\tilde{w}_j)_{j\in T^*}, j^*, \tilde{w}_{j^*}),$$
$$(k_1, \mathbf{sh}_{1,1}, \ldots, \mathbf{sh}_{1,N}) \leftarrow \mathsf{ProgShare}_2(\mathbb{A}, C^*, (\tilde{\mathbf{sh}}_i)_{i\in C^*}, k).$$

Then for each $b \in \{0,1\}$, it holds that $\mathsf{Combine}\big(\mathbb{A}, (\mathbf{sh}_{b,i})_{i\in A}\big) = k_b$.

Moreover, programmed share values are consistent with output party shares: if $j \in T^* \cup \{j^*\}$ is the ℓ-th element of T_i (with a fixed ordering), then $\tilde{w}_j$ equals the ℓ-th coordinate inside $\mathbf{sh}_{0,i}$. Moreover, $k_1 = k$ and for $i \in C^*$ it holds that $\mathbf{sh}_{1,i} = \tilde{\mathbf{sh}}_i$.

Mode Indistinguishability. For every access structure $\mathbb{A}$, every maximally invalid share set $T^* \subseteq [L^*]$, every invalid set of parties $C^* \subseteq [N]$, and every $j^* \in [L^*] \setminus T^*$, the output distributions of the following two experiments are identically distributed:

- Exp_1: sample $\tilde{w}_j \leftarrow \mathbb{Z}_q$ for $j \in T^*$ and $\tilde{w}_{j^*} \leftarrow \mathbb{Z}_q$ uniformly, and output
$$(k_0, \mathbf{sh}_{0,1}, \ldots, \mathbf{sh}_{0,N}) = \mathsf{ProgShare}_1(\mathbb{A}, T^*, (\tilde{w}_j)_{j\in T^*}, j^*, \tilde{w}_{j^*}).$$
- Exp_2: sample $\tilde{\mathbf{sh}}_i \leftarrow (\mathbb{Z}_q)^{|T_i|}$ for $i \in C^*$ and $k \leftarrow \mathbb{Z}_q$ uniformly, and output
$$(k_1, \mathbf{sh}_{1,1}, \ldots, \mathbf{sh}_{1,N}) \leftarrow \mathsf{ProgShare}_2(\mathbb{A}, C^*, (\tilde{\mathbf{sh}}_i)_{i\in C^*}, k).$$

5 Constructing Programmable (α, β)-Scaled LSS Schemes

In this section, we construct programmable (α, β)-Scaled LSS schemes for threshold access structures (in two parameter regimes) and for DNF-formula access structures. We also show that these constructions are closed under recursive composition, while preserving all desired properties.

5.1 Threshold Access Structures: Scaling and Magnifying Shamir's scheme

We demonstrate that Shamir's secret sharing scheme [Sha79] can be modified to be a scaled LSS. In particular, we show two variations, where the first variation gives the desired scalability properties for sublinear thresholds (i.e., $t = o(\sqrt{N})$); while the second variant works for larger thresholds (i.e., $t = N - o(N)$). Let us start by briefly recalling Shamir's scheme.

Reviewing Shamir's scheme [Sha79]. Let N denote the number of parties, and $\mathbb{F}_q$ be a finite field of order q. For threshold access structures on $[N]$, Shamir's scheme works as follows:

$\mathsf{Share}(k, t, N)$. For a secret $k \in \mathbb{F}_q$, let p_k be a random degree-$(t-1)$ polynomial defined over $\mathbb{F}_q$ such that $p(0) = k$. Such a random polynomial is usually sampled by sampling $t-1$ random coefficients for each monomial of the polynomial p_k. Let $e_1, \ldots, e_N$ be any N distinct field elements. The i-th party's share consists of a single field element $\mathsf{sh}_i = p_k(e_i) \in \mathbb{F}_q$, that is i-th share is polynomial evaluation on e_i.

$\mathsf{Combine}(\{\mathsf{sh}_i\}_{i\in S})$. Let $S \subseteq [N]$ of size t. The reconstruction algorithm works by computing the Lagrange coefficients $\lambda_i^S \in \mathbb{F}_q$ such that $k = \sum_{i\in S} \lambda_i^S \cdot \mathsf{sh}_i$.

More generally, the reconstruction algorithm can also be extended to extrapolate any party's share given shares for any t-sized subset $T \subseteq [N] \cup \{0\}$. Here sh_0 denotes the secret k. In particular, one can more generally compute $\mathsf{sh}_j = \sum_{i\in T} \lambda_{i,j}^T \cdot \mathsf{sh}_i$ for all $j \in [N]$.

Shamir's scheme has large reconstruction coefficients. Shamir's scheme is an LSSS, where the share generating matrix $\mathbf{M}$ is a Vandermonde matrix corresponding to the set of evaluation points $\{e_1, \ldots, e_N\} \subset \mathbb{F}_q$. For any t-sized subset $S \subset [N]$, the reconstruction coefficients are $c_{S,i} = \lambda_i^S$. It is well known that these Lagrange coefficients, when represented as field elements over $\mathbb{F}_q$, could be arbitrarily large. This is because the denominators (in their rational representations) have to be inverted over $\mathbb{F}_q$, and these inverses could be in general large elements.

Prior works, such as [ABV+12, BLMR13, BGG+18], get around this issue by *"clearing denominators"* and turning the fractional Lagrange coefficients into bounded integral coefficients. However, the resulting field size q must grow with at least $(N!)$. While this trick of clearing denominators suffices for some applications, it kills the succinctness in threshold cryptosystems. In particular, the constraint $q > N!$ requires $\log q > N \log(N)$. Therefore, the size of each field element grows with the number of parties N.

Next, we show how to obtain a scalable LSSS via Shamir's scheme across two distinct parameter regimes.

5.1.1 Regime I: Sublinear Thresholds ($t = o(\sqrt{N})$)

In the setting, where threshold t is sublinear in the number of parties, N, we show that Shamir's scheme already satisfies our new notion of scalability. Our core observation is that, for $t = o(\sqrt{N})$, the classic 'clearing denominators' trick was already sufficient. Essentially, we only needed to do the analysis more carefully after relaxing the reconstruction guarantee.

In this section, we give an approach for Scalable LSSS for sublinear thresholds. Namely, we prove Shamir's scheme to be an $(N^{t^2/2}, N^{t^2/2})$-Scaled LSS scheme. In the context of lattice-based threshold cryptography, this offers non-trivial efficiency as long as $t = o(\sqrt{N})$ (i.e., t is sublinear in square-root of N).

Let N denote the number of parties and t denote the threshold. For a field $\mathbb{F}_q$, recall that each reconstruction coefficient in Shamir's scheme is an (appropriate) Lagrange coefficient. Observe that, for threshold t, each Lagrange coefficient is computed as a product of $2t$ field elements. Moreover, each field element in this product is of the form e_i or $(e_i - e_j)^{-1}$ for some i and j, where all algebraic computations are performed over $\mathbb{F}_q$. (Recall that $\{e_1, \ldots, e_N\} \subset \mathbb{F}_q$ is the set of evaluation points.)

Although it is known that Lagrange coefficients can be arbitrarily large, even for sublinear thresholds, our intuition is that if we select the evaluation points to be small field elements *as well as* scale all reconstruction coefficients by the lowest common multiple (LCM) field element, then we could ensure that the "scaled" coefficients are themselves bounded values. Let us elaborate. For any t-sized subset $S \subset [N]$, the Lagrange coefficients are

$$\lambda_{i,0}^S = \prod_{j\in S\setminus\{i\}} \frac{-e_j}{e_i - e_j} = \frac{n_i}{d_i}, \text{ where } n_i = \prod_{j\in S\setminus\{i\}} (-e_j), \quad d_i = \prod_{j\in S\setminus\{i\}} (e_i - e_j).$$

First, we set $\{e_1, \ldots, e_N\} := [N]$; that is, the N smallest field elements. Given this, we can compute the following terms:

$$D = \mathsf{LCM}\big(d_i : i \in S\big), \qquad \gamma_S = D, \qquad \left\{\delta_{S,i} = D \cdot \lambda_{i,0}^S = D \cdot \frac{n_i}{d_i}\right\}_{i\in S}$$

Given the above, our coefficient-generation procedure $\mathsf{GetCoeffs}$ for Shamir's scheme works as follows.

— $\mathsf{GetCoeffs}(N, t, S)$: Output $\big((\delta_{S,i})_{i\in S}, \gamma_S\big)$ computed as above. (W.l.o.g. we assume $|S| = t$.)

Clearly, this still satisfies the scaled correctness property. This can be verified as follows:

$$\sum_{i\in S} \delta_{S,i} \cdot \mathsf{sh}_i = \sum_{i\in S} D \cdot \lambda^S_{i,0} \cdot \mathsf{sh}_i = \sum_{i\in S} D \cdot \lambda^S_{i,0} \cdot p_k(e_i) = \gamma_S \cdot k.$$

Next, we need to show that the 'scaled' coefficients are indeed bounded. Since each $e_i \in [N]$ and $|S| = t$, we have

$$|n_i| = \prod_{j\in S\setminus\{i\}} |e_j| \le N^{t-1}, \qquad |d_i| = \prod_{j\in S\setminus\{i\}} |e_i - e_j| \le N^{t-1}.$$

Let $V := \{e_i - e_j\}_{i\neq j\in S}$. Each d_i is a product of exactly t elements from this set V. Clearly, the maximum number of distinct elements in V is $\binom{t}{2}$, and each element will also be small. Thus, it follows that

$$|\gamma_S| = |D| \le N^{\binom{t}{2}} < N^{t^2/2}.$$

Next, we derive the bound on the scaled reconstruction coefficients $\delta_{S,i}$. Recall that each $\delta_{S,i}$ is set as $D\cdot\lambda^S_{i,0} = D\cdot\dfrac{n_i}{d_i}$. Note D/d_i is a product of all pairwise differences among the $t-1$ points in $S\setminus\{i\}$, and hence is a product of $\binom{t-1}{2}$ terms, each of absolute value at most N. Thus, we have $|\delta_{S,i}| \le N^{t-1}\cdot N^{\binom{t-1}{2}} < N^{t^2/2}$. Combining the above, we write the following.

Theorem 5.1. For any number of parties $N \in \mathbb{N}$, and threshold $t \le N$, Shamir's secret sharing scheme (with modifications as described above) is an $(N^{t^2/2}, N^{t^2/2})$-Scaled LSS for sublinear thresholds as per Definition 4.1.

Essentially, our observation is that prior works [ABV+12, BLMR13, BGG+18] relied on *uniform worst-case* scaling of Lagrange coefficients; while as we show in this work, a *reconstruction-set-dependent* scaling can significantly reduce the scaling factor, thereby leading to exponentially better parameters ($N!$ gets reduced to N^{t^2}), while still being sufficient for numerous applications.

In addition to the above, we prove that the above LSSS also satisfies the *targeted* reconstruction property. As we explain later in the paper, targeted reconstruction is extremely useful when trying to prove simulatability of partial decryption shares across our different threshold encryption systems. Formally, we prove that:

Theorem 5.2. For any number of parties $N \in \mathbb{N}$, and threshold $t \le N$, Shamir's secret sharing scheme (with modifications as described above) satisfies $(N^{t^2/2}, N^{t^2/2})$-*targeted* reconstruction property as per Definition 4.3.

Proof. Fix any maximally invalid set $C \subseteq [N]$. Let $S = C\cup\{0\}$, where, as before, we identify the secret with the value at point $e_0 = 0$ and write $\mathsf{sh}_0 = k$. Fix any target index $j \in [N]\setminus C$. Since $|S| = t$, via Langrange's interpolation, we get coefficients $\lambda^S_{i,j} \in \mathbb{F}_q$ such that $\mathsf{sh}_j = \sum_{i\in S} \lambda^S_{i,j}\cdot\mathsf{sh}_i$. Here each $\lambda^S_{i,j}$ can we expanded as:

$$\lambda^S_{i,j} = \prod_{\ell\in S\setminus\{i\}} \frac{e_j - e_\ell}{e_i - e_\ell} = \frac{n_{i,j}}{d_i}, \text{ where } n_{i,j} = \prod_{\ell\in S\setminus\{i\}} (e_j - e_\ell), \quad d_i = \prod_{\ell\in S\setminus\{i\}} (e_i - e_\ell).$$

As earlier, we can define the following to be the *targeted* coefficient-generation procedure $\mathsf{GetTargetedCoeffs}$:

— $\mathsf{GetTargetedCoeffs}(N, t, C, j)$: Output $\big((\delta_{T,i})_{i\in S}, \gamma_C\big)$, where $T = C\cup\{j\}$, $\gamma_C = D_C = \mathsf{LCM}\big(d_i : i \in S\big)$ and $\delta_{T,i} = D_C \cdot \frac{n_{i,j}}{d_i}$ for all $i \in S$.

As for the standard reconstruction, correctness and bounded *targeted* coefficient properties follow immediately. Moreover, this also satisfies the *unique target scaling* property. This is due to the fact that $S = C\cup\{0\}$, and thus is fixed and only depends on C but not the target index j. Therefore, $\mathsf{GetTargetedCoeffs}$ produces the same scaling term γ, even when the target j is changed, as long as C is fixed. □

Later, we also prove that the above LSSS also satisfies programmability (Definition 4.4). Since the proof of programmability is largely similar for all thresholds (sublinear as well as linear-minus-sublinear thresholds), thus we provide it in Section 5.1.3 after we finish describing linear-minus-sublinear threshold.

5.1.2 Regime II: Linear-minus-Sublinear Thresholds ($t = N - o(N)$)

Next, we move to the 'linear-minus-sublinear' threshold regime, i.e. $t = N - o(N)$. In words, the threshold is set to be sublinearly close to the full set size N. A natural thought would be to try the same approach as we did in Regime I, which is scale by LCM of the denominators of the Lagrange coefficients. While this would ensure that each field element (multiplied to get the full scaled coefficient) is bounded, the total number of such elements in each coefficient would be $2t$ which is greater than N itself for such large thresholds. Thus, to handle this regime, we need a different approach.

Our idea is to "magnify" all secret shares themselves. Earlier, the shares in our scaled LSSS for sublinear thresholds were identical to the Shamir LSSS shares, and the only difference was that each reconstruction coefficient was appropriately scaled. Now we instead change the distribution of each individual share itself while secret sharing for the linear-minus-sublinear regime. In particular, let $\Lambda_i = \lambda_{i,0}^{[N]}$ be the Lagrange coefficient for the i-th share when reconstructing from the full set $[N]$. Our observation is that for any t-sized set $S \subset [N]$, the reconstruction coefficient $\lambda_{i,0}^{S}$ can be alternatively written as $\lambda_{i,0}^{S} = \Lambda_i \cdot \overline{\lambda}_{i,0}^{S}$, where $\overline{\lambda}_{i,0}^{S}$ contains the inverses of all additional elements multiplied in Λ_i. In other words, the numerator of $\overline{\lambda}_{i,0}^{S}$ contains the product of all *inverses* of e_j for all $j \in [N] \setminus S$, while its denominator contains the product of all *inverses* of $(e_i - e_j)$ for all $j \in [N] \setminus S$.

The special property implied the rewriting of the Lagrange coefficients above is that the maximum number of distinct field elements multiplied in each $\overline{\lambda}_{i,0}^{S}$ is at most $2 \cdot (N-t) = o(N)$, and Λ_i is fixed for each i. Building on above, our solution to 'magnify' Shamir shares is to multiply each share by Λ_i, therefore during reconstruction one only needs to derive the *complementary* Lagrange coefficient $\overline{\lambda}_{i,0}^{S}$ for each i. Since $\overline{\lambda}_{i,0}^{S}$ can be scaled analogously to our sublinear threshold LSS construction, we can prove the resulting LSSS to be a (N^t, N^t)-Scaled LSSS. We provide the full construction below.

$\mathsf{Share}(N, t, k)$. Let p_k be a random degree-$(t-1)$ polynomial defined over $\mathbb{F}_q$ such that $p(0) = k$. Let $e_1, \ldots, e_N$ be the N smallest distinct field elements, i.e. $e_i = i$. The i-th party's share consists of a single field element $\mathsf{sh}_i = \Lambda_i \cdot p_k(e_i) \in \mathbb{F}_q$, that is i-th share is polynomial evaluation on e_i scaled by factor Λ_i. For each $i \in [N]$, the term Λ_i is computed as $\Lambda_i = \prod_{j \in [N] \setminus \{i\}} \frac{-e_j}{e_i - e_j}$.

$\mathsf{GetCoeffs}(N, t, S)$. For set S and any $i \in S$, let the 'complementary' Lagrange coefficient $\overline{\lambda}_{i,0}^{S}$ be as follows:

$$\overline{\lambda}_{i,0}^{S} = \prod_{j \in [N] \setminus S} \frac{e_i - e_j}{-e_j} = \frac{d_i}{n_i}, \text{ where } N = \prod_{j \in [N] \setminus S} (-e_j), \quad d_i = \prod_{j \in [N] \setminus S} (e_i - e_j).$$

Output the scaled reconstruction coefficients as $\gamma_S = N, \left\{\delta_{S,i} = N \cdot \overline{\lambda}_{i,0}^{S} = d_i\right\}_{i \in S}$.

Next, we prove the following for the above LSSS.

Theorem 5.3. For any number of parties $N \in \mathbb{N}$, and threshold $t \leq N$, Shamir's secret sharing scheme (with modifications as described above) is an (N^{N-t}, N^{N-t})-Scaled LSS for linear-minus-sublinear thresholds as per Definition 4.1.

Proof. Clearly, the above LSS scheme still satisfies the scaled correctness property. This can be verified as follows:

$$\sum_{i \in S} \delta_{S,i} \cdot \mathsf{sh}_i = \sum_{i \in S} N \cdot \overline{\lambda}_{i,0}^{S} \cdot \mathsf{sh}_i = \sum_{i \in S} N \cdot \overline{\lambda}_{i,0}^{S} \cdot \Lambda_i \cdot p_k(e_i) = \sum_{i \in S} N \cdot \lambda_{i,0}^{S} \cdot p_k(e_i) = \gamma_S \cdot k.$$

Next, we need to show that the 'scaled' coefficients are indeed bounded. Since each $e_i \in [N]$ and $|S| = N - t$, we have

$$|N| = \prod_{j \in [N] \setminus S} |e_j| \leq N^{N-t}, \qquad |d_i| = \prod_{j \in [N] \setminus S} |e_i - e_j| \leq N^{N-t}.$$

Since $\gamma_S = N$ and $\delta_{S,i} = d_i$, thus we immediately get that $|\gamma| \leq N^{N-t}$ and $|\delta_{S,i}| \leq N^{N-t}$ for all $i \in S$. □

We next prove that the magnified Shamir is also programmable. Since magnification is a fixed public invertible transformation applied coordinate-wise to the ordinary Shamir shares, programmability is inherited directly from the ordinary Shamir construction.

5.1.3 Programmability of Shamir-style LSSS

Matrix view. Shamir is an LSSS with $d = t$, $L^* = N$, and $T_i = \{i\}$ for all $i \in [N]$, where the share matrix $\mathbf{M} \in \mathbb{F}_q^{N \times t}$ is the Vandermonde matrix

$$\mathbf{M}[i] = \left[1, e_i, e_i^2, \ldots, e_i^{t-1}\right] \in \mathbb{F}_q^t,$$

and $\mathbf{k} = [k, a_1, \ldots, a_{t-1}]^\top \in \mathbb{F}_q^t$, so that $\mathbf{w} = \mathbf{M}\mathbf{k}$ and $w_i = \langle \mathbf{M}[i], \mathbf{k} \rangle$. For threshold policies, invalid sets are exactly those of size at most $t-1$.

Fact 5.4 (Vandermonde Independence)**.** Let $U \subseteq [N]$ with $|U| \le t$. Then the rows $(\mathbf{M}[i])_{i \in U}$ are linearly independent, that is, $\mathsf{rank}((\mathbf{M}[i])_{i \in U}) = |U|$.

Theorem 5.5 (Programmability of Shamir)**.** For every threshold access structure TAS_t and prime $q > N$, Shamir secret sharing instantiated at points $e_i = i$ is programmable in the sense of Definition 4.4.

Proof. We instantiate the programmed sharing algorithms by interpolating the unique degree-$(t-1)$ polynomial consistent with the programmed constraints.

Mode 1 ($\mathsf{ProgShare}_1$). On input $(\mathsf{TAS}_t, T, (\tilde{w}_i)_{i \in T}, i^*, \tilde{w}_{i^*})$ where $|T| = t-1$ and $i^* \notin T$:

1. Let $U = T \cup \{i^*\}$, so $|U| = t$. Interpolate the unique polynomial $p_U(x) \in \mathbb{F}_q[x]$ of degree at most $t-1$ satisfying $p_U(e_i) = \tilde{w}_i$ for all $i \in U$.
2. Set $k = p_U(0)$ and define $w_j = p_U(e_j)$ for all $j \in [N]$ and output $(k, \mathsf{sh}_1, \ldots, \mathsf{sh}_N)$ where $\mathsf{sh}_j = w_j$.

Mode 2 ($\mathsf{ProgShare}_2$). On input $(\mathsf{TAS}_t, C, (\tilde{\mathsf{sh}}_i)_{i \in C}, k)$ where $C \subseteq [N]$ is an invalid set of parties:

1. Let $\mathcal{P}_{C,k} = \left\{ p(x) \in \mathbb{F}_q[x] : \deg(p) \le t-1,\ p(0) = k,\ \text{and } p(e_i) = \tilde{\mathsf{sh}}_i \text{ for all } i \in C \right\}$.
   Since $|C| \le t-1$, the interpolation constraints are consistent and $\mathcal{P}_{C,k}$ is nonempty.
2. Sample $p(x)$ uniformly from $\mathcal{P}_{C,k}$, for every $j \in [N]$, set $\mathsf{sh}_j = p(e_j)$, and output $(k, \mathsf{sh}_1, \ldots, \mathsf{sh}_N)$.

We now verify Definition 4.4. Consistency holds in both modes because the output shares are evaluations of a polynomial of degree at most $t-1$. In Mode 1, the output secret is $k_0 = p(0)$, while in Mode 2 the sampled polynomial satisfies $p(0) = k$, so the output secret is $k_1 = k$. Therefore, every valid set $A \in \mathsf{TAS}_t$ with $|A| \ge t$ reconstructs the corresponding output secret by Lagrange interpolation, and thus $\mathsf{Combine}(\mathsf{TAS}_t, (\mathsf{sh}_i)_{i \in A}) = k_b$ for each $b \in \{0, 1\}$. Moreover, the programmed values are preserved by construction: in Mode 1, $p(e_i) = \tilde{w}_i$ for all $i \in T \cup \{i^*\}$, and in Mode 2, $p(e_i) = \tilde{\mathsf{sh}}_i$ for all $i \in C$.

It remains to prove mode indistinguishability. The key point is that in both modes the output is ultimately determined by a uniformly random degree-$(t-1)$ polynomial, and hence by a uniformly random coefficient vector in $\mathbb{F}_q^t$.

Fix any maximally invalid share set $T^* \subseteq [N]$, any invalid set of parties $C^* \subseteq [N]$, and any $i^* \in [N] \setminus T^*$. In Exp_1, the programmed vector $(\tilde{w}_i)_{i \in T^* \cup \{i^*\}}$ is uniform in $\mathbb{F}_q^t$. The linear map that sends the coefficient vector $\mathbf{k} = [k, a_1, \ldots, a_{t-1}]^\top \in \mathbb{F}_q^t$ of the polynomial $p(x) = k + a_1 x + \cdots + a_{t-1} x^{t-1}$ to the evaluation vector $(p(e_i))_{i \in T^* \cup \{i^*\}}$ is represented by a $t \times t$ Vandermonde matrix on t distinct points, and is therefore invertible. Hence the resulting coefficient vector $\mathbf{k}$ is uniform in $\mathbb{F}_q^t$. It follows that the output of Exp_1 is distributed exactly as a standard Shamir sharing with uniform coefficient vector $\mathbf{k} \leftarrow \mathbb{F}_q^t$.

In Exp_2, let $m = |C^*| + 1 \le t$, and choose any set $D \subseteq [N] \setminus C^*$ of size $t - m$. Then $\{0\} \cup C^* \cup D$ has size t, and a polynomial of degree at most $t-1$ is uniquely determined by its values on these t points. In Exp_2, we first sample $k \leftarrow \mathbb{F}_q$ and $\tilde{\mathsf{sh}}_i \leftarrow \mathbb{F}_q$ for $i \in C^*$ uniformly. After fixing these values, the polynomial is still

unconstrained on the points in D, so we may equivalently sample $\tilde{\mathsf{sh}}_j \leftarrow \mathbb{F}_q$ for $j \in D$ uniformly as well, and then interpolate the unique polynomial $p(x)$ of degree at most $t-1$ satisfying

$$p(0) = k, \qquad p(e_i) = \tilde{\mathsf{sh}}_i \text{ for } i \in C^*, \qquad p(e_j) = \tilde{\mathsf{sh}}_j \text{ for } j \in D.$$

Since these t values are jointly uniform in $\mathbb{F}_q^t$, the resulting polynomial is uniform over all degree-($\leq t-1$) polynomials. Therefore the corresponding coefficient vector $\mathbf{k} \in \mathbb{F}_q^t$ is uniform, and the output of Exp_2 is distributed exactly as a standard Shamir sharing with uniform coefficient vector $\mathbf{k} \leftarrow \mathbb{F}_q^t$.

Hence the joint output distributions in Exp_1 and Exp_2 are identical. □

Theorem 5.6 (Programmability of Magnified Shamir)**.** For every threshold access structure TAS_t and prime $q > N$, the magnified Shamir secret sharing scheme is programmable in the sense of Definition 4.4.

Proof. Let Λ_i be the public nonzero magnification factor of party i. To run $\mathsf{ProgShare}_1$ or $\mathsf{ProgShare}_2$ for magnified Shamir, first divide each programmed share value by the corresponding Λ_i, invoke the corresponding programmed sharing algorithm for ordinary Shamir from Theorem 5.5, and then multiply every output share by the corresponding Λ_i. Since each Λ_i is public and nonzero, this transformation is efficiently invertible and preserves the programmed coordinates and correctness. Moreover, applying a fixed bijection to the shares preserves the output distribution in each mode, so mode indistinguishability follows from Theorem 5.5. □

By above theorems, we obtain the following corollaries.

Corollary 5.7. For any number of parties $N \in \mathbb{N}$, and threshold $t \leq N$, Shamir's secret sharing scheme (with modifications as described in Section 5.1.1) is a programmable (α, β)-Scaled LSS scheme with $\alpha = \beta = N^{t^2/2}$.

Corollary 5.8. For any number of parties $N \in \mathbb{N}$, and threshold $t \leq N$, Shamir's secret sharing scheme (with modifications as described in Section 5.1.2) is a programmable (α, β)-Scaled LSS scheme with $\alpha = \beta = N^{N-t}$.

5.2 DNF Formula Access Structures

We now turn to DNF formula access structures. For these policies, we use the standard Lewko–Waters [BL90, LW11] LSSS representation. The scalability part is immediate from a special structural feature of this representation: every authorized set admits reconstruction coefficients in $\{0, 1\}$. Thus, unlike the threshold constructions above, there is no need to introduce a nontrivial scaling factor in order to keep the coefficients bounded. The reconstruction rule is already as simple as possible, and therefore the resulting scaled reconstruction parameters are trivially optimal.

The more interesting point in this setting is programmability. For that, we use a linear-independence property of unauthorized share sets. Informally, the idea is that the Lewko–Waters representation for DNF formulas behaves in a particularly clean linear-algebraic way on unauthorized sets, and this is exactly what allows one to solve for a sharing vector consistent with programmed values while preserving the correct global distribution. We first record the easy scalability statement and then turn to the programmability argument.

Theorem 5.9 (Lewko–Waters DNF Is a $(1, 1)$-Scaled LSS)**.** Let $\mathbb{A}$ be a DNF formula access structure, and let $\mathsf{SS} = (\mathsf{Share}, \mathsf{Combine})$ be the corresponding Lewko–Waters LSSS. Then SS satisfies Definition 4.1 with $\alpha = \beta = 1$.

Proof. For the Lewko–Waters representation of a DNF formula, every valid set $S \in \mathbb{A}$ admits recovery coefficients in $\{0, 1\}$. Equivalently, for every valid set S, there exists a subset $T \subseteq T_S = \bigcup_{i \in S} T_i$ such that

$$k = \sum_{j \in T} w_j,$$

where $\mathbf{w} = \mathbf{M} \cdot [k, r_2, \ldots, r_d]^\top$ is the share vector output by $\mathsf{Share}(k, \mathbb{A})$. For each $i \in S$, define $\boldsymbol{\delta}_i$ coordinatewise as follows: the ℓ-th coordinate of $\boldsymbol{\delta}_i$ is 1 if the ℓ-th element of T_i belongs to T, and is 0 otherwise. Let $\gamma = 1$. Then

$$\sum_{i \in S} \langle \boldsymbol{\delta}_i, \mathbf{sh}_i \rangle = \sum_{j \in T} w_j = k = \gamma \cdot k.$$

Moreover, every entry of every $\boldsymbol{\delta}_i$ is in $\{0,1\}$, so $\|\boldsymbol{\delta}_i\|_\infty \le 1$ for all $i \in S$, and also $|\gamma| = 1$. Hence the Lewko–Waters LSSS satisfies Definition 4.1 with $\alpha = \beta = 1$. □

To prove programmability, we isolate the linear-independence property that we need.

Definition 5.10 (LI-Unauthorized Property)**.** Let SS be an LSSS for an access structure $\mathbb{A}$ with share matrix $\mathbf{M} \in \mathbb{F}_q^{L^* \times d}$ and partition $T_1, \dots, T_N$ of $[L^*]$. Write $\mathbb{1}_1 = [1, 0, \dots, 0] \in \mathbb{F}_q^d$. We say that $(\mathbf{M}, (T_i)_i)$ has the *LI-unauthorized* property if for every invalid share set $T \subseteq [L^*]$, that is, every T such that $\mathbb{1}_1 \notin \mathrm{span}((\mathbf{M}[j])_{j\in T})$, the rows $(\mathbf{M}[j])_{j\in T}$ are linearly independent over $\mathbb{F}_q$.

The LI-unauthorized property does not hold for arbitrary monotone formulas, but it does hold for DNF policies under the Lewko–Waters representation. In particular, [DKW21] notes that for DNF formulas the Lewko–Waters construction yields linearly independent rows for unauthorized sets, and also uses the observation, attributed there to [ABN+22], that one can ensure this property by computing the DNF monotone LSSS through the Lewko–Waters transformation. We next record two linear-algebraic consequences of this property: the first handles maximally invalid share sets and is used in Mode 1, while the second handles arbitrary invalid party sets and is used in Mode 2.

Lemma 5.11 (Maximally Invalid Share Sets Have Rank $d-1$)**.** Assume $(\mathbf{M}, (T_i)_i)$ satisfies Definition 5.10 and that $\mathsf{rank}(\mathbf{M}) = d$. Let $T \subseteq [L^*]$ be a maximally invalid share set. Then:

1. $\mathsf{rank}((\mathbf{M}[j])_{j\in T}) = |T| = d - 1$.
2. For every $j^* \in [L^*] \setminus T$, the $d \times d$ matrix $\mathbf{A}_{T,j^*} = [\mathbf{M}[j_1]^\top, \dots, \mathbf{M}[j_{d-1}]^\top, \mathbf{M}[j^*]^\top]^\top$. whose rows are indexed by T in any fixed order, followed by j^*, is invertible over $\mathbb{F}_q$.
3. The $d \times d$ matrix $\mathbf{B}_T$ obtained by stacking the rows indexed by T in the same fixed order and then appending $\mathbb{1}_1$ as the last row is invertible.

Proof. Since T is invalid, $\mathbb{1}_1 \notin \mathrm{span}((\mathbf{M}[j])_{j\in T})$. By the LI-unauthorized property, the rows indexed by T are linearly independent, hence $\mathsf{rank}((\mathbf{M}[j])_{j\in T}) = |T|$. Maximality means that for every $j^* \notin T$, the set $T \cup \{j^*\}$ is valid, that is, $\mathbb{1}_1 \in \mathrm{span}((\mathbf{M}[j])_{j\in T\cup\{j^*\}})$. In particular, j^* cannot lie in the span of the rows indexed by T, since otherwise validity would already hold for T. Therefore adding any $j^* \notin T$ increases the rank by 1.

We now show that $\mathsf{rank}((\mathbf{M}[j])_{j\in T}) = d - 1$. Otherwise, if $\mathsf{rank}((\mathbf{M}[j])_{j\in T}) \le d - 2$, then

$$\dim\left(\mathrm{span}((\mathbf{M}[j])_{j\in T}, \mathbb{1}_1)\right) \le d - 1.$$

Maximality implies that for every $j^* \notin T$ we have $\mathbb{1}_1 \in \mathrm{span}((\mathbf{M}[j])_{j\in T\cup\{j^*\}})$, and therefore every row $\mathbf{M}[j^*]$ lies in $\mathrm{span}((\mathbf{M}[j])_{j\in T}, \mathbb{1}_1)$. Thus every row of $\mathbf{M}$ lies in a subspace of dimension at most $d-1$, contradicting $\mathsf{rank}(\mathbf{M}) = d$. Hence $\mathsf{rank}((\mathbf{M}[j])_{j\in T}) = d - 1$. Since the rows indexed by T are linearly independent, it follows that $|T| = d - 1$, which proves the first item. The second and third items now follow immediately: appending either j^* or $\mathbb{1}_1$ raises the rank from $d-1$ to d, so the corresponding matrices are invertible. □

Lemma 5.12 (Invalid Party Sets Can Be Extended to a Basis)**.** Assume $(\mathbf{M}, (T_i)_i)$ satisfies Definition 5.10 and that $\mathsf{rank}(\mathbf{M}) = d$. Let $C \subseteq [N]$ be an invalid set of parties, and define $T_C = \bigcup_{i\in C} T_i$. Then:

1. T_C is an invalid share set.
2. The rows $(\mathbf{M}[j])_{j\in T_C}$ together with $\mathbb{1}_1$ are linearly independent.
3. Consequently, these vectors can be efficiently extended to a basis of $\mathbb{F}_q^d$. Equivalently, there exist vectors $\mathbf{u}_1, \dots, \mathbf{u}_r \in \mathbb{F}_q^d$, where $r = d - (|T_C| + 1)$, such that the matrix

$$\mathbf{B}_C = [\mathbf{M}[j_1]^\top, \dots, \mathbf{M}[j_{|T_C|}]^\top, \mathbb{1}_1^\top, \mathbf{u}_1^\top \dots, \mathbf{u}_r^\top]^\top \in \mathbb{F}_q^{d\times d}$$

is invertible, where $T_C = \{j_1, \dots, j_{|T_C|}\}$ is written in any fixed order.

Proof. Since C is an invalid set of parties, the shares held by C do not reconstruct the secret. Equivalently, the corresponding share set $T_C = \bigcup_{i \in C} T_i$ is invalid, which proves the first item.

By Definition 5.10, every invalid share set has linearly independent rows. Hence the rows $(\mathbf{M}[j])_{j \in T_C}$ are linearly independent. Since T_C is invalid, we also have

$$\mathbb{1}_1 \notin \operatorname{span}\big((\mathbf{M}[j])_{j \in T_C}\big).$$

Therefore the rows $(\mathbf{M}[j])_{j \in T_C}$ together with $\mathbb{1}_1$ are linearly independent, proving the second item.

Finally, any linearly independent set of vectors in $\mathbb{F}_q^d$ can be efficiently extended to a basis, for example by Gaussian elimination. This yields vectors $\mathbf{u}_1, \ldots, \mathbf{u}_r$ such that the matrix $\mathbf{B}_C$ above is invertible. This proves the third item. □

Theorem 5.13 (Programmability from LI-Unauthorized)**.** Every LSSS for $\mathbb{A}$ with $\mathsf{rank}(\mathbf{M}) = d$ and satisfying Definition 5.10 is programmable in the sense of Definition 4.4.

Proof. We define $\mathsf{ProgShare}_1$ and $\mathsf{ProgShare}_2$ by solving linear systems for a sharing vector $\mathbf{k} = [k, r_2, \ldots, r_d]^\top$ consistent with the programmed data. Mode 1 uses the codimension-one structure of maximally invalid share sets from Lemma 5.11, while Mode 2 uses the basis-extension property for arbitrary invalid party sets from Lemma 5.12.

Mode 1 ($\mathsf{ProgShare}_1$). On input $(\mathbb{A}, T, (\tilde{w}_j)_{j \in T}, j^*, \tilde{w}_{j^*})$ where T is a maximally invalid share set:

1. Form the invertible matrix $\mathbf{A}_{T,j^*}$ from Lemma 5.11, and form the programming constraint vector
$$\mathbf{b}_{T,j^*} = \big[(\tilde{w}_j)_{j \in T}, \tilde{w}_{j^*}\big]^\top \in \mathbb{F}_q^d.$$
2. Compute the unique solution $\mathbf{k} = \mathbf{A}_{T,j^*}^{-1} \mathbf{b}_{T,j^*}$.
3. Set $k = \langle \mathbb{1}_1, \mathbf{k} \rangle$, compute $\mathbf{w} = \mathbf{M}\mathbf{k}$, and output $(k, \mathbf{sh}_1, \ldots, \mathbf{sh}_N)$ where $\mathbf{sh}_i = (w_j)_{j \in T_i}$.

Mode 2 ($\mathsf{ProgShare}_2$). On input $(\mathbb{A}, C, (\tilde{\mathbf{sh}}_i)_{i \in C}, k)$ where C is an invalid set of parties:

1. Let $T_C = \bigcup_{i \in C} T_i$, and let $(\tilde{w}_j)_{j \in T_C}$ denote the coordinates of the programmed party shares $(\tilde{\mathbf{sh}}_i)_{i \in C}$.
2. Let $\mathbf{B}_C \in \mathbb{F}_q^{d \times d}$ be any invertible matrix of the form guaranteed by Lemma 5.12, and let $r = d - (|T_C| + 1)$. Sample $z_1, \ldots, z_r \leftarrow \mathbb{F}_q$ uniformly, and form
$$\mathbf{b}_{C,k} = \big[(\tilde{w}_j)_{j \in T_C}, k, z_1, \ldots, z_r\big]^\top \in \mathbb{F}_q^d.$$
.
3. Compute the unique solution $\mathbf{k} = \mathbf{B}_C^{-1} \mathbf{b}_{C,k}$.
4. Compute $\mathbf{w} = \mathbf{M}\mathbf{k}$ and output $(k, \mathbf{sh}_1, \ldots, \mathbf{sh}_N)$ where $\mathbf{sh}_i = (w_j)_{j \in T_i}$.

We now verify Definition 4.4. Consistency holds because in both modes the output shares are of the form $\mathbf{w} = \mathbf{M}\mathbf{k}$ for some vector $\mathbf{k} = [k_b, r_2, \ldots, r_d]^\top$. Hence, by correctness of the underlying LSSS, every valid set of parties $A \in \mathbb{A}$ reconstructs the corresponding output secret k_b from the shares $(\mathbf{sh}_{b,i})_{i \in A}$, and therefore $\mathsf{Combine}(\mathbb{A}, (\mathbf{sh}_{b,i})_{i \in A}) = k_b$ for each $b \in \{0, 1\}$. For Mode 1, the defining linear system enforces $w_j = \tilde{w}_j$ for all $j \in T$ and also $w_{j^*} = \tilde{w}_{j^*}$, which implies consistency with the programmed coordinates. For Mode 2, the defining linear system enforces $w_j = \tilde{w}_j$ for all $j \in T_C$, and hence $\mathbf{sh}_i = \tilde{\mathbf{sh}}_i$ for every $i \in C$. Moreover, the output secret is exactly k.

It remains to prove mode indistinguishability. Fix any maximally invalid share set $T^* \subseteq [L^*]$, any invalid set of parties $C^* \subseteq [N]$, and any $j^* \in [L^*] \setminus T^*$. In Exp_1, the input vector $\mathbf{b}_{T^*,j^*} = \big[(\tilde{w}_j)_{j \in T^*}, \tilde{w}_{j^*}\big]$ is uniform in $\mathbb{F}_q^d$. Since $\mathbf{A}_{T^*,j^*}$ is invertible, the resulting vector $\mathbf{k} = \mathbf{A}_{T^*,j^*}^{-1} \mathbf{b}_{T^*,j^*}$ is uniform in $\mathbb{F}_q^d$. Thus the output of Exp_1 is distributed exactly as $(\langle \mathbb{1}_1, \mathbf{k} \rangle, (\mathbf{M}\mathbf{k})_{T_1}, \ldots, (\mathbf{M}\mathbf{k})_{T_N})$ for uniform $\mathbf{k} \leftarrow \mathbb{F}_q^d$.

In Exp_2, let $T_{C^*} = \bigcup_{i \in C^*} T_i$, and let $(\tilde{w}_j)_{j \in T_{C^*}}$ be the coordinates of the programmed party shares $(\tilde{\mathbf{sh}}_i)_{i \in C^*}$. Let $\mathbf{B}_{C^*}$ be the invertible matrix fixed by Mode 2, as guaranteed by Lemma 5.12. The vector

$$\mathbf{b}_{C^*,k} = \left[(\tilde{w}_j)_{j \in T_{C^*}}, k, z_1, \ldots, z_r\right]^\top$$

is uniform in $\mathbb{F}_q^d$, since $(\tilde{\mathbf{sh}}_i)_{i \in C^*}$ and k are sampled uniformly and independently, and the additional values $z_1, \ldots, z_r$ are also sampled uniformly and independently. Since $\mathbf{B}_{C^*}$ is invertible, the resulting vector $\mathbf{k} = \mathbf{B}_{C^*}^{-1}\mathbf{b}_{C^*,k}$ is uniform in $\mathbb{F}_q^d$. Hence the output of Exp_2 has exactly the same distribution as in Exp_1. Therefore Exp_1 and Exp_2 are identically distributed. □

Corollary 5.14. The standard Lewko–Waters LSSS for DNF formulas is a programmable $(1, 1)$-Scaled LSS scheme.

Proof. Theorem 5.9 gives the scaled reconstruction claim. For programmability, the Lewko–Waters representation for DNF formulas has full column rank and satisfies the LI-unauthorized property, so Theorem 5.13 applies. □

5.3 Composing Scaled LSS: OR-Threshold-AND Policies

In this section, we generalize the class of access structures for which we construct programmable (α, β)-Scaled LSS schemes. Recursive secret-sharing constructions already appear in the classical monotone-formula construction of Benaloh and Leichter [BL90], while threshold access trees provide a closely related tree-based policy language that is widely used in later cryptographic constructions [OSW07]. Our point here is not to introduce a new tree syntax, but rather to characterize exactly when the concrete programmable (α, β)-Scaled LSS constructions proved above remain available under recursive composition.

The basic idea is simple. Each internal node in the tree carries a local access structure from one of the families already handled above, while the leaves correspond to the actual parties. The secret at the root is first shared according to the access policy at that node; each resulting child share is then recursively shared inside the corresponding subtree, and this process continues until one reaches the leaves. Correctness, privacy, and bounded scaled reconstruction are local statements that compose through this recursion for *any* such tree. *Programmability, however, does not compose in general*: the linear independence of unauthorized shares that our reductions rely on can be destroyed by nesting one satisfiable node inside another. We therefore characterize exactly which trees preserve programmability, and show that the maximal such policies are three-layer OR-of-Thresholds-of-ANDs formulas, which subsume DNFs.

Definition 5.15 (Composition Tree over a Base Family)**.** Let $\mathcal{F}$ be a family of monotone access structures, where every $\mathbb{A} \in \mathcal{F}$ is defined on some finite index set $[m]$. A *composition tree* over $\mathcal{F}$ is a rooted tree $\mathcal{T}$ with the following data:

- Every leaf ℓ is labeled by a distinct party in $[N]$.
- Every internal node v has an ordered list of children $\mathsf{ch}(v) = (u_1, \ldots, u_{m_v})$ and is labeled by an access structure $\mathbb{A}_v \in \mathcal{F}$ on $[m_v]$.

For a set of parties $S \subseteq [N]$, define recursively when a node v is satisfied by S as follows:

- If v is a leaf labeled by party i, then v is satisfied by S if and only if $i \in S$.
- If v is an internal node with children $(u_1, \ldots, u_{m_v})$, then v is satisfied by S if and only if $\{j \in [m_v] : u_j \text{ is satisfied by } S\} \in \mathbb{A}_v$.

Let r be the root of $\mathcal{T}$. The induced access structure is

$$\mathbb{A}_{\mathcal{T}} = \{S \subseteq [N] : r \text{ is satisfied by } S\}.$$

We consider composition trees (Definition 5.15) whose internal nodes are labeled by *OR* (replicate the node secret to every child), *AND* (additive sharing), or *threshold* gates (t_v-of-m_v Shamir), and treat the ingredients of a programmable scaled LSS separately. We assume w.l.o.g. that every gate has at least two children (a single-child gate is the identity); moreover, a t-of-t threshold is an AND, and a 1-of-m threshold is an OR (a degree-0 sharing polynomial is replication). Call a tree *rigid* if all its internal nodes are AND gates, and call a node *redundant* if it is an OR or a threshold with $t_v < m_v$, equivalently, a gate that can be satisfied without using all of its children. Note that in the composed scheme every leaf holds a single field element, so share sets and party sets coincide.

Correctness, privacy, and scaled reconstruction compose freely.

Lemma 5.16 (Reconstruction composes)**.** Let $\mathcal{T}$ be *any* composition tree whose every internal node v admits a scaled LSS with correctness, privacy, and (α_v, β_v)-bounded scaled reconstruction. Sharing the root secret recursively (share s_v among the children of v with the local scheme, then recurse into each child subtree) and reconstructing bottom-up yields a scaled LSS for $\mathbb{A}_{\mathcal{T}}$ with correctness, privacy, and (α', β')-bounded scaled reconstruction, where $\alpha' \le \prod_v \alpha_v$ and $\beta' \le \prod_v \beta_v$ over the nodes v on a root-to-leaf reconstruction path.

Proof. Scaled reconstruction: substituting the child identities $\gamma_u s_u = \sum \langle \boldsymbol{\delta}_{u,\cdot}, \cdot \rangle$ into the parent identity and rescaling, leaves-up, gives $\gamma' k = \sum_\ell \langle \boldsymbol{\delta}'_\ell, \mathbf{sh}_\ell \rangle$ with $\gamma' = \prod_v \gamma_v$ and $\left\|\boldsymbol{\delta}'_\ell\right\|_\infty \le \prod_v \alpha_v$ along the path; correctness follows.

Privacy: by induction, for every node v unsatisfied by a leaf set S, the view of S in v's subtree is independent of s_v. The satisfied children B are unauthorized for $\mathbb{A}_v$, so their secrets $(s_u)_{u \in B}$ are jointly independent of s_v (local privacy); sample them, share their subtrees honestly, and sample each unsatisfied subtree's view by induction. Disjoint fresh randomness makes this exactly the view of S. □

Programmability does not compose in general. Recall (Theorem 5.13) that a full-column-rank LSS is programmable once it is *LI-unauthorized* (Definition 5.10): every invalid share set has linearly independent rows. This property is fragile under composition. We use the following restatement, together with two observations.

Lemma 5.17 (Dependency form)**.** A full-column-rank LSS with share matrix $\mathbf{M}$ is LI-unauthorized iff every linear dependency among its rows has *authorized* support; that is, for every $\mathbf{c} \neq \mathbf{0}$ with $\sum_j c_j \mathbf{M}[j] = \mathbf{0}$, the set $\{j : c_j \neq 0\}$ is a valid share set.

Proof. If some invalid T had dependent rows, it would contain a dependency whose support is valid by hypothesis, making T valid by monotonicity. Conversely, a dependency's support has dependent rows, so it cannot be invalid. □

Throughout, we view each leaf share as a fixed linear function of the root secret k and of the fresh random values used by the internal nodes. A row dependency is then simply a linear combination of leaf shares that is *identically zero*: it cancels for every secret and every choice of randomness.

Observation 5.18 (Rigidity)**.** No nontrivial linear combination of the leaf shares of a rigid tree is identically zero. Indeed, an AND gate is invertible (from its m child secrets one can recover the node secret and the $m-1$ random values, and vice versa), so the leaf shares of a rigid tree are just an invertible linear re-encoding of (secret, randomness).

Observation 5.19 (Valid = authorized)**.** In every composed scheme, a set of leaves can compute some nonzero multiple of the node secret as a linear combination of its shares if and only if the set is authorized: authorized sets can by correctness, and unauthorized sets cannot by privacy (Lemma 5.16).

Theorem 5.20 (Which compositions are programmable)**.** The scaled LSS induced by a composition tree of OR/AND/threshold gates is LI-unauthorized, and hence programmable, if and only if every redundant node is either the root or has *all* of its proper ancestors labeled OR. Moreover, when this condition fails, the induced scheme is not programmable.

Proof. ($\Leftarrow$) Assume the condition. Then no AND and no threshold ever appears above a redundant node, so after merging nested ORs and nested ANDs (which changes neither the policy nor the shares; if the root is not an OR, place a single-child OR above it), the tree has a simple shape: one OR whose branches are either rigid trees or a threshold gate over rigid trees, and every branch carries the same secret k. Moreover, from *all* leaf shares one can recover every node secret and every random value bottom-up, so distinct (secret, randomness) give distinct share vectors and $\mathbf{M}$ has full column rank, as Theorem 5.13 requires.

Now take a dependency: coefficients c_ℓ, not all zero, with $\sum_\ell c_\ell \cdot (\text{share}_\ell)$ identically zero, and split this sum by branches. The randomness of a branch appears in no other branch, so it must already cancel inside its own branch; what remains of branch B_i's part is therefore $\gamma_i \cdot k$ for a scalar γ_i. If some $\gamma_i \neq 0$: the support leaves inside B_i compute $\gamma_i k$ from their shares, so they are authorized (Fact 5.19), and one authorized branch already satisfies the OR root, so the support is authorized. If all $\gamma_i = 0$: each branch part is itself identically zero. On a rigid branch all its coefficients must vanish (Fact 5.18). On a threshold branch, the children are rigid trees holding the Shamir evaluations $p(e_1), \ldots, p(e_b)$ of a random degree-$(t-1)$ polynomial p; splitting the branch part by these subtrees in the same way, subtree j contributes $\mu_j \cdot p(e_j)$, and $\sum_j \mu_j\, p(e_j)$ must be zero for *every* polynomial p. Since a degree-$(t-1)$ polynomial can take arbitrary values on any t points, this is impossible unless either all μ_j vanish (then the branch coefficients vanish, as before) or at least $t+1$ of the μ_j are nonzero. In the latter case, $t+1$ subtrees compute a nonzero multiple of their own secret $p(e_j)$, so they are authorized (Fact 5.19); with $t+1 \geq t$ children satisfied, the threshold and then the OR root are satisfied, so the support is again authorized. So every dependency has authorized support: by Lemma 5.17 the scheme is LI-unauthorized, and by Theorem 5.13 programmable.

($\Rightarrow$) Suppose some redundant node v has a proper ancestor that is not an OR; let a be the nearest such ancestor and let T be all leaves of v's subtree. Because v is redundant, T can compute v's secret s_v in two different ways: from two different children (if v is an OR), or from two different sets of t_v children (if v is a threshold). These two linear expressions are different functions of the shares (otherwise s_v would be computable from the shares the two ways have in common, which form an unauthorized set, contradicting privacy), so their difference is a dependency whose support lies inside T. On the other hand T is unauthorized: it satisfies v, and every node between v and a is an OR and stays satisfied, but at a only this one child is satisfied while a needs at least two (a is an AND or a threshold with $t_a \geq 2$), and T has no leaves elsewhere. An unauthorized set with a dependency means LI-unauthorized fails. Programmability fails as well: pick leaves X outside v's subtree such that X together with either of the two computing sets is authorized. Both authorized sets must reconstruct the same secret from any programmed sharing, which forces the shares of T to satisfy the dependency equation; but $\mathsf{ProgShare}_2$ must copy the uniformly random programmed values into the shares of the unauthorized set T, and uniform values satisfy a fixed nontrivial linear equation only with probability $1/q$. □

In words, Theorem 5.20 says: an AND may sit *below* any gate, since AND-trees create no dependencies (Fact 5.18), but *only an OR may sit above an OR or a threshold.* This explains the asymmetry the theorem draws between threshold-of-ANDs (programmable) and AND-of-thresholds (not programmable): a redundant gate creates a dependency among the leaves below it, and that dependency is harmless exactly when satisfying the gate satisfies the whole tree: true when only ORs are above it, false below an AND whose other children remain unsatisfied. Merging equal gates, the admissible trees are exactly a top OR, at most one threshold layer, and ANDs above the leaves: the OR-of-Thresholds-of-ANDs form. Table 1 summarizes the resulting four policy classes.

Theorem 5.21 (OR of Thresholds of ANDs)**.** Let $\mathbb{A}$ be computed by the depth-3 tree: an *OR* over threshold nodes $T_1, \ldots, T_a$, where each T_i is a t_i-of-b_i *threshold* over *AND* nodes, and each AND node is over a set of leaves. Then $\mathbb{A}$ admits a programmable (α, β)-Scaled LSS with $\alpha = \beta = \max_i b_i^{t_i^2/2}$ over any field (or $\max_i b_i^{b_i - t_i}$ under the magnified instantiation of Corollary 5.8). Moreover, dropping any one layer preserves programmability:

- dropping the AND layer gives an *OR of thresholds* (over leaves);
- dropping the threshold layer ($t_i = 1$) gives an *OR of ANDs*, i.e. a *DNF formula*, recovering Corollary 5.14 with $\alpha = \beta = 1$;

Composition	Special case of Theorem 5.21	$\alpha = \beta$
OR of Thresholds of ANDs	general form	$\max_i b_i^{\min(t_i^2/2,\, b_i - t_i)}$
OR of ANDs (DNF, Corollary 5.14)	$t_i = 1$	1
OR of Thresholds	singleton ANDs	$\max_i b_i^{\min(t_i^2/2,\, b_i - t_i)}$
Threshold of ANDs	$a = 1$	$b^{\min(t^2/2,\, b-t)}$

Table 1: All composition trees admitting a programmable Scaled LSS (Theorem 5.20). The bounds choose, per threshold gate, the better of Corollaries 5.7 and 5.8.

- dropping the OR layer ($a = 1$) gives a single *threshold of ANDs*.

Proof. Every redundant gate (T_i or the root OR) has only OR ancestors, so Theorem 5.20 gives programmability. OR and AND layers are $(1, 1)$-scaled, so along any reconstruction path only one threshold contributes to the bound of Lemma 5.16, giving the Shamir bound of Corollary 5.7 maximized over i. Omitting any layer preserves the condition. □

Finally, we show that the characterization is tight, that is, stacking any non-OR gate above a threshold layer fails. The following covers both a threshold above thresholds ($t_1 < m_1$) and an AND above thresholds ($t_1 = m_1$) in one statement.

Remark 5.22 (Counterexample: Programmability of Nested Thresholds)**.** Let $\mathbb{A}$ be a t_1-of-m_1 threshold with $t_1 \geq 2$ over threshold nodes that are each t_2-of-m_2 with $t_2 < m_2$; the case $t_1 = m_1$ is an AND over thresholds. Then $\mathbb{A}$ is not LI-unauthorized, so the recursive construction is not programmable. Indeed, take all m_2 leaves below a single middle node: their shares are m_2 evaluations of one degree-(t_2-1) polynomial, and since $m_2 > t_2$, some linear combination of them is identically zero, so the rows are dependent. Yet only one of the m_1 middle nodes is satisfied and $t_1 \geq 2$, so the set is unauthorized.

Remark 5.23 (Privacy vs. programmability)**.** Why does privacy survive where programmability fails? A row dependency $\sum_j c_j \mathbf{M}[j] = \mathbf{0}$ holds for *every* secret and randomness, so observing it reveals nothing about the secret; it only certifies that the shares are not *jointly uniform*, which is what programmability needs and privacy does not. In Proposition 5.22 the unauthorized set learns one middle-node secret in full, but that is a single top-level Shamir share ($t_1 \geq 2$), uniform and independent of the root secret.

6 Monotone Policy Encryption with Silent Setup

6.1 Definition

Syntax. A monotone policy encryption scheme with silent setup (DPE) for a class of access policies $\mathbb{S}$ has the following polynomial time algorithms.

$\mathsf{GSetup}(1^\lambda, N) \to \mathsf{crs}$**.** The possibly randomized *public-coin* global setup algorithm takes as input security parameter λ, number of users N, and outputs the common reference string crs. The following algorithms take crs implicitly.

$\mathsf{KGen}(i) \to (\mathsf{pk}_i, \mathsf{sk}_i)$**.** The randomized key generation algorithm takes as input $i \in [N]$, and outputs public-secret key pair $(\mathsf{pk}_i, \mathsf{sk}_i)$.

$\mathsf{PreProcess}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}) \to (\mathsf{mpk}, \mathsf{aux})$**.** The randomized *public-coin* pre-processing algorithm takes as input public keys for a users in set S, $\{\mathsf{pk}_i\}_{i\in S}$, access structure $\mathbb{A} \in \mathbb{S}$ for $|S|$ parties, outputs master public key mpk and auxiliary information aux.

$\mathsf{Enc}(\mathsf{mpk}, \mathsf{aux}, \mu) \to \mathsf{ct}$**.** The randomized encryption algorithm takes as input master public key mpk, auxiliary information aux, message $\mu \in \{0, 1\}$, and outputs ciphertext ct.

$\mathsf{PartDec}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}, (j, \mathsf{sk}_j), \mathsf{ct}) \to \mathsf{dsh}_j$**.** The possibly randomized partial decryption algorithm takes as input public keys for users in S, $\{\mathsf{pk}_i\}_{i\in S}$, access structure $\mathbb{A}$, secret key for the j-th user sk_j, ciphertext ct, and outputs decryption share dsh_j.

$\mathsf{Recon}(\{\mathsf{dsh}_j\}_{j\in S'}, \mathbb{A}, \mathsf{ct}) \to \mu'$**.** The deterministic reconstruction algorithm takes as input decryption shares from users in set S', $\{\mathsf{dsh}_j\}_{j\in S'}$, access structure $\mathbb{A}$, ciphertext ct, and outputs message $\mu' \in \{0,1\} \cup \{\perp\}$.

Definition 6.1 (DPE)**.** A DPE scheme $(\mathsf{GSetup}, \mathsf{KGen}, \mathsf{PreProcess}, \mathsf{Enc}, \mathsf{PartDec}, \mathsf{Recon})$ is said to be a DPE scheme for access policies $\mathbb{S}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}, N \le 2^\lambda$, $\mu \in \{0,1\}$, monotone access policy $\mathbb{A} \in \mathbb{S}, S' \in \mathbb{A}$,

$$\Pr\left[\begin{array}{l|ll} & \mathsf{crs} \leftarrow \mathsf{GSetup}(1^\lambda, N), & \\ & \forall\ i \in S, (\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KGen}(i), & \\ \mu = \mathsf{Recon}(\{\mathsf{dsh}_j\}_{j\in S'}, \mathbb{A}, \mathsf{ct}) & : & (\mathsf{mpk}, \mathsf{aux}) \leftarrow \mathsf{PreProcess}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}), \\ & \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathsf{aux}, \mu), \forall\ j \in S', & \\ & \mathsf{dsh}_j \leftarrow \mathsf{PartDec}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}, (j, \mathsf{sk}_j), \mathsf{ct}) & \end{array}\right] = 1$$

Compactness. There exists a polynomial $\mathsf{poly}(\cdot)$ such that for any $\lambda \in \mathbb{N}, N \le 2^\lambda, S \subseteq [N], \mathbb{A} \in \mathbb{S}$, $|\mathsf{ct}| = o(|S|)\mathsf{poly}(\lambda)$.

Selective Security. For any stateful admissible PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\left|\Pr\left[\begin{array}{l|ll} & (N, \mathbb{A}, S, C) \leftarrow \mathcal{A}(1^\lambda), & \\ & \mathsf{crs} \leftarrow \mathsf{GSetup}(1^\lambda, N), & \\ \mu \leftarrow \mathcal{A}\left(\begin{array}{c}\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in S}, \\ \{\mathsf{sk}_j\}_{j\in C}, \mathsf{aux}, \mathsf{ct}\end{array}\right) & : & \forall\ i \in S, (\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KGen}(i), \\ & (\mathsf{mpk}, \mathsf{aux}) \leftarrow \mathsf{PreProcess}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}), & \\ & \mu \leftarrow \{0,1\}, \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathsf{aux}, \mu) & \end{array}\right] - \frac{1}{2}\right| \le \mathsf{negl}(\lambda)$$

A stateful PPT machine $\mathcal{A}$ is said to be admissible if $C \subseteq S \subseteq [N]$ and $C \notin \mathbb{A}$.

Remark 6.2 (Encrypting Larger Messages.)**.** By a standard hybrid encryption technique, we can encrypt messages of arbitrary length using a DPE scheme for bit messages.

6.2 Construction

In this section, for simplicity of exposition, we assume that each user receives a maximum of one share. This can be generalized to any L shares by expanding the size of public and secret keys.

Lattice parameter convention. Throughout, we fix a constant $\epsilon \in (0, 1/2)$ and instantiate the underlying lattice assumptions with $q/B_{\mathsf{LWE}} = 2^{\widetilde{O}(n^\epsilon)}$. For a threshold t among N parties, let $\tau = \min(t^2, N - t)$. Correctness requires $\log q = O^\dagger(\tau)$, and we therefore choose $n = O^\dagger(\tau^{1/\epsilon})$ and $m = O^\dagger(\tau^{1+1/\epsilon})$. All parameter bounds below use this convention.

Parameters. We define the parameters used in our construction below.

- $\chi = \mathsf{poly}(n, \log N)$. n is chosen according to the lattice parameter convention as above, so that $q/B_{LWE} = 2^{\widetilde{O}(n^\epsilon)}$ for a fixed $\epsilon \in (0, 1/2)$. (Decomposed LWE assumption)
- $m = \Omega(n \log q)$ (Leftover hash lemma)
- $\sigma = \mathsf{poly}(\lambda, m)$ (Decomposed LWE assumption)
- $\chi' > \lambda^{\omega(1)} \cdot p \cdot \max\{\alpha, \beta\} \cdot N \cdot O(\sigma\sigma_{\mathsf{er}}\chi m^{14} \cdot \log q \cdot \log N)$ (Smudging)
- $\sigma_{\mathsf{er}} > \lambda^{\omega(1)} \cdot O(\sigma m^{13} \cdot \log q \cdot \log N)$ (Security)
- Odd primes $p > 3N$, $q > 4(\beta + pN\sqrt{m}\chi')$ (Correctness)

Construction 6.3. We provide the construction of a DPE scheme (Definition 6.1) for access structures $\mathbb{S}$ that admit programmable (α, β)-Scaled LSSS (Definition 4.4, Definition 4.1) using $2m^2$-decomposed LWE (Assumption 3.5), matrix commitment scheme MCOM (Definition 3.7), and explainable re-randomizer ER (Definition 3.8) as follows. Here, $\mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\rho$ are hash functions which are modeled as random oracles in the security proof.

$\mathsf{GSetup}(1^\lambda, N)$**.** Sample $(\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{MCOM.Setup}(1^n, 1^m, q, \sigma)$. Sample hash function $\mathcal{H}_\rho : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}$ and $\mathcal{H}_{\mathsf{sh}} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \mathbb{Z}_q^n$.

Output $\mathsf{crs} := (N, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}})$.

$\mathsf{KGen}(i)$**.** Sample $\mathbf{r}_i \leftarrow \{\pm 1\}^{4m^5}$ and compute $\mathbf{t}_i = \mathbf{B}\mathbf{r}_i$. Output $(\mathsf{pk}_i, \mathsf{sk}_i) := ((i, \mathbf{t}_i), \mathbf{r}_i)$.

$\mathsf{PreProcess}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A})$**.** Parse pk_i as $(i, \mathbf{t}_i)$ for each $i \in S$. If $\mathbf{1}_{|S|} \notin \mathbb{A}$, abort and output $(\perp, \perp)$. Otherwise, do the following.

1. Sample $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$.
2. Let $\rho = \mathsf{poly}(\lambda, |S|)$ be the length of randomness used by ER. Sample $r_k = \mathcal{H}_\rho(\mathsf{sd}_\rho, k)$ for each $k \in [\rho]$.
3. Compute $(\mathbf{C}_0, \mathbf{Z}_0) = \mathsf{ER.Sample}(1^\lambda, 1^{|S|}, \sigma_{\mathsf{er}}; (r_1, \ldots, r_\rho))$.
4. Let A be the set of a minimally valid parties for $\mathbb{A}$. For each $j \in A$, compute $\widetilde{\mathsf{sh}}_j = \mathcal{H}_{\mathsf{sh}}(\mathsf{sd}_{\mathsf{sh}}, j)$.
5. Compute $(\mathbf{p}, \mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) = \mathsf{ProgShare}_1(1^{|S|}, \mathbb{A}, A, \{\widetilde{\mathsf{sh}}_j\}_{j\in A})$.
6. Compute matrix $\mathbf{X} = [\mathbf{x}_1 \mid \ldots \mid \mathbf{x}_{|S|}] \in \mathbb{Z}_q^{n\times|S|}$ with $\mathbf{x}_j = \mathbf{t}_j + \mathsf{sh}_j$.
7. Compute $\mathbf{C} = \mathsf{MCOM.Com}(\mathbf{X})$.

Output $\mathsf{mpk} := \mathbf{C}_0 + \mathbf{C}, \mathsf{aux} := (\mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}, \mathbf{p})$.

$\mathsf{Enc}(\mathsf{mpk}, \mathsf{aux}, \mu)$**.** If mpk or aux is $\perp$, output $\perp$. Otherwise, parse mpk as $\widehat{\mathbf{C}} \in \mathbb{Z}_q^{n\times m}$ and aux as $(\mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}, \mathbf{p})$.

1. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5\times m}, \mathbf{r}_{\mathbf{p}} \leftarrow \{\pm 1\}^{4m^5}$.
2. Compute $\mathbf{c}_1^\top = \mathbf{s}^\top\mathbf{B} + p\mathbf{e}^\top, \mathbf{c}_2^\top = \mathbf{s}^\top\widehat{\mathbf{C}} + p\mathbf{e}^\top\mathbf{R}_{\mathbf{C}}, c_3 = \mathbf{s}^\top\mathbf{p} + p\mathbf{e}^\top\mathbf{r}_{\mathbf{p}} + \mu$.

Output $\mathsf{ct} := (\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}) \in \mathbb{Z}_q^{1\times(m+4m^5)} \times \{0,1\}^{2\lambda}$.

$\mathsf{PartDec}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}, (j, \mathsf{sk}_j), \mathsf{ct})$**.** If ct is $\perp$, output $\perp$. Otherwise, parse pk_i as $(i, \mathbf{t}_i)$ for each $i \in S$. Parse sk_j as $\mathbf{r}_j \in \{\pm 1\}^{4m^5}$. If $j \notin S$, abort and output $\perp$. Otherwise, parse ct as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}) \in \mathbb{Z}_q^{1\times(4m^5+m+1)} \times \{0,1\}^{2\lambda}$ and perform the following steps.

1. Perform steps 2-7 from PreProcess using $\mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}, \mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\rho$. From these, use $\mathbf{X}, \mathbf{Z}_0$.
2. Compute $\mathbf{V}_{|S|} = \mathsf{MCOM.Verify}(1^{|S|})$ and $\mathbf{Z}_{\mathbf{X}} = \mathsf{MCOM.Open}(\mathbf{X})$.
3. Compute $\mathbf{c}_{exp}^\top = \mathbf{c}_2^\top\mathbf{V}_{|S|} + \mathbf{c}_1^\top(\mathbf{Z}_0 + \mathbf{Z}_{\mathbf{X}}) \in \mathbb{Z}_q^{1\times|S|}$.
4. Let $i_j \in [|S|]$ denote the index of j in the set S according to lexicographical ordering. Let $\widetilde{c} \in \mathbb{Z}_q$ be the i_j-th entry of $\mathbf{c}_{exp}^\top$.
5. Sample $e' \leftarrow \mathcal{D}_{\mathbb{Z},\chi'}$.

Output $\mathsf{dsh}_j := \widetilde{c} - \mathbf{c}_1^\top\mathbf{r}_j + pe' \in \mathbb{Z}_q$.

$\mathsf{Recon}(\{\mathsf{dsh}_j\}_{j\in S'}, \mathbb{A}, \mathsf{ct})$**.** If ct is $\perp$ or any of $\{\mathsf{dsh}_j\}_{j\in S'}$ are $\perp$, output $\perp$. Otherwise, parse ct as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}) \in \mathbb{Z}_q^{1\times(2m+1)} \times \{0,1\}^{2\lambda}$. If $S' \notin \mathbb{A}$, abort and output $\perp$. Otherwise,

1. Compute coefficients $((\delta_{S',j})_{j\in S'}, \gamma_{S'}) = \mathsf{GetCoeffs}(\mathbb{A}, S')$.

2. Compute $c' = \sum_{j \in S'} \delta_{S',j} \mathsf{dsh}_j$.
3. Compute $y = \gamma_{S'} c_3 - c'$ and $y' = y \mod p$.

Output $(\gamma_{S'})^{-1} y' \mod p$.

Remark 6.4 (Reducing Partial Decryption Running Time)**.** In Construction 6.3, the partial decryption procedure requires each user j to recompute the matrices $\mathbf{V}_{|S|}$, $\mathbf{Z}_0$, and $\mathbf{Z}_{\mathbf{X}}$ from scratch, which causes its running time to grow with $|S|$. This can be avoided by having the $\mathsf{PreProcess}$ algorithm provide per-user hints $(\mathbf{v}_j, \mathbf{z}_j)$, the columns of $\mathbf{V}_{|S|}$ and $\mathbf{Z}_0 + \mathbf{Z}_{\mathbf{X}}$ corresponding to user j. This way, partial decryption requires only local computation independent of $|S|$.

Remark 6.5 (Fully Silent Setup)**.** The global setup algorithm GSetup in our DPE scheme is public-coin. Its output $\mathsf{crs} = (N, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}})$ is determined entirely by public randomness. Therefore, GSetup can be eliminated by deriving crs from a hash function modeled as a random oracle. With this modification, the setup is *fully* silent. Each party generates its key pair using only public randomness and local computation, with no trusted setup or coordination whatsoever.

Correctness. Note that Recon outputs $\perp$ if $S' \notin \mathbb{A}$. So, we will only focus on the case when $S' \in \mathbb{A}$. For each $i \in S$, we have that $\mathsf{pk}_i = (i, \mathbf{t}_i), \mathsf{sk}_i = \mathbf{r}_i$ where $\mathbf{t}_i = \mathbf{B}\mathbf{r}_i$. In $\mathsf{PreProcess}, \mathsf{PartDec}$, $\mathbf{X} = [\mathbf{x}_1 \mid \ldots \mid \mathbf{x}_{|S|}]$ where $\mathbf{x}_j = \mathbf{t}_j + \mathsf{sh}_j$ for $j \in [|S|]$. Here, sh_j is computed as mentioned in $\mathsf{PreProcess}$ steps 4-5. Similarly, in $\mathsf{PreProcess}, \mathsf{PartDec}$, we have $(\mathbf{C}_0, \mathbf{Z}_0)$ such that $\mathbf{C}_0 \mathbf{V}_{|S|} = -\mathbf{B}\mathbf{Z}_0$ by correctness of ER where $\mathbf{V}_{|S|} = \mathsf{MCOM.Verify}(1^{|S|})$. By correctness of MCOM, we have $\mathbf{C}\mathbf{V}_{|S|} = \mathbf{X} - \mathbf{B}\mathbf{Z}_{\mathbf{X}}$ where $\mathbf{C} = \mathsf{MCOM.Com}(\mathbf{X}), \mathbf{Z}_{\mathbf{X}} = \mathsf{MCOM.Open}(\mathbf{X})$. Note that we also compute $(\mathbf{C}_0, \mathbf{Z}_0)$ using the deterministic procedure from steps 2-3 in $\mathsf{PreProcess}$ so they are the same in $\mathsf{PreProcess}, \mathsf{PartDec}$. Thus,

$$\begin{aligned}
\mathbf{c}_{exp}^\top &= \mathbf{c}_2^\top \mathbf{V}_{|S|} + \mathbf{c}_1^\top (\mathbf{Z}_0 + \mathbf{Z}_{\mathbf{X}}) \\
&= \mathbf{s}^\top (\mathbf{C}_0 \mathbf{V}_{|S|} + \mathbf{C}\mathbf{V}_{|S|} + \mathbf{B}\mathbf{Z}_0 + \mathbf{B}\mathbf{Z}_{\mathbf{X}}) + \overbrace{p\mathbf{e}^\top (\mathbf{Z}_0 + \mathbf{Z}_{\mathbf{X}} + \mathbf{R}_{\mathbf{C}} \mathbf{V}_{|S|})}^{p\widetilde{\mathbf{e}}^\top} \\
&= \mathbf{s}^\top \mathbf{X} + p\widetilde{\mathbf{e}}^\top
\end{aligned}$$

Hence, $\widetilde{c}_j$ from $\mathsf{PartDec}$ with sk_j is $\mathbf{s}^\top (\mathbf{t}_j + \mathsf{sh}_j) + p\widetilde{e}_j + pe'_j$. This means that $\mathsf{dsh}_j = \mathbf{s}^\top (\mathbf{t}_j + \mathsf{sh}_j) + p\widetilde{e}_j - \mathbf{s}^\top \mathbf{B}\mathbf{r}_j - p\mathbf{e}^\top \mathbf{r}_j + pe'_j = \mathbf{s}^\top \mathsf{sh}_j + p\widetilde{e}_j - p\mathbf{e}^\top \mathbf{r}_j + pe'_j$.

In Recon, as $|S'| \in \mathbb{A}$, we have that $\sum_{j \in S'} \delta_{S',j} \mathsf{sh}_j = \gamma_{S'} \mathbf{p}$ as $\mathbf{p}$ is the secret generated in $\mathsf{PreProcess}, \mathsf{PartDec}$ by correctness of programmable linear secret sharing schemes with (α, β)-bounded reconstruction. Thus, $c' = \mathbf{s}^\top (\gamma_{S'} \mathbf{p}) + p\big(\sum_{j \in S'} \delta_{S',j} (\widetilde{e}_j - \mathbf{e}^\top \mathbf{r}_j + e'_j)\big)$. Then, $y = \gamma_{S'} \mu + p\big(\gamma_{S'} \mathbf{e}^\top \mathbf{r}_{\mathbf{p}} - \sum_{j \in S'} \delta_{S',j} (\widetilde{e}_j - \mathbf{e}^\top \mathbf{r}_j + e'_j)\big)$.

With high probability, $\|\mathbf{Z}_0\| \le \sqrt{m}\sigma_{\mathsf{er}}, \|\mathbf{T}\| \le \sqrt{m}\sigma$, and $\|\mathbf{e}\| \le \sqrt{m}\chi, |e'| \le \sqrt{m}\chi'$. Moreover, $\|\mathbf{R}_{\mathbf{C}}\| = 1, \|\mathbf{Z}_{\mathbf{X}}\| \le O(\sigma m^{15/2} \log q \log N)$ and $\|\mathbf{V}_{|S|}\| \le O(\sigma m^{9/2} \log q)$. Thus, $\|\mathbf{Z}_0 + \mathbf{Z}_{\mathbf{X}} + \mathbf{R}_{\mathbf{C}} \mathbf{V}_{|S|}\| \le O(\sigma\sigma_{\mathsf{er}} m^{15/2} \log q \log N)$. Thus, $\|\widetilde{\mathbf{e}}^\top\| \le O(\sigma\sigma_{\mathsf{er}} \chi m^{13} \log q \log N)$. As $|e'| = \lambda^{\omega(1)} \|\widetilde{\mathbf{e}}^\top\|$, we have $|p\big(\gamma_{S'} \mathbf{e}^\top \mathbf{r}_{\mathbf{p}} - \sum_{j \in S'} \delta_{S',j} (\widetilde{e}_j - \mathbf{e}^\top \mathbf{r}_j + e'_j)\big)| \le pN \max\{\alpha, \beta\} \lambda^{\omega(1)} O(\sigma\sigma_{\mathsf{er}} \chi m^{13} \log q \log N)$.

If $q > 4(\beta + pN \max\{\alpha, \beta\} \lambda^{\omega(1)} O(\sigma\sigma_{\mathsf{er}} \chi m^{13} \log q \log N))$, then $y \mod q = y$ and thus $y' = \gamma_{S'} \mu \mod p$ and correctness holds.

Compactness. Under the parameter convention of Section 6.2, for a threshold policy with $\tau = \min(t^2, |S| - t)$, the public key, secret key, ciphertext, and partial decryption have size $O^\dagger(\tau^3)$, $O^\dagger(\tau^{15})$, $O^\dagger(\tau^{16})$, and $O^\dagger(\tau)$, respectively. Thus all parameters are sublinear in $|S|$ whenever $\tau = o(|S|^{1/16})$.

Remark 6.6 (DPE from ℓ-Succinct LWE)**.** The $O(m^5)$ ciphertext dimension stems from the matrix commitment instantiated over $2m^2$-decomposed LWE, whose public matrix $\mathbf{B}$ has $\Theta(m^5)$ columns; this is also what makes GSetup public-coin. Trading the transparent setup for a *structured common reference string*, the same construction can be instantiated with the matrix commitment from the ℓ-succinct LWE assumption [Wee24], where $\mathbf{B}$ has only $O(n \log q)$ columns. The ciphertext size then drops from $O(m^5 \log q)$ to

$O(m \log q) = O(n \log^2 q)$, and the keys and partial decryptions shrink accordingly: for $\tau = \min(t^2, |S| - t)$, the public keys, secret keys, ciphertexts, and partial decryptions then have size $O^\dagger(\tau^3)$, $O^\dagger(\tau^3)$, $O^\dagger(\tau^4)$, and $O^\dagger(\tau)$, respectively, and all parameters are sublinear whenever $\tau = o(|S|^{1/4})$.

DPE for DNF Formulae. The construction above extends naturally to DNF formula access structures using programmable $(1,1)$-Scaled LSSS, which recovers $\mathbf{s}^\top \mathbf{p}$ exactly ($\gamma_{S'} = 1$) and gives fully compact ciphertexts independent of N. The one caveat is that the simplifying assumption made at the beginning of the construction that each party receives a single share no longer holds for DNF formulae. In the Lewko–Waters LSSS, each party i may receive up to L shares, where L is the maximum fan-out in the monotone access structure. Accordingly, each party's key pair must accommodate L shares. The secret key becomes $\mathbf{R}_i \in \{\pm 1\}^{4m^5 \times L}$ and the public key becomes $\mathbf{T}_i = \mathbf{B}\mathbf{R}_i \in \mathbb{Z}_q^{n \times L}$, so that both $|\mathsf{pk}_i|$ and $|\mathsf{sk}_i|$ grow with L. The ciphertext size, however, remains $\mathsf{poly}(\lambda, \log N)$, fully independent of both $|S|$ and L.

Theorem 6.7. If $2m^2$-decomposed LWE assumption holds (Assumption 3.5), MCOM satisfies mode indistinguishability (Definition 3.7), ER satisfies randomness and explainability properties (Definition 3.8), then Construction 6.3 satisfies selective security (Definition 6.1) in random oracle model.

6.3 Security Analysis

We prove Theorem 6.7 using the following hybrids and lemmas.

$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$. This is the security experiment from Definition 6.1 with $b = 0/1$.

Setup Phase. $\mathcal{A}$ sends $(N, S, \mathbb{A}, C)$. Do the following.

1. Sample $(\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{MCOM.Setup}(1^n, 1^m, q, \sigma)$.
2. For each $i \in S$, sample $\mathbf{r}_i \leftarrow \{\pm 1\}^{4m^5}$ and set $\mathsf{pk}_i := (i, \mathbf{B}\mathbf{r}_i), \mathsf{sk}_i := \mathbf{r}_i$.
3. Sample $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$.
4. Let $\rho = \mathsf{poly}(\lambda, |S|)$ be the length of randomness used by ER. Sample $r_k = \mathcal{H}_\rho(\mathsf{sd}_\rho, k)$ for each $k \in [\rho]$.
5. Compute $(\mathbf{C}_0, \mathbf{Z}_0) = \mathsf{ER.Sample}(1^\lambda, 1^{|S|}, \sigma_{\mathsf{er}}; (r_1, \ldots, r_\rho))$.
6. Let A be the minimally valid party set for $\mathbb{A}$ in S. For each $j \in A$, compute $\widetilde{\mathsf{sh}}_j = \mathcal{H}_{\mathsf{sh}}(\mathsf{sd}_{\mathsf{sh}}, j)$.
7. Compute $(\mathbf{p}, \mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) = \mathsf{ProgShare}_1(\mathbb{A}, A, \{\widetilde{\mathsf{sh}}_j\}_{j \in A})$.
8. Compute matrix $\mathbf{X} = [\mathbf{x}_1 \mid \ldots \mid \mathbf{x}_{|S|}] \in \mathbb{Z}_q^{n \times |S|}$ with $\mathbf{x}_j = \mathbf{t}_j + \mathsf{sh}_j$.
9. Compute $\mathbf{C} = \mathsf{MCOM.Com}(\mathbf{X})$.
10. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5 \times m}, \mathbf{r}_{\mathbf{p}} \leftarrow \{\pm 1\}^{4m^5}$.
11. Compute $\mathbf{c}_1^\top = \mathbf{s}^\top \mathbf{B} + p\mathbf{e}^\top, \mathbf{c}_2^\top = \mathbf{s}^\top(\mathbf{C}_0 + \mathbf{C}) + p\mathbf{e}^\top \mathbf{R}_{\mathbf{C}}, c_3 = \mathbf{s}^\top \mathbf{p} + p\mathbf{e}^\top \mathbf{r}_{\mathbf{p}} + b$.
12. Set $\mathsf{ct} := (\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}})$.

Send $(\mathsf{crs} := (N, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}}), \{\mathsf{pk}_i\}_{i \in S}, \{\mathsf{sk}_j\}_{j \in C}, \mathsf{aux} := (\mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}, \mathbf{p}), \mathsf{ct})$ to $\mathcal{A}$. Here, $\mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}}$ are handles to random oracles.

Random Oracle Queries. Given a query $(\mathsf{sd}, i) \in \{0,1\}^\lambda, \{0,1\}^\lambda$ for $\mathcal{H}_\rho$ or $\mathcal{H}_{\mathsf{sh}}$, if the query is for $\mathcal{H}_\rho$, sample $\beta \leftarrow \{0,1\}$ and send β. Otherwise, if the query is for $\mathcal{H}_{\mathsf{sh}}$, sample $\mathsf{sh} \leftarrow \mathbb{Z}_q^n$ and send sh.

Guess Phase. $\mathcal{A}$ outputs its guess b'. Output $b' \stackrel{?}{=} b$.

$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ except that we use a dictionary H and store the sampled values from random oracle queries in this dictionary. Changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $(N, S, \mathbb{A}, C)$. Initiate an empty dictionary H. Then proceed exactly as $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$.

Random Oracle Queries. Given a query $(\mathsf{sd}, i) \in \{0,1\}^\lambda, \{0,1\}^\lambda$ for $\mathcal{H}_\rho$ or $\mathcal{H}_{\mathsf{sh}}$, if the query is for $\mathcal{H}_\rho$, check if there is an entry for (ρ, sd, i) in H. If yes, return that value. Otherwise, sample $\beta \leftarrow \{0,1\}$ and set $\mathsf{H}[(\rho, \mathsf{sd}, i)] = \beta$ and send β.

Otherwise, if the query is for $\mathcal{H}_{\mathsf{sh}}$, check if there is an entry for $(\mathsf{sh}, \mathsf{sd}, i)$ in H. If yes, return that value. Otherwise, sample $\mathsf{sh} \leftarrow \mathbb{Z}_q^n$ store $\mathsf{H}[(\mathsf{sh}, \mathsf{sd}, i)] = \mathsf{sh}$ and send sh.

Guess Phase. $\mathcal{A}$ outputs its guess b'. Output $b' \stackrel{?}{=} b$.

$\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ except that during setup phase, we will randomly sample $(r_1, \ldots, r_\rho) \leftarrow \{0,1\}^\rho$, $(\mathsf{sh}_i)_{i \in A}$ and program these is H using $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\rho$.

$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ except that we sample $\mathsf{pk}_i = (i, \mathbf{t}_i)$ where $\mathbf{t}_i \leftarrow \mathbb{Z}_q^n$ for each $i \in S \setminus C$.

$\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ except that we will change the way $\mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}$ generated. Specifically, we will use C to generate a maximally invalid set A' and sample random shares $\mathsf{sh}_j \leftarrow \mathbb{Z}_q^n$ for $j \in A'$. We will also sample a random secret $\mathbf{p} \leftarrow \mathbb{Z}_q^n$. Then, $(\mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) \leftarrow \mathsf{ProgShare}_2(\mathbb{A}, A', (\mathsf{sh}_j)_{j \in A'}, \mathbf{p})$. We will then find A similarly to $\mathsf{PreProcess}$ and program these values.

$\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ except that we will use $\mathsf{ER.Explain}$ to generate $(r'_1, \ldots, r'_\rho)$ using T'. We will define T' later.

$\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$ except that we sample $(\mathbf{W}, \mathbf{D}, \mathbf{B})$ along with $\mathbf{T_B}$ using $\mathsf{MCOM.}\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma)$.

$\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$ except that we sample $\mathbf{C}_0 \leftarrow \mathbb{Z}_q^{n \times m}$ and set $\mathbf{Z}_0 \leftarrow \mathsf{SamplePre}(\mathbf{B}, \mathbf{T_B}, -\mathbf{C}_0 \mathbf{V}_{|S|}, \sigma_{\mathsf{er}})$. We write the full hybrid for clarity.

Setup Phase. $\mathcal{A}$ sends $(N, S, \mathbb{A}, C)$. Initiate an empty dictionary H. Do the following.

1. Sample $(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \mathsf{MCOM.}\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma)$.
2. For each $i \in C$, sample $\mathbf{r}_i \leftarrow \{\pm 1\}^{4m^5}$ and set $\mathsf{pk}_i := (i, \mathbf{B}\mathbf{r}_i), \mathsf{sk}_i := \mathbf{r}_i$. For each $i \in S \setminus C$, sample $\mathbf{t}_i \leftarrow \mathbb{Z}_q^n$ and set $\mathsf{pk}_i = (i, \mathbf{t}_i)$.
3. Sample $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$.
4. Sample $\mathbf{C}_0 \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{Z}_0 \leftarrow \mathsf{SamplePre}(\mathbf{B}, \mathbf{T_B}, -\mathbf{C}_0 \mathbf{V}_{|S|}, \sigma_{\mathsf{er}})$.
5. Sample $r' \leftarrow \mathsf{ER.Explain}(1^\lambda, 1^{T'}, \mathbf{C}_0, \mathbf{Z}_0, \sigma_{\mathsf{er}})$ and set $\mathsf{H}[(\rho, \mathsf{sd}_\rho, k)] = r'_k$ for each $k \in [\rho]$ and $\rho = \mathsf{poly}(\lambda, |S|)$.
6. Use C to generate a maximally invalid set A' and sample random shares $\mathsf{sh}_j \leftarrow \mathbb{Z}_q^n$ for $j \in A'$. Also sample a random secret $\mathbf{p} \leftarrow \mathbb{Z}_q^n$. Then, $(\mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) \leftarrow \mathsf{ProgShare}_2(\mathbb{A}, A', (\mathsf{sh}_j)_{j \in A'}, \mathbf{p})$.
7. Now, let $\widetilde{A}$ be the minimally valid party set for $\mathbb{A}$ in S. Set $j \in \widetilde{A}$, $\mathsf{H}[(\mathsf{sh}, \mathsf{sd}_{\mathsf{sh}}, j)] = \mathsf{sh}_j$.
8. Compute matrix $\mathbf{X} = [\mathbf{x}_1 \mid \ldots \mid \mathbf{x}_{|S|}] \in \mathbb{Z}_q^{n \times |S|}$ with $\mathbf{x}_j = \mathbf{t}_j + \mathsf{sh}_j$.
9. Compute $\mathbf{C} = \mathsf{MCOM.Com}(\mathbf{X})$.

10. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{R_C} \leftarrow \{\pm 1\}^{4m^5 \times m}, \mathbf{r_p} \leftarrow \{\pm 1\}^{4m^5}$.
11. Compute $\mathbf{c}_1^\top = \mathbf{s}^\top \mathbf{B} + \mathbf{e}^\top, \mathbf{c}_2^\top = \mathbf{s}^\top(\mathbf{C}_0 + \mathbf{C}) + \mathbf{e}^\top \mathbf{R_C}, c_3 = \mathbf{s}^\top \mathbf{p} + \mathbf{e}^\top \mathbf{r_p} + \lfloor q/2 \rfloor b$.
12. Set $\mathsf{ct} := (\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}})$.

Send $(\mathsf{crs} := (N, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}}), \{\mathsf{pk}_i\}_{i \in S}, \{\mathsf{sk}_j\}_{j \in C}, \mathsf{aux} := (\mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}, \mathbf{p}), \mathsf{ct})$ to $\mathcal{A}$. Here, $\mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}}$ are handles to random oracles.

Random Oracle Queries. Given a query $(\mathsf{sd}, i) \in \{0,1\}^\lambda, \{0,1\}^\lambda$ for $\mathcal{H}_\rho$ or $\mathcal{H}_{\mathsf{sh}}$, if the query is for $\mathcal{H}_\rho$, check if there is an entry for (ρ, sd, i) in H. If yes, return that value. Otherwise, sample $\beta \leftarrow \{0,1\}$ and set $\mathsf{H}[(\rho, \mathsf{sd}, i)] = \beta$ and send β.

Otherwise, if the query is for $\mathcal{H}_{\mathsf{sh}}$, check if there is an entry for $(\mathsf{sh}, \mathsf{sd}, i)$ in H. If yes, return that value. Otherwise, sample $\mathsf{sh} \leftarrow \mathbb{Z}_q^n$ store $\mathsf{H}[(\mathsf{sh}, \mathsf{sd}, i)] = \mathsf{sh}$ and send sh.

Guess Phase. $\mathcal{A}$ outputs its guess b'. Output $b' \stackrel{?}{=} b$.

$\mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$ except that we delay sampling $(\mathbf{C}_0, \mathbf{Z}_0)$ and set $\mathbf{C}_0$ as $\mathbf{BR_C} - \mathbf{C}$ and sample $\mathbf{Z}_0 \leftarrow \mathsf{SamplePre}(\mathbf{B}, \mathbf{T_B}, \mathbf{X} - \mathbf{B}(\mathbf{R_C V}_{|S|} + \mathbf{Z_X}), \sigma_{\mathsf{er}})$ where $\mathbf{Z_X} = \mathsf{MCOM.Open}(\mathbf{X})$ and $\mathbf{V}_{|S|} = \mathsf{MCOM.Verify}(1^{|S|})$. Note that $\mathbf{C}_0 \mathbf{V}_{|S|} = -\mathbf{B}\mathbf{Z}_0$.

$\mathbf{Hyb}_9^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)$ except that we sample $\mathbf{Z}_0 = [\mathbf{z}_{0,1} \mid \ldots \mid \mathbf{z}_{0,|S|}] \in \mathbb{Z}_q^{m \times |S|}$ as follows.

$$\mathbf{z}_{0,j} \leftarrow \begin{cases} \mathsf{SamplePre}(\mathbf{B}, \mathbf{T_B}, \mathbf{t}_j + \mathsf{sh}_j, \sigma_{\mathsf{er}}) - \mathbf{R_C v}_{|S|,j} - \mathbf{z_X}_{,j} & \text{if } j \in S \setminus C \\ \mathsf{SamplePre}(\mathbf{B}, \mathbf{T_B}, \mathsf{sh}_j, \sigma_{\mathsf{er}}) + \mathbf{r}_j - \mathbf{R_C v}_{|S|,j} - \mathbf{z_X}_{,j} & \text{if } j \in C \end{cases}$$

$\mathbf{Hyb}_{10}^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_9^{\mathcal{A}}(1^\lambda)$ except that we set $\mathbf{t}_j = \mathbf{B}\mathbf{d}'_j - \mathsf{sh}_j$ for $j \in S \setminus C$ and $\mathsf{sh}_j = \mathbf{B}\mathbf{d}'_j$ where every $\mathbf{d}'_j \leftarrow \mathcal{D}_{\mathbb{Z},\sigma_{\mathsf{er}}}^{4m^5}$. In addition, we set

$$\mathbf{z}_{0,j} = \begin{cases} \mathbf{d}'_j - \mathbf{R_C v}_{|S|,j} - \mathbf{z_X}_{,j} & \text{if } j \in S \setminus C \\ \mathbf{d}'_j + \mathbf{r}_j - \mathbf{R_C v}_{|S|,j} - \mathbf{z_X}_{,j} & \text{if } j \in C \end{cases}$$

where $\mathbf{v}_{|S|,j} \in \mathbb{Z}_q^{4m^5}$ is the column corresponding to i_j-th element in S from $\mathbf{V}_{|S|}$ and $\mathbf{z_X}_{,j}$ is the column corresponding to i_j in $\mathbf{Z_X}$.

$\mathbf{Hyb}_{11}^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_{10}^{\mathcal{A}}(1^\lambda)$ except that we sample $(\mathbf{W}, \mathbf{D}, \mathbf{B})$ from $\mathsf{MCOM.Setup}$.

$\mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_{11}^{\mathcal{A}}(1^\lambda)$ except that we set $\mathbf{p} = \mathbf{B}\mathbf{r_p}, \mathbf{r_p} \leftarrow \{\pm 1\}^{4m^5}$.

$\mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)$ except that we sample $\mathbf{c}_1^\top \leftarrow \mathbb{Z}_q^m$, set $\mathbf{c}_2^\top = \mathbf{c}_1^\top \mathbf{R_C}, c_3 = \mathbf{c}_1^\top \mathbf{r_p}$. We write the full hybrid for clarity.

Setup Phase. $\mathcal{A}$ sends $(N, \mathbb{A}, S, C)$. Initiate an empty dictionary H. Do the following.

1. Sample $(\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{MCOM.Setup}(1^n, 1^m, q, \sigma)$.
2. Sample $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$.
3. Use C to generate a maximally invalid set A' and sample random shares $\mathbf{d}'_j \leftarrow \mathcal{D}_{\mathbb{Z},\sigma_{\mathsf{er}}}^{4m^5}$ and $\mathsf{sh}_j = \mathbf{B}\mathbf{d}'_j$ for $j \in A'$. Also sample a random $\mathbf{r_p} \leftarrow \{\pm 1\}^{4m^5}$ and $\mathbf{p} = \mathbf{B}\mathbf{r_p}$. Then, $(\mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) \leftarrow \mathsf{ProgShare}_2(\mathbb{A}, A', (\mathsf{sh}_j)_{j \in A'}, \mathbf{p})$.

4. Now, let $\widetilde{A}$ be the minimally valid party set for $\mathbb{A}$ in S. Set $j \in \widetilde{A}$, $\mathsf{H}[(\mathsf{sh}, \mathsf{sd}_{\mathsf{sh}}, j)] = \mathsf{sh}_j$.
5. For each $i \in C$, sample $\mathbf{r}_i \leftarrow \{\pm 1\}^{4m^5}$ and set $\mathsf{pk}_i := (i, \mathbf{B}\mathbf{r}_i), \mathsf{sk}_i := \mathbf{r}_i$. For each $i \in S \setminus C$, sample $\mathbf{t}_i = \mathbf{B}\mathbf{d}'_i - \mathsf{sh}_i$ where $\mathbf{d}'_i \leftarrow \mathcal{D}^{4m^5}_{\mathbb{Z},\sigma_{\mathsf{er}}}$ and set $\mathsf{pk}_i = (i, \mathbf{t}_i)$.
6. Compute matrix $\mathbf{X} = [\mathbf{x}_1 \mid \ldots \mid \mathbf{x}_{|S|}] \in \mathbb{Z}_q^{n \times |S|}$ with $\mathbf{x}_j = \mathbf{t}_j + \mathsf{sh}_j$.
7. Compute $\mathbf{C} = \mathsf{MCOM.Com}(\mathbf{X}), \mathbf{Z}_{\mathbf{X}} = \mathsf{MCOM.Open}(\mathbf{X}), \mathbf{V}_{|S|} = \mathsf{MCOM.Verify}(1^{|S|})$.
8. Sample $\mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5 \times m}$, $\mathbf{C}_0 = \mathbf{B}\mathbf{R}_{\mathbf{C}} - \mathbf{C}, \mathbf{Z}_0 = [\mathbf{z}_{0,1} \mid \ldots \mid \mathbf{z}_{0,|S|}] \in \mathbb{Z}_q^{m \times |S|}$ as

$$\mathbf{z}_{0,j} = \begin{cases} \mathbf{d}'_j - \mathbf{R}_{\mathbf{C}}\mathbf{v}_{|S|,j} - \mathbf{z}_{\mathbf{X},j} & \text{if } j \in S \setminus C \\ \mathbf{d}'_j + \mathbf{r}_j - \mathbf{R}_{\mathbf{C}}\mathbf{v}_{|S|,j} - \mathbf{z}_{\mathbf{X},j} & \text{if } j \in C \end{cases}$$

9. Sample $r' \leftarrow \mathsf{ER.Explain}(1^\lambda, 1^{T'}, \mathbf{C}_0, \mathbf{Z}_0, \sigma_{\mathsf{er}})$ and set $\mathsf{H}[(\rho, \mathsf{sd}_\rho, k)] = r'_k$ for each $k \in [\rho]$ and $\rho = \mathsf{poly}(\lambda, |S|)$.
10. Sample $\mathbf{c}_1^\top \leftarrow \mathbb{Z}_q^m, \mathbf{c}_2^\top = \mathbf{c}_1^\top \mathbf{R}_{\mathbf{C}}, c_3 = \mathbf{c}_1^\top \mathbf{r}_{\mathbf{p}} + b$.
11. Set $\mathsf{ct} := (\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}})$.

Send $(\mathsf{crs} := (N, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}}), \{\mathsf{pk}_i\}_{i \in S}, \{\mathsf{sk}_j\}_{j \in C}, \mathsf{aux} := (\mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}, \mathbf{p}), \mathsf{ct})$ to $\mathcal{A}$. Here, $\mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}}$ are handles to random oracles.

Random Oracle Queries. Given a query $(\mathsf{sd}, i) \in \{0,1\}^\lambda, \{0,1\}^\lambda$ for $\mathcal{H}_\rho$ or $\mathcal{H}_{\mathsf{sh}}$, if the query is for $\mathcal{H}_\rho$, check if there is an entry for (ρ, sd, i) in H. If yes, return that value. Otherwise, sample $\beta \leftarrow \{0,1\}$ and set $\mathsf{H}[(\rho, \mathsf{sd}, i)] = \beta$ and send β.

Otherwise, if the query is for $\mathcal{H}_{\mathsf{sh}}$, check if there is an entry for $(\mathsf{sh}, \mathsf{sd}, i)$ in H. If yes, return that value. Otherwise, sample $\mathsf{sh} \leftarrow \mathbb{Z}_q^n$ store $\mathsf{H}[(\mathsf{sh}, \mathsf{sd}, i)] = \mathsf{sh}$ and send sh.

Guess Phase. $\mathcal{A}$ outputs its guess b'. Output $b' \stackrel{?}{=} b$.

$\mathbf{Hyb}^{\mathcal{A}}_{14}(1^\lambda)$. This is the same as $\mathbf{Hyb}^{\mathcal{A}}_{13}(1^\lambda)$ except that we sample $\mathbf{p} \leftarrow \mathbb{Z}_q^n$ and $c_3 \leftarrow \mathbb{Z}_q$. Note that in this hybrid all information about b is gone.

Indistinguishability of Hybrids. We will now show the indistinguishability of above hybrids using the following lemmas.

Lemma 6.8. $\Pr[1 \leftarrow \mathbf{Hyb}^{\mathcal{A}}_0(1^\lambda)] = \Pr[1 \leftarrow \mathbf{Hyb}^{\mathcal{A}}_1(1^\lambda)]$.

Proof. These are just semantic changes. So, lemma holds. □

Lemma 6.9. $\Pr[1 \leftarrow \mathbf{Hyb}^{\mathcal{A}}_1(1^\lambda)] = \Pr[1 \leftarrow \mathbf{Hyb}^{\mathcal{A}}_2(1^\lambda)]$.

Proof. The only difference is that we sample the values of $(r_1, \ldots, r_\rho), \{\mathsf{sh}_j\}_{j \in A}$ from random oracle query phase to setup phase. As this is nothing more than a semantic change, lemma holds. □

Lemma 6.10. There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}^{\mathcal{A}}_2(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}^{\mathcal{A}}_3(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. This follows from generalized leftover hash lemma (Lemma 3.6). Specifically, as $\mathbf{B}$ is statistically close to a uniformly random matrix, $(\mathbf{B}, \mathbf{B}\mathbf{r}_i)$ is statistically close to $(\mathbf{B}, \mathbf{t}_i)$ where $\mathbf{t}_i \leftarrow \mathbb{Z}_q^n$, $\mathbf{r}_i \leftarrow \{\pm 1\}^{4m^5}$. □

Lemma 6.11. $\Pr[1 \leftarrow \mathbf{Hyb}^{\mathcal{A}}_3(1^\lambda)] = \Pr[1 \leftarrow \mathbf{Hyb}^{\mathcal{A}}_4(1^\lambda)]$.

Proof. This follows from the mode indistinguishability property of Programmable LSSS (Definition 4.4). □

Lemma 6.12. If ER satisfies explainability property, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for any $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)]| \leq 1/T' + \mathsf{negl}(\lambda)$.

Proof. The only difference between these two hybrids is that in $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$, we do $(\mathbf{C}_0, \mathbf{Z}_0) = \mathsf{ER.Sample}(1^\lambda, 1^{|S|}, \sigma_{\mathsf{er}}; (r_1, \ldots, r_\rho))$ and program $(r_1, \ldots, r_\rho)$ in $\mathcal{H}_\rho$. In $\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$, we do $(\mathbf{C}_0, \mathbf{Z}_0) \leftarrow \mathsf{ER.Sample}(1^\lambda, 1^{|S|}, \sigma_{\mathsf{er}})$, $(r'_1, \ldots, r'_\rho) \leftarrow \mathsf{ER.Explain}(1^\lambda, 1^{T'}, \mathbf{C}_0, \mathbf{Z}_0, \sigma_{\mathsf{er}})$ and program $(r'_1, \ldots, r'_\rho)$ in H_ρ. Thus, this lemma follows from explainability property of ER. □

Lemma 6.13. If MCOM satisfies mode indistinguishability property, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for any $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. This follows from mode indistinguishability property of MCOM. Also check Lemma A.7. □

Lemma 6.14. If ER satisfies randomness property, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for any $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. Note that as random coins used in ER.Sample aren't leaked in $\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda), \mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$, this follows from the randomness property of ER. □

Lemma 6.15. There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. This follows from generalized leftover hash lemma (Lemma 3.6). Specifically, as $\mathbf{B}$ is statistically close to random, $(\mathbf{B}, \mathbf{B}\mathbf{R}_\mathbf{C}, \mathbf{e}^\top \mathbf{R}_\mathbf{C})$ is statistically close to $(\mathbf{B}, \mathbf{C}_0, \mathbf{e}^\top \mathbf{R}_\mathbf{C})$ where $\mathbf{R}_\mathbf{C} \leftarrow \{\pm 1\}^{4m^5 \times m}, \mathbf{C}_0 \leftarrow \mathbb{Z}_q^{n \times m}$. □

Lemma 6.16. There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_9^{\mathcal{A}}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. This follows from Lemma A.9 as $\left\|\mathbf{r}_j - \mathbf{R}_\mathbf{C}\mathbf{v}_{|S|,j} - \mathbf{z}_{\mathbf{X},j}\right\| \leq O(\|\mathbf{z}_{\mathbf{X},j}\|) = O(\sigma m^{15/2} \log q \log N)$ and $\sigma_{\mathsf{er}} > \lambda^{\omega(1)} O(\sigma m^{13} \log q \log N)$, the lemma holds. □

Lemma 6.17. There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_9^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_{10}^{\mathcal{A}}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. This follows from Lemma A.8. Specifically, for $j \in C$, $(\mathbf{B}, \mathsf{SamplePre}(\mathbf{B}, \mathbf{T}_\mathbf{B}, \mathsf{sh}_j, \sigma_{\mathsf{er}}), \mathsf{sh}_j)$ is statistically close to $(\mathbf{B}, \mathbf{d}'_j, \mathbf{B}\mathbf{d}'_j)$ where $\mathbf{d}'_j \leftarrow \mathcal{D}_{\mathbb{Z}, \sigma_{\mathsf{er}}}^{4m^5}$ and $\mathsf{sh}_j \leftarrow \mathbb{Z}_q^n$.

Similarly, for $j \in S \setminus C$, $(\mathbf{B}, \mathsf{SamplePre}(\mathbf{B}, \mathbf{T}_\mathbf{B}, \mathbf{t}_j + \mathsf{sh}_j, \sigma_{\mathsf{er}}), \mathbf{T}_j, \mathsf{sh}_j)$ is statistically close to $(\mathbf{B}, \mathsf{SamplePre}(\mathbf{B}, \mathbf{T}_\mathbf{B}, \mathbf{t}'_j, \sigma_{\mathsf{er}}), \mathbf{t}'_j - \mathsf{sh}_j, \mathsf{sh}_j)$ where $\mathbf{t}_j, \mathbf{t}'_j \leftarrow \mathbb{Z}_q^n$. Which is then statistically close to $(\mathbf{B}, \mathbf{d}'_j, \mathbf{B}\mathbf{d}'_j - \mathsf{sh}_j, \mathsf{sh}_j)$ by Lemma A.8 where $\mathbf{d}'_j \leftarrow \mathcal{D}_{\mathbb{Z}, \sigma_{\mathsf{er}}}^{4m^5}$. □

Lemma 6.18. If MCOM satisfies mode indistinguishability property, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for any $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_{10}^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_{11}^{\mathcal{A}}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. This follows from mode indistinguishability property of MCOM. Also check Lemma A.7. □

Lemma 6.19. There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_{11}^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. This follows from generalized leftover hash lemma (Lemma 3.6). Specifically, since $\mathbf{B}$ is statistically close to random, $(\mathbf{B}, \mathbf{B}\mathbf{r}_\mathbf{p}, \mathbf{e}^\top \mathbf{r}_\mathbf{p})$ is statistically close to $(\mathbf{B}, \mathbf{p}, \mathbf{e}^\top \mathbf{r}_\mathbf{p})$. □

Lemma 6.20. Assuming the hardness of $2m^2$-decomposed LWE assumption (Assumption 3.5), for any PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. Assume that there exists an adversary $\mathcal{A}$ such that $|\Pr[1 \leftarrow \mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)]| = \epsilon(\lambda)$ for some non-negligible function $\epsilon(\cdot)$. Then, we will construct an adversary $\mathcal{B}$ that breaks the $2m^2$-decomposed assumption with non-negligible probability.

$\mathcal{B}$ receives $(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{c}^\top)$ from challenger. It then interacts with $\mathcal{A}$ and receives $(N, S, \mathbb{A}, C)$. $\mathcal{B}$ initiates empty dictionary H, samples $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$. Using C, $\mathcal{B}$ generates a maximally invalid set A' and samples $\mathsf{sh}_j = \mathbf{B}\mathbf{d}'_j$, $\mathbf{d}'_j \leftarrow \mathcal{D}_{\mathbb{Z},\sigma_{\mathsf{er}}}^{4m^5}$ for $j \in A'$. It also samples $\mathbf{p} = \mathbf{B}\mathbf{r}_{\mathbf{p}}, \mathbf{r}_{\mathbf{p}} \leftarrow \{\pm 1\}^{4m^5}$. Using these values, it generates $\mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}$ by $\mathsf{ProgShare}_2$. Then, $\mathcal{B}$ generates minimally valid party set for $\mathbb{A}$ in S, $\widetilde{A}$ using procedure from $\mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)$. It then programs $\mathsf{H}[(\mathsf{sh}, \mathsf{sd}_{\mathsf{sh}}, j)] = \mathsf{sh}_j$ for $j \in \widetilde{A}$.

$\mathcal{B}$ now sample $\mathsf{pk}_i = (i, \mathbf{B}\mathbf{d}_i - \mathsf{sh}_i), \mathbf{d}_i \leftarrow \mathcal{D}_{\mathbb{Z},\sigma_{\mathsf{er}}}^{4m^5}$ for $i \in S \setminus C$ and $\mathsf{pk}_i = (i, \mathbf{B}\mathbf{r}_i), \mathsf{sk}_i = \mathbf{r}_i \leftarrow \{\pm 1\}^{4m^5 \times m}$. $\mathcal{B}$ computes $\mathbf{X} = [\mathbf{t}_j + \mathsf{sh}_j \text{ for } j \in S]$, $\mathbf{C} = \mathsf{MCOM.Com}(\mathbf{X})$, $\mathbf{Z}_{\mathbf{X}} = \mathsf{MCOM.Open}(\mathbf{X})$, and $\mathbf{V}_{|S|} = \mathsf{MCOM.Verify}(1^{|S|})$. $\mathcal{B}$ samples $\mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5 \times m}$, computes $\mathbf{C}_0 = \mathbf{B}\mathbf{R}_{\mathbf{C}} - \mathbf{C}$, $\mathbf{Z}_0$ as mentioned in $\mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)$. It will then sample r' using $\mathsf{ER.Explain}$ and T' and programs this in H.

$\mathcal{B}$ now computes $\mathsf{ct} := (p\mathbf{c}^\top, p\mathbf{c}^\top \mathbf{R}_{\mathbf{C}}, p\mathbf{c}^\top \mathbf{r}_{\mathbf{p}} + b, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}})$. It sends $\mathsf{crs} := (N, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}}), \{\mathsf{pk}_i\}_{i \in S}$, $\{\mathsf{sk}_j\}_{j \in C}, \mathsf{aux} = (\mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}}, \mathbf{p}), \mathsf{ct}$ to $\mathcal{A}$. Any random oracle queries are handled as mentioned in $\mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)$.

If challenger's coin toss is 0, then $\mathcal{B}$ behaves like $\mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)$ and if the coin toss is 1, then $\mathcal{B}$ behaves like $\mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)$. Note that multiplication by $p \in \mathbb{Z}_q$ is bijective for prime q. Thus, $\mathcal{B}$ can break the $2m^2$-decomposed LWE assumption with probability $\epsilon(\lambda)$. □

Lemma 6.21. There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any adversary $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_{14}^{\mathcal{A}}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$.

Proof. This follows from generalized leftover hash lemma (Lemma 3.6). Specifically, since $\mathbf{B}$ is statistically close to random,

$$\left\{\left(\begin{bmatrix}\mathbf{B}\\ \mathbf{c}_1^\top\end{bmatrix}, \begin{bmatrix}\mathbf{B}\\ \mathbf{c}_1^\top\end{bmatrix}\mathbf{r}_{\mathbf{p}}\right)\right\} \approx_s \left\{\left(\begin{bmatrix}\mathbf{B}\\ \mathbf{c}_1^\top\end{bmatrix}, \begin{bmatrix}\mathbf{p}\\ c_3\end{bmatrix}\right)\right\}$$

□

Deriving T'. Let b-$\mathbf{Hyb}_h^{\mathcal{A}}(1^\lambda)$ denote $\mathbf{Hyb}_h^{\mathcal{A}}(1^\lambda)$ for $h \in [0, 14]$ from above where we encrypt $b \in \{0,1\}$. Note that $\Pr[1 \leftarrow 0\text{-}\mathbf{Hyb}_{14}^{\mathcal{A}}(1^\lambda)] = \Pr[1 \leftarrow 1\text{-}\mathbf{Hyb}_{14}^{\mathcal{A}}(1^\lambda)]$. By triangle inequality, we have

$$\begin{aligned}|\Pr[1 \leftarrow 0\text{-}\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow 1\text{-}\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)]| &\leq \sum_{i=1}^{14} |\Pr[1 \leftarrow 0\text{-}\mathbf{Hyb}_{i-1}^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow 0\text{-}\mathbf{Hyb}_i^{\mathcal{A}}(1^\lambda)]| \\ &\quad + |\Pr[1 \leftarrow 1\text{-}\mathbf{Hyb}_{i-1}^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow 1\text{-}\mathbf{Hyb}_i^{\mathcal{A}}(1^\lambda)]| \\ &\leq 2/T' + \mathsf{negl}(\lambda)\end{aligned}$$

If there exists an adversary such that $|\Pr[1 \leftarrow 0\text{-}\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow 1\text{-}\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)]| = \epsilon(\lambda)$ for some non-negligible function $\epsilon(\cdot)$, then there exists a polynomial $T(\lambda)$ and infinite $\Lambda_{\mathcal{A}} \subseteq \mathbb{N}$ such that $\epsilon(\lambda) > 1/T(\lambda)$ for each $\lambda \in \Lambda_{\mathcal{A}}$. Then, we set $T'(\cdot) = 3T(\cdot)$. This would mean for each $\lambda \in \Lambda_{\mathcal{A}}$, $3/T'(\lambda) < \epsilon(\lambda) < 2/T'(\lambda) + \mathsf{negl}(\lambda)$ which is a contradiction. Thus, setting $T'(\cdot) = 3T(\cdot)$ suffices.

7 Monotone Policy Homomorphic Encryption with Silent Setup

7.1 Definition

Syntax. A monotone policy homomorphic encryption scheme with silent setup (SFHE) for a class of access policies $\mathbb{S}$, circuit class $\mathcal{F}_{\ell,d} = \{\mathcal{F}_d : \{0,1\}^\ell \to \{0,1\}\}_{\ell \in \mathbb{N}, d \in \mathbb{N}}$ has the following polynomial time algorithms.

$\mathsf{GSetup}(1^\lambda, 1^\ell, 1^d, N) \to \mathsf{crs}$**.** The possibly randomized global setup algorithm takes as input security parameter λ, input length ℓ, maximum evaluation circuit depth d, number of users N, and outputs the common reference string crs. The following algorithms take crs implicitly.

$\mathsf{KGen}(i) \to (\mathsf{pk}_i, \mathsf{sk}_i)$**.** The randomized key generation algorithm takes as input $i \in [N]$, and outputs public-secret key pair $(\mathsf{pk}_i, \mathsf{sk}_i)$.

$\mathsf{Enc}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}, \mu) \to \mathsf{ct}$**.** The randomized encryption algorithm takes as input public keys for a users in set S, $\{\mathsf{pk}_i\}_{i\in S}$, access structure $\mathbb{A}$ for $|S|$ parties, message $\mu \in \{0,1\}^\ell$, and outputs ciphertext ct.

$\mathsf{Eval}(C, \mathsf{ct}) \to \widetilde{\mathsf{ct}}$**.** The deterministic evaluation algorithm takes as input circuit $C \in \mathcal{F}_{\ell,d}$, ciphertext ct, and outputs evaluated ciphertext $\widetilde{\mathsf{ct}}$.

$\mathsf{PartDec}(\{\mathsf{pk}_i\}_{i\in S}, (j, \mathsf{sk}_j), \widetilde{\mathsf{ct}}) \to \mathsf{dsh}_j$**.** The possibly randomized partial decryption algorithm takes as input public keys for users in S, $\{\mathsf{pk}_i\}_{i\in S}$, secret key for the j-th user sk_j, evaluated ciphertext $\widetilde{\mathsf{ct}}$, and outputs decryption share dsh_j.

$\mathsf{Recon}(\{\mathsf{dsh}_j\}_{j\in S'}, \mathbb{A}, \widetilde{\mathsf{ct}}) \to \mu'$**.** The deterministic reconstruction algorithm takes as input decryption shares from users in set S', $\{\mathsf{dsh}_j\}_{j\in S'}$, access structure $\mathbb{A}$, and outputs message $\mu' \in \{0,1\} \cup \{\perp\}$.

Definition 7.1 (SFHE)**.** An SFHE scheme $(\mathsf{GSetup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{PartDec}, \mathsf{Recon})$ is said to be an SFHE scheme for access policies $\mathbb{S}$, function class $\mathcal{F}_{\ell,d}$ if it satisfies the following properties.

Evaluation Correctness. For any $\lambda \in \mathbb{N}, N \le 2^\lambda, \ell = \mathsf{poly}(\lambda), d = \mathsf{poly}(\lambda)$, $\mu \in \{0,1\}^\ell$, monotone access policy $\mathbb{A} \in \mathbb{S}, S' \in \mathbb{A}, C \in \mathcal{F}_{\ell,d}$,

$$\Pr\left[\begin{array}{l} C(\mu) = \mathsf{Recon}(\{\mathsf{dsh}_j\}_{j\in S'}, \mathbb{A}) \\ \quad : \\ \end{array} \begin{array}{l} \mathsf{crs} \leftarrow \mathsf{GSetup}(1^\lambda, 1^\ell, 1^d, N), \\ \forall\ i \in S, (\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KGen}(i), \\ \mathsf{ct} \leftarrow \mathsf{Enc}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}, \mu), \\ \widetilde{\mathsf{ct}} \leftarrow \mathsf{Eval}(C, \mathsf{ct}), \forall\ j \in S', \\ \mathsf{dsh}_j \leftarrow \mathsf{PartDec}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}, (j, \mathsf{sk}_j), \widetilde{\mathsf{ct}}) \end{array}\right] = 1$$

Compactness. There exists a polynomial $\mathsf{poly}(\cdot,\cdot,\cdot,\cdot)$ such that for any $\lambda \in \mathbb{N}, N \le 2^\lambda, S \subseteq [N], \mathbb{A} \in \mathbb{S}$, $|\widetilde{\mathsf{ct}}| = o(|S|)\mathsf{poly}(\lambda, d, \ell, \log N)$.

Security. There exists a PPT simulator[4] $\mathsf{Sim} = (\mathsf{Enc\text{-}Sim}, \mathsf{PartDec\text{-}Sim})$ such that

- *Mode Indistinguishability:* For any admissible PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\left|\Pr\left[1 \leftarrow \mathcal{A}^{\mathcal{O}(\cdot,\cdot)}\left(\begin{array}{c}\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in S}, \\ \{\mathsf{sk}_j\}_{j\in C}, \mathsf{ct}\end{array}\right) : \begin{array}{l}(1^\ell, 1^d, N, \mathbb{A}, S, \mathcal{C}, \mu) \leftarrow \mathcal{A}(1^\lambda), \\ \mathsf{crs} \leftarrow \mathsf{GSetup}(1^\lambda, 1^\ell, 1^d, N), \\ \forall\ i \in S, (\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KGen}(i), \\ \mathsf{ct} \leftarrow \mathsf{Enc}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}, \mu)\end{array}\right] -$$
$$\Pr\left[1 \leftarrow \mathcal{A}^{\mathcal{O}'(\cdot,\cdot)}\left(\begin{array}{c}\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in S}, \\ \{\mathsf{sk}_j\}_{j\in C}, \mathsf{ct}\end{array}\right) : \begin{array}{l}(1^\ell, 1^d, N, \mathbb{A}, S, \mathcal{C}, \mu) \leftarrow \mathcal{A}(1^\lambda), \\ \mathsf{crs} \leftarrow \mathsf{GSetup}(1^\lambda, 1^\ell, 1^d, N), \\ \forall\ i \in S, (\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KGen}(i), \\ (\mathsf{ct}, \mathsf{st}) \leftarrow \mathsf{Enc\text{-}Sim}(\{\mathsf{pk}_i\}_{i\in S}, \mathbb{A}, \mathcal{C}, \mu)\end{array}\right]\Bigg| \le \mathsf{negl}(\lambda)$$

  A stateful PPT machine $\mathcal{A}$ is said to be admissible if $\mathcal{C} \subseteq S$ is a maximally invalid set and $\mathcal{O}$ is an oracle that takes as input $(C \in \mathcal{F}_{\ell,d}, j \in S \setminus \mathcal{C})$ and outputs dsh_j using sk_j for $\widetilde{\mathsf{ct}} = \mathsf{Eval}(C, \mathsf{ct})$. $\mathcal{O}'$ on the other hand, uses $\mathsf{st}, \{\mathsf{sk}_i\}_{i\in\mathcal{C}}, \mathsf{ct}, C, \mathsf{PartDec\text{-}Sim}$, and $C(\mu)$ to simulate dsh_j.

[4] We slightly refine the definition of [BGG+18] by formulating the semantic-security adversary in the ideal world. This is necessary for the definition to capture the intended notion of security.

- *Selective Security Under Simulation:* For any admissible PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$, the following probability is at most $1/2 + \mathsf{negl}(\lambda)$.

$$\Pr\left[b \leftarrow \mathcal{A}\left(\begin{array}{c} \mathsf{crs}, \{\mathsf{pk}_i\}_{i\in S}, \\ \{\mathsf{sk}_j\}_{j\in C}, \mathsf{ct}, \mathsf{st} \end{array} \right) \;:\; \begin{array}{l} (1^\ell, 1^d, N, S, \mathcal{C}, \mathbb{A}, \mu_0, \mu_1) \leftarrow \mathcal{A}(1^\lambda), \\ \mathsf{crs} \leftarrow \mathsf{GSetup}(1^\lambda, 1^\ell, 1^d, N), \\ \forall\, i \in S, (\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KGen}(i), \\ b \leftarrow \{0,1\}, \\ (\mathsf{ct}, \mathsf{st}) \leftarrow \mathsf{Enc\text{-}Sim}(\{\mathsf{pk}_i\}_{i\in S}, \mathcal{C}, \mathbb{A}, \mu_b) \end{array} \right]$$

  A PPT machine $\mathcal{A}$ is said to be admissible if $\mathcal{C} \subseteq S$ is a maximally invalid set, $|\mu_0| = |\mu_1| = \ell$.

Remark 7.2 (SFHE without maximal corruptions)**.** In the above definition, we consider an SFHE scheme where $\mathcal{A}$ declares a maximally invalid party set. This can be avoided by using key homomorphic PRFs as demonstrated by [EY24].

7.2 Abstracting Homomorphic Evaluation Procedure from [GSW13]

In this section, we prove the following lemma. Let $\mathsf{GSW.Eval}$ be the evaluation procedure using the celebrated 'approximate eigenvector' procedure from [GSW13].

Lemma 7.3 (Adapted from [GSW13])**.** Let n, m, q be LWE parameters where $m = \Omega(n \log q)$, q is an odd prime. For any $d \in \mathbb{N}$, all depth-d bounded circuits $C : \{0,1\}^\ell \to \{0,1\}$, there exists a deterministic procedure $\mathbf{H}_{C,\Psi} = \mathsf{EvalF}(C, (\Psi_i)_{i\in[\ell]})$ where each $\Psi_i \in \mathbb{Z}_q^{n \times n\log q}$ and $\Psi = [\Psi_1 \mid \ldots \mid \Psi_\ell]$ such that the following properties hold.

1. $\mathbf{H}_{C,\Psi} \in \mathbb{Z}_q^{\ell n \log q \times n \log q}$, $\|\mathbf{H}_{C,\mathsf{ct}}\| \le m^{O(d)}$.
2. $\Psi \mathbf{H}_{C,\mathsf{ct}} = \mathsf{GSW.Eval}(C, \Psi_1, \ldots, \Psi_\ell)$.

Proof. We will prove this for an addition gate, multiplication gate, and scalar multiplication. For any general circuit C, the property will then hold recursively layer-by-layer.

Addition Gate: Note that $\mathsf{GSW.Eval}(+, \Psi_1, \Psi_2) = \Psi_1 + \Psi_2$. Thus, we can have $\mathbf{H}_{+,\Psi} = \begin{bmatrix} \mathbf{I}_{n\log q} \\ \mathbf{I}_{n\log q} \end{bmatrix}$.

Multiplication Gate: Note that $\mathsf{GSW.Eval}(\times, \Psi_1, \Psi_2) = \Psi_1 \mathbf{G}^{-1}(\Psi_2)$. Thus, we can have $\mathbf{H}_{\times,\Psi} = \begin{bmatrix} \mathbf{G}^{-1}(\Psi_2) \\ \mathbf{0}_{n\log q} \end{bmatrix}$.

Scalar Multiplication Gate. Note that $\mathsf{GSW.Eval}(\alpha, \Psi) = \Psi \mathbf{G}^{-1}(\alpha \mathbf{G})$. Thus, we can have $\mathbf{H}_{\alpha,\Psi} = \mathbf{G}^{-1}(\alpha \mathbf{G})$. □

7.3 Construction

Parameters. We provide the parameters used in the construction below.

- n is chosen according to the lattice parameter convention of Section 6.2, so that $q/B_{\mathsf{LWE}} = 2^{\widetilde{O}(n^\epsilon)}$ for a fixed $\epsilon \in (0, 1/2)$. $\chi = \mathsf{poly}(\lambda)$ (Decomposed LWE security)
- $m = \Omega(n \log q)$ (Leftover hash lemma)
- $\sigma = \mathsf{poly}(\lambda, m)$ (Decomposed LWE security)
- $\sigma_{\mathsf{er}} = \lambda^{\omega(1)} \cdot O(\sigma m^{13} \cdot \log q \cdot \log N)$ (Security)
- $\widetilde{\chi} = \lambda^{\omega(1)} \cdot 4m^{13/2} \cdot \beta\chi$ (Simulation Security)
- $\chi' = \lambda^{\omega(1)} \cdot N \cdot \alpha\beta \cdot O(\sigma\sigma_{\mathsf{er}}\chi m^{O(d)} \cdot \log q \cdot \log N)$ (Simulation Security)
- Odd prime $q > O(N \cdot \alpha\beta \cdot m^{O(d)}\chi')$ (Correctness)

Construction 7.4. We provide the construction of an SFHE scheme (Definition 7.1) for access structures $\mathbb{S}$ that admit programmable (α, β)-scaled LSSS (Definition 4.4, Definition 4.1) and circuit class $\mathcal{F}_{\ell,d}$ using $2m^2$-decomposed LWE (Assumption 3.5), matrix commitment scheme MCOM (Definition 3.7), and explainable re-randomizer ER (Definition 3.8) as follows. Here, $\mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\rho$ are hash functions which are modeled as random oracles in the security proof.

$\mathsf{GSetup}(1^\lambda, 1^\ell, 1^d, N)$**.** Sample $(\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{MCOM.Setup}(1^n, 1^m, q, \sigma)$. Sample hash function $\mathcal{H}_\rho : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}$ and $\mathcal{H}_{\mathsf{sh}} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \mathbb{Z}_q^{n \times \ell m}$.

Output $\mathsf{crs} := (N, \ell, d, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}})$.

$\mathsf{KGen}(i)$**.** Sample $\mathbf{R}_i \leftarrow \{\pm 1\}^{4m^5 \times \ell m}$ and compute $\mathbf{T}_i = \mathbf{B}\mathbf{R}_i$. Output $(\mathsf{pk}_i, \mathsf{sk}_i) = ((i, \mathbf{T}_i), \mathbf{R}_i)$.

$\mathsf{Enc}(\{\mathsf{pk}_i\}_{i \in S}, \mathbb{A}, \mu)$**.** Parse pk_i as $(i, \mathbf{T}_i)$ for each $i \in S$, $\mu \in \{0,1\}^\ell$. If $\mathbf{1}_{|S|} \notin \mathbb{A}$, output $\perp$. Otherwise, do the following.

1. Sample $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{R_A} \leftarrow \{\pm 1\}^{4m^5 \times m}, \mathbf{R_C} \leftarrow \{\pm 1\}^{4m^5 \times m}$, and $\mathbf{R} \leftarrow \{\pm 1\}^{m \times \ell n \log q}$. Sample $\widetilde{\mathbf{e}}_1 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{4m^5}, \widetilde{\mathbf{e}}_2 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{m}, \widetilde{\mathbf{e}}_3 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{\ell m}$.
2. Compute $\Psi = \begin{bmatrix} \mathbf{AR} \\ \mathbf{s}^\top \mathbf{AR} + \mathbf{e}^\top \mathbf{R_A R} + \widetilde{\mathbf{e}}_3^\top \end{bmatrix} + \mu^\top \otimes \mathbf{G} \in \mathbb{Z}_q^{n+1 \times \ell m}$. Let $\overline{\Psi} \in \mathbb{Z}_q^{n \times \ell n \log q}$ denote the top n rows of Ψ and $\underline{\Psi}$ be the last row of Ψ.
3. Sample $\mathsf{sd_{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$.
4. Let $\rho = \mathsf{poly}(\lambda, |S|\ell m)$ be the length of randomness used by ER. Sample $r_k = \mathcal{H}_\rho(\mathsf{sd}_\rho, k)$ for each $k \in [\rho]$.
5. Compute $(\mathbf{C}_0, \mathbf{Z}_0) = \mathsf{ER.Sample}(1^\lambda, 1^{|S|\ell m}, \sigma_{\mathsf{er}}; (r_1, \ldots, r_\rho))$.
6. Let A be a maximally invalid set for $\mathbb{A}$. For each $j \in A$, sample $\widetilde{\mathsf{sh}}_j = \mathcal{H}_{\mathsf{sh}}(\mathsf{sd_{sh}}, j)$.
7. Compute $(\overline{\Psi}, \mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) = \mathsf{ProgShare}_2(\mathbb{A}, A, \{\widetilde{\mathsf{sh}}_j\}_{j \in A}, \overline{\Psi})$[5].
8. Compute matrix $\mathbf{X} = [\mathbf{X}_1 \mid \ldots \mid \mathbf{X}_{|S|}]$ with $\mathbf{X}_j = \mathbf{T}_{i_j} + \mathsf{sh}_j$.

Output $\mathsf{ct} = (\mathbf{s}^\top \mathbf{B} + \mathbf{e}^\top + \widetilde{\mathbf{e}}_1^\top, \mathbf{s}^\top(\mathbf{C}_0 + \mathsf{MCOM.Com}(\mathbf{X})) + \mathbf{e}^\top \mathbf{R_C} + \widetilde{\mathbf{e}}_2^\top, \Psi, \mathsf{sd}_\rho, \mathsf{sd_{sh}}) \in \mathbb{Z}_q^{1 \times (4m^5 + m)} \times \mathbb{Z}_q^{(n+1) \times \ell m} \times \{0,1\}^{2\lambda}$.

$\mathsf{Eval}(C, \mathsf{ct})$**.** If ct is $\perp$, output $\perp$, Otherwise, parse ct as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, \Psi, \mathsf{sd}_\rho, \mathsf{sd_{sh}})$.

Compute $\mathbf{H}_{C,\Psi} = \mathsf{EvalF}(C, \Psi)$.

Output $\widetilde{\mathsf{ct}} := (\mathsf{ct}, \mathbf{H}_{C,\Psi}) \in \mathbb{Z}_q^{1 \times (4m^5 + m)} \times \mathbb{Z}_q^{(n+1) \times \ell m} \times \{0,1\}^{2\lambda} \times \mathbb{Z}_q^{\ell m \times m}$.

$\mathsf{PartDec}(\{\mathsf{pk}_i\}_{i \in S}, \mathbb{A}, (j, \mathsf{sk}_j), \widetilde{\mathsf{ct}})$ If $\widetilde{\mathsf{ct}}$ is $\perp$, output $\perp$. Otherwise, parse pk_i as $(i, \mathbf{T}_i)$ for each $i \in S$. Parse sk_j as $\mathbf{R}_j \in \{\pm 1\}^{4m^5 \times \ell m}$. If $j \notin S$, abort and output $\perp$. Otherwise, parse $\widetilde{\mathsf{ct}}$ as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, \Psi, \mathsf{sd}_\rho, \mathsf{sd_{sh}}, \mathbf{H}_{C,\Psi})$ and perform the following steps.

1. Perform steps 4-8 from Enc using $\Psi, \mathsf{sd}_\rho, \mathsf{sd_{sh}}, \mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\rho$. From these, use $\mathbf{X}, \mathbf{Z}_0$.
2. Compute $\mathbf{V} = \mathsf{MCOM.Verify}(1^{|S|\ell m})$ and $\mathbf{Z_X} = \mathsf{MCOM.Open}(\mathbf{X})$.
3. Compute $\mathbf{c}_{exp}^\top = \mathbf{c}_2^\top \mathbf{V} + \mathbf{c}_1^\top(\mathbf{Z}_0 + \mathbf{Z_X})$.
4. Let $\mathbb{J} \subseteq [|S|\ell m]$ denote the indices for user j in $\mathbf{X}$. Let $\widetilde{\mathbf{c}}^\top \in \mathbb{Z}_q^{1 \times \ell m}$ be the elements in $\mathbb{J}$ locations of $\mathbf{c}_{exp}^\top$.
5. Sample $e' \leftarrow \mathcal{D}_{\mathbb{Z},\chi'}$.

[5]Recall that $\mathsf{ProgShare}_2$ is deterministic if a maximally invalid share set is provided.

Output $\mathsf{dsh}_j := \left(\left(\widetilde{\mathbf{c}}^\top - \mathbf{c}_1^\top \mathbf{R}_j\right) \mathbf{H}_{C,\Psi} \mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1}) + e', \Psi \mathbf{H}_{C,\Psi} \mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1})\right) \in \mathbb{Z}_q \times \mathbb{Z}_q^{n+1}$.

$\mathsf{Recon}(\{\mathsf{dsh}_j\}_{j\in S'}, \mathbb{A}, \widetilde{\mathsf{ct}})$. If ct is $\perp$ or any of $\{\mathsf{dsh}_j\}_{j\in S'}$ are $\perp$, output $\perp$. Otherwise, parse dsh_j as $(d_j, \widetilde{\Psi})$. If all $\widetilde{\Psi}$ are not the same, abort and output $\perp$. If $S' \notin \mathbb{A}$, abort and output $\perp$. Otherwise,

1. Compute coefficients $((\delta_j)_{j\in S'}, \gamma) = \mathsf{GetCoeffs}(\mathbb{A}, S')$.
2. Compute $c' = \sum_{j\in S'} \delta_j d_j$.
3. Compute $y = c' - \gamma \underline{\widetilde{\Psi}}$.

Output $\mu' := 0$ if $|y| \le q/4|\gamma_{S'}/\beta|$ otherwise $\mu' := 1$.

Evaluation Correctness. Similar to correctness of DPE from Construction 6.3, we have that $\widetilde{\mathbf{c}}_j^\top - \mathbf{c}_1^\top \mathbf{R}_j = \mathbf{s}^\top(\mathsf{sh}_j) + \widetilde{\mathbf{e}}_j^\top$ where $\left\|\widetilde{\mathbf{e}}_j^\top\right\| \le O(\sigma\sigma_{\mathsf{er}}\chi m^{13} \log q \log N)$ where sh_j is a share of $\overline{\Psi}$. By Lemma 7.3, each decryption share dsh_j is of the form $(\mathbf{s}^\top(\mathsf{sh}_j \mathbf{H}_{C,\Psi} \mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1})) + \widetilde{\mathbf{e}}_j^\top \mathbf{H}_{C,\Psi} \mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1}), \mathsf{GSW.Eval}(C, \Psi)\mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1}))$. Thus, by correctness of (α, β)-Scaled LSSS, we have that $c' \approx \gamma_{S'} \mathbf{s}^\top \overline{\Psi} \mathbf{H}_{C,\Psi} \mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1}) \approx \gamma_{S'} \mathbf{s}^\top \overline{\widetilde{\Psi}}$. Thus, $y \approx [\mathbf{s}^\top \mid -1]\widetilde{\Psi}$ which if $C(\mu) = 0$ is small as $|\gamma_{S'}| \le \beta$.

Compactness. Under the lattice parameter convention of Section 6.2, for a threshold policy with $\tau = \min(t^2, |S| - t)$, the public key, secret key, evaluated ciphertext, and partial decryption have size $O^\dagger_{\ell,d}(\tau^6)$, $O^\dagger_{\ell,d}(\tau^{18})$, $O^\dagger_{\ell,d}(\tau^{16})$, and $O^\dagger_{\ell,d}(\tau^3)$, respectively. Hence the evaluated ciphertext is sublinear in $|S|$ whenever $\tau = o(|S|^{1/16})$, and all parameters are sublinear whenever $\tau = o(|S|^{1/18})$.

Theorem 7.5. Assuming programmable (α, β)-Scaled LSSS scheme satisfies correctness and targeted scaled reconstruction, Construction 7.4 satisfies mode indistinguishability.

Theorem 7.6. If $2m^2$-decomposed LWE assumption holds (Assumption 3.5), MCOM satisfies mode indistinguishability (Definition 3.7), ER satisfies randomness and explainability properties (Definition 3.8), then Construction 7.4 satisfies Selective security under simulation (Definition 7.1) in random oracle model.

Instantiation and Corollaries. We instantiate our SFHE construction (Construction 7.4) for the two policy classes supported by our programmable (α, β)-Scaled LSSS.

Corollary 7.7 (SFHE for Threshold Access Structures)**.** Assuming the hardness of the $2m^2$-decomposed LWE assumption, there exists a secure SFHE scheme in the random oracle model supporting threshold access structures. For threshold t over $|S|$ users and $\tau = \min(t^2, |S| - t)$, the public keys, secret keys, evaluated ciphertexts, and partial decryption shares have size $O^\dagger_{\ell,d}(\tau^6)$, $O^\dagger_{\ell,d}(\tau^{18})$, $O^\dagger_{\ell,d}(\tau^{16})$, and $O^\dagger_{\ell,d}(\tau^3)$, respectively. In particular, all parameters are sublinear in $|S|$ whenever $\tau = o(|S|^{1/18})$.

Corollary 7.8 (SFHE for DNF Formulas)**.** Assuming the hardness of the $2m^2$-decomposed LWE assumption, there exists a secure SFHE scheme in the random oracle model supporting DNF formula access structures. Concretely, the evaluated ciphertext, public keys, secret keys, and partial decryption shares all have size $O^\dagger_{\ell,d}(1)$, where $O^\dagger_{\ell,d}$ hides $\mathsf{poly}(\lambda, \ell, d, \log N)$ factors. In particular, all parameters are fully compact, i.e., independent of $|S|$.

Similar to Remark 6.6, instantiating the matrix commitment in our SFHE construction from the ℓ-succinct LWE assumption (with a structured common reference string) yields the following improved parameters.

Corollary 7.9 (SFHE for Threshold Access Structures from ℓ-Succinct LWE)**.** Assuming the hardness of the ℓ-succinct LWE assumption, there exists a secure SFHE scheme in the random oracle model with a *structured common reference string*, supporting threshold access structures. For threshold t over $|S|$ users and $\tau = \min(t^2, |S| - t)$, and using the bounded-integer representation of the evaluation matrix, the public keys, secret keys, and evaluated ciphertexts have size $O^\dagger_{\ell,d}(\tau^6)$, while the partial decryption shares have size $O^\dagger_{\ell,d}(\tau^3)$. In particular, all parameters are sublinear in $|S|$ whenever $\tau = o(|S|^{1/6})$.

7.4 Security Analysis

We prove Theorem 7.5 and Theorem 7.6. Before proceeding to the proofs, we define the simulator algorithms below.

$\mathsf{Enc\text{-}Sim}(\{\mathsf{pk}_i\}_{i\in S}, \mathcal{C}, \mathbb{A}, \mu)$**.** Parse pk_i as $(i, \mathbf{T}_i)$ for each $i \in S$, $\mu \in \{0,1\}^\ell$. If $\mathbf{1}_{|S|} \notin \mathbb{A}$, output $\perp$. Otherwise, do the following.

1. If the threshold is sublinear then compute $((\delta_{T,i})_{i\in\mathcal{C}\cup\{0\}}, \gamma_\mathcal{C}) = \mathsf{GetTargetedCoeffs}(\mathbb{A}, \mathcal{C}, j)$ where $T = \mathcal{C} \cup \{j\}$ for some arbitrary $j \in [N] \setminus \mathcal{C}$. Then let $\theta = \gamma_\mathcal{C}$.
2. If the threshold is linear-minus-sublinear then let $\theta = \prod_{i\in[N]\setminus\mathcal{C}} \delta_{T_i,i}$ where $T_i = \mathcal{C} \cup \{i\}$ and $\delta_{T_i,i}$ is computed by running $\mathsf{SS.GetCoeffs}(\mathbb{A}, T_i)$.
3. If $\mathbb{A}$ is a DNF formula, let $\theta = 1$.
4. Sample $\mathbf{A} \leftarrow \mathbb{Z}_q^{n\times m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{R_A} \leftarrow \{\pm1\}^{4m^5\times m}, \mathbf{R_C} \leftarrow \{\pm1\}^{4m^5\times m}$, and $\mathbf{R} \leftarrow \{\pm1\}^{m\times \ell n\log q}$. Sample $\widetilde{\mathbf{e}}_1 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{4m^5}, \widetilde{\mathbf{e}}_2 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{m}$, and $\widetilde{\mathbf{e}_3} \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{m}$.
5. Compute $\Psi = \begin{bmatrix} \mathbf{A} \\ \mathbf{s}^\top\mathbf{A} + \theta\mathbf{e}^\top\mathbf{R_A} + \widetilde{\mathbf{e}}_3^\top \end{bmatrix} \mathbf{R} + \mu^\top \otimes \mathbf{G} \in \mathbb{Z}_q^{n+1\times \ell m}$. Let $\overline{\Psi} \in \mathbb{Z}_q^{n\times \ell n\log q}$ denote the top n rows of Ψ and $\underline{\Psi}$ be the last row of Ψ.
6. Sample $\mathsf{sd_{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$.
7. Let $\rho = \mathsf{poly}(\lambda, |S|\ell m)$ be the length of randomness used by ER. Sample $r_k = \mathcal{H}_\rho(\mathsf{sd}_\rho, k)$ for each $k \in [\rho]$.
8. Compute $(\mathbf{C}_0, \mathbf{Z}_0) = \mathsf{ER.Sample}(1^\lambda, 1^{|S|\ell m}, \sigma_{\mathsf{er}}; (r_1, \ldots, r_\rho))$.
9. Let A be a maximally invalid set for $\mathbb{A}$. For each $j \in A$, sample $\widetilde{\mathsf{sh}}_j = \mathcal{H}_{\mathsf{sh}}(\mathsf{sd_{sh}}, j)$.
10. Compute $(\overline{\Psi}, \mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) = \mathsf{ProgShare}_2(\mathbb{A}, A, \{\widetilde{\mathsf{sh}}_j\}_{j\in A}, \overline{\Psi})$.
11. Compute matrix $\mathbf{X} = [\mathbf{X}_1 \mid \ldots \mid \mathbf{X}_{|S|}]$ with $\mathbf{X}_j = \mathbf{T}_{i_j} + \mathsf{sh}_j$.

Set $\mathsf{ct} = (\mathbf{s}^\top\mathbf{B} + \theta\mathbf{e}^\top + \widetilde{\mathbf{e}}_1^\top, \mathbf{s}^\top(\mathbf{C}_0 + \mathsf{MCOM.Com}(\mathbf{X})) + \theta\mathbf{e}^\top\mathbf{R_C} + \widetilde{\mathbf{e}}_2^\top, \Psi, \mathsf{sd}_\rho, \mathsf{sd_{sh}})$ and $\mathsf{st} := (\widetilde{\mathbf{e}}_1^\top, \widetilde{\mathbf{e}}_2^\top, \widetilde{\mathbf{e}}_3^\top)$. Output $(\mathsf{ct}, \mathsf{st})$.

$\mathsf{PartDec\text{-}Sim}(\mathsf{st}, \{\mathsf{sk}_i\}_{i\in\mathcal{C}}, \mathbb{A}, j, \mathsf{ct}, C, y)$**.** Given $\mathsf{st} := (\widetilde{\mathbf{e}}_1^\top, \widetilde{\mathbf{e}}_2^\top, \widetilde{\mathbf{e}}_3^\top)$, $\{\mathbf{R}_i\}_{i\in\mathcal{C}}$, $j \notin \mathcal{C}$, ciphertext $\mathsf{ct} = (\mathbf{c}_1^\top, \mathbf{c}_2^\top, \Psi, \mathsf{sd}_\rho, \mathsf{sd_{sh}})$, $C \in \mathcal{F}_{\ell,d}$, and $y \in \{0,1\}$, do the following.

1. Compute $\mathbf{H}_{C,\Psi} = \mathsf{EvalF}(C, \Psi)$.
2. Compute $\widetilde{\Psi} = \Psi\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta\rfloor \mathbb{1}_{n+1,n+1})$.
3. Perform steps 4-8 from Enc algorithm using $\Psi, \mathsf{sd}_\rho, \mathsf{sd_{sh}}, \mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\rho$. From these, use $\mathbf{X}, \mathbf{Z}_0$. Compute $\mathbf{V} = \mathsf{MCOM.Verify}(1^{|S|\ell m})$ and $\mathbf{Z_X} = \mathsf{MCOM.Open}(\mathbf{X})$.
4. Compute $\mathbf{c}_{\text{exp}}^\top = (\mathbf{c}_2^\top - \widetilde{\mathbf{e}}^\top)\mathbf{V} + (\mathbf{c}_1^\top - \widetilde{\mathbf{e}}_1)(\mathbf{Z}_0 + \mathbf{Z_X})$.
5. Let $\{\widetilde{\mathbf{c}}_i^\top\}_{i\in\mathcal{C}}$ be the entries corresponding to corrupt parties $i \in \mathcal{C}$ in $\mathbf{c}_{exp}^\top$.
6. If the threshold is sublinear, compute $((\delta_{T,i})_{i\in\mathcal{C}\cup\{0\}}, \gamma_\mathcal{C}) = \mathsf{GetTargetedCoeffs}(\mathbb{A}, \mathcal{C}, j)$ where $T = \mathcal{C} \cup \{j\}$.
   - Compute $u = \sum_{i\in\mathcal{C}\cup\{0\}} \delta_{T,i} \left(\widetilde{\mathbf{c}}_i^\top - (\mathbf{c}_1^\top - \widetilde{\mathbf{e}}_1)\mathbf{R}_i)\right) \mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta\rfloor \mathbb{1}_{n+1,n+1})$ where $\mathbf{V}_i, \mathbf{Z}_{0,i}, \mathbf{Z_X}_{,i}$ are the corresponding sub-matrices of $\mathbf{V}, \mathbf{Z}_0, \mathbf{Z_X}$ for the i-th user respectively.
   - Compute $\mathsf{dsh}_j = \gamma_\mathcal{C}^{-1}\delta_0(\underline{\widetilde{\Psi}} - \lceil q/2\beta\rfloor C(\mu) - \widetilde{\mathbf{e}}_3\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta\rfloor \mathbb{1}_{n+1,n+1})) + \gamma_\mathcal{C}^{-1}u + e'$ where $e' \leftarrow \mathcal{D}_{\mathbb{Z},\chi'}$.
7. If the threshold is linear-minus-sublinear, compute $((\delta_{T,k})_{k\in\mathcal{C}\cup\{j\}}, \gamma_T) = \mathsf{SS.GetCoeffs}(\mathbb{A}, \mathcal{C}\cup\{j\})$ where $T = \mathcal{C} \cup \{j\}$.

- Compute $u = \sum_{i\in\mathcal{C}} \delta_{T,i}\left(\widetilde{\mathbf{c}}_i^\top - (\mathbf{c}_1^\top - \widetilde{\mathbf{e}}_1)\mathbf{R}_i\right)\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta\rfloor \mathbb{1}_{n+1,n+1})$ where $\mathbf{V}_i, \mathbf{Z}_{0,i}, \mathbf{Z}_{\mathbf{X},i}$ are the corresponding sub-matrices of $\mathbf{V}, \mathbf{Z}_0, \mathbf{Z}_{\mathbf{X}}$ for the i-th user respectively.
- Compute $\mathsf{dsh}_j = \delta_{T,j}^{-1}\gamma_T(\widetilde{\underline{\Psi}} - \lceil q/2\beta\rfloor C(\mu) - \widetilde{\mathbf{e}}_3\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta\rfloor \mathbb{1}_{n+1,n+1})) - \delta_{T,j}^{-1}u + e'$ where $e' \leftarrow \mathcal{D}_{\mathbb{Z},\chi'}$.

8. If $\mathbb{A}$ is a DNF formula, compute reconstruction coefficients $(w_k)_{k\in\mathcal{C}\cup\{j\}}$ where each $w_k \in \{0,1\}$.
   - Compute $u = \sum_{i\in\mathcal{C}} \omega i\left(\widetilde{\mathbf{c}}_i^\top - (\mathbf{c}_1^\top - \widetilde{\mathbf{e}}_1)\mathbf{R}_i\right)\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta\rfloor \mathbb{1}_{n+1,n+1})$ where $\mathbf{V}_i, \mathbf{Z}_{0,i}, \mathbf{Z}_{\mathbf{X},i}$ are the corresponding sub-matrices of $\mathbf{V}, \mathbf{Z}_0, \mathbf{Z}_{\mathbf{X}}$ for the i-th user respectively.
   - Compute $\mathsf{dsh}_j = (\widetilde{\underline{\Psi}} - \lceil q/2\beta\rfloor C(\mu) - \widetilde{\mathbf{e}}_3\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta\rfloor \mathbb{1}_{n+1,n+1})) - u + e'$ where $e' \leftarrow \mathcal{D}_{\mathbb{Z},\chi'}$.

Output dsh_j.

7.4.1 Proof of Theorem 7.5

We define the following hybrids.

$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$. This is the mode indistinguishability game using $\mathsf{Enc}, \mathsf{PartDec}$.

$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$. In this hybrid, during encryption, we will change $(\mathbf{e}_1^\top + \widetilde{\mathbf{e}}_1, \mathbf{e}_1^\top\mathbf{R}_{\mathbf{C}} + \widetilde{\mathbf{e}}_2, \mathbf{e}_1^\top\mathbf{R}_{\mathbf{A}}\mathbf{R} + \widetilde{\mathbf{e}}_3)$ as $(\theta\mathbf{e}_1^\top + \widetilde{\mathbf{e}}_1, \theta\mathbf{e}_1^\top\mathbf{R}_{\mathbf{C}} + \widetilde{\mathbf{e}}_2, \theta\mathbf{e}_1^\top\mathbf{R}_{\mathbf{A}}\mathbf{R} + \widetilde{\mathbf{e}}_3)$. Essentially this hybrid uses $\mathsf{Enc\text{-}Sim}$.

Setup Phase. $\mathcal{A}$ sends $(1^\ell, 1^d, N, S, \mathcal{C}, \mathbb{A}, \mu)$. Do the following.

1. Sample $(\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{MCOM.Setup}(1^n, 1^m, q, \sigma)$. Set $\mathsf{crs} := (N, \ell, d, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}})$ where $\mathcal{H}_\rho$ and $\mathcal{H}_{\mathsf{sh}}$ are handles for random oracles.
2. For each $i \in S$, sample $\mathbf{R}_i \leftarrow \{\pm 1\}^{4m^5\times \ell m}$ and set $(\mathsf{pk}_i, \mathsf{sk}_i) = ((i, \mathbf{B}\mathbf{R}_i), \mathbf{R}_i)$.
3. If the threshold is sublinear then compute $((\delta_{T,i})_{i\in\mathcal{C}\cup\{0\}}, \gamma_{\mathcal{C}}) = \mathsf{GetTargetedCoeffs}(\mathbb{A}, \mathcal{C}, j)$ where $T = \mathcal{C}\cup\{j\}$ for some arbitrary $j \in [N]\setminus\mathcal{C}$. Then let $\theta = \gamma_{\mathcal{C}}$.
4. If the threshold is linear-minus-sublinear then let $\theta = \prod_{i\in[N]\setminus\mathcal{C}} \delta_{T_i,i}$ where $T_i = \mathcal{C}\cup\{i\}$ and $\delta_{T_i,i}$ is computed by running $\mathsf{SS.GetCoeffs}(\mathbb{A}, T_i)$.
5. If $\mathbb{A}$ is a DNF formula, let $\theta = 1$.
6. Sample $\mathbf{A} \leftarrow \mathbb{Z}_q^{n\times m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{R}_{\mathbf{A}} \leftarrow \{\pm 1\}^{4m^5\times m}, \mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5\times m}$, and $\mathbf{R} \leftarrow \{\pm 1\}^{m\times \ell n\log q}$. Sample $\widetilde{\mathbf{e}}_1 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{4m^5}, \widetilde{\mathbf{e}}_2 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{m}$, and $\widetilde{\mathbf{e}}_3 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{m}$.
7. Compute $\Psi = \begin{bmatrix} \mathbf{A}\mathbf{R} \\ \mathbf{s}^\top\mathbf{A}\mathbf{R} + \theta\mathbf{e}^\top\mathbf{R}_{\mathbf{A}}\mathbf{R} + \widetilde{\mathbf{e}}_3^\top \end{bmatrix} + \mu_b^\top \otimes \mathbf{G} \in \mathbb{Z}_q^{n+1\times \ell m}$. Let $\overline{\Psi} \in \mathbb{Z}_q^{n\times \ell n\log q}$ denote the top n rows of Ψ and $\underline{\Psi}$ be the last row of Ψ.
8. Sample $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$.
9. Let $\rho = \mathsf{poly}(\lambda, |S|\ell m)$ be the length of randomness used by ER. Sample $r_k = \mathcal{H}_\rho(\mathsf{sd}_\rho, k)$ for each $k \in [\rho]$.
10. Compute $(\mathbf{C}_0, \mathbf{Z}_0) = \mathsf{ER.Sample}(1^\lambda, 1^{|S|\ell m}, \sigma_{\mathsf{er}}; (r_1, \ldots, r_\rho))$.
11. Let A be a maximally invalid set for $\mathbb{A}$. For each $j \in A$, sample $\widetilde{\mathsf{sh}}_j = \mathcal{H}_{\mathsf{sh}}(\mathsf{sd}_{\mathsf{sh}}, j)$.
12. Compute $(\overline{\Psi}, \mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) = \mathsf{ProgShare}_2(\mathbb{A}, A, \{\widetilde{\mathsf{sh}}_j\}_{j\in A}, \overline{\Psi})$.
13. Compute matrix $\mathbf{X} = [\mathbf{X}_1 \mid \ldots \mid \mathbf{X}_{|S|}]$ with $\mathbf{X}_j = \mathbf{T}_{i_j} + \mathsf{sh}_j$.

Set $\mathsf{ct} = (\mathbf{s}^\top\mathbf{B} + \theta\mathbf{e}^\top + \widetilde{\mathbf{e}}_1^\top, \mathbf{s}^\top(\mathbf{C}_0 + \mathsf{MCOM.Com}(\mathbf{X})) + \theta\mathbf{e}^\top\mathbf{R}_{\mathbf{C}} + \widetilde{\mathbf{e}}_2^\top, \Psi, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}})$ and $\mathsf{st} := (\widetilde{\mathbf{e}}_1^\top, \widetilde{\mathbf{e}}_2^\top, \widetilde{\mathbf{e}}_3^\top)$. Send $(\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in S}, \{\mathsf{sk}_i\}_{i\in\mathcal{C}}, \mathsf{ct})$ to $\mathcal{A}$.

Partial Decryption Queries. $\mathcal{A}$ sends circuit $C \in \mathcal{F}_{\ell,d}$ and $j \in S\setminus\mathcal{C}$. Compute $\mathbf{H}_{C,\Psi} = \mathsf{EvalF}(C, \Psi)$. Do the following.

1. Perform steps 9 to 13 from setup phase using $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_{\rho}, \mathcal{H}_{\mathsf{sh}}, \mathcal{H}_{\rho}, \Psi$ from ct. From these, use $\mathbf{X}, \mathbf{Z}_0$.
2. Compute $\mathbf{V} = \mathsf{MCOM.Verify}(1^{|S|\ell m})$ and $\mathbf{Z}_{\mathbf{X}} = \mathsf{MCOM.Open}(\mathbf{X})$.
3. Compute $\mathbf{c}_{exp}^{\top} = \mathbf{c}_2^{\top}\mathbf{V} + \mathbf{c}_1^{\top}(\mathbf{Z}_0 + \mathbf{Z}_{\mathbf{X}})$.
4. Let $\mathbb{J} \subseteq [|S|\ell m]$ denote the indices for user j in $\mathbf{X}$. Let $\widetilde{\mathbf{c}}^{\top} \in \mathbb{Z}_q^{1\times \ell m}$ be the elements in $\mathbb{J}$ locations of $\mathbf{c}_{exp}^{\top}$.
5. Sample $e' \leftarrow \mathcal{D}_{\mathbb{Z},\chi'}$.

Send $\mathsf{dsh}_j := \left(\left(\widetilde{\mathbf{c}}^{\top} - \mathbf{c}_1^{\top}\mathbf{R}_j\right)\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1}) + e', \Psi\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1})\right)$ to $\mathcal{A}$.

$\mathbf{Hyb}_2^{\mathcal{A}}(1^{\lambda})$. This is the mode indistinguishability game using $\mathsf{Enc\text{-}Sim}, \mathsf{PartDec\text{-}Sim}$.

Indistinguishability of Hybrids. We will now show the indistinguishability of above hybrids using the following lemmas.

Lemma 7.10. There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\mathcal{A}$, for any $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_0^{\mathcal{A}}(1^{\lambda})] - \Pr[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^{\lambda})]| \leq \mathsf{negl}(\lambda)$.

Proof. Note that the only difference between $\mathbf{Hyb}_0^{\mathcal{A}}(1^{\lambda})$ and $\mathbf{Hyb}_1^{\mathcal{A}}(1^{\lambda})$ is that we use errors $(\theta\mathbf{e}_1^{\top} + \widetilde{\mathbf{e}}_1, \theta\mathbf{e}_1^{\top}\mathbf{R}_{\mathbf{C}} + \widetilde{\mathbf{e}}_2, \theta\mathbf{e}_1^{\top}\mathbf{R}_{\mathbf{A}}\mathbf{R} + \widetilde{\mathbf{e}}_3)$ in $(\mathbf{c}_1^{\top}, \mathbf{c}_2^{\top}, \Psi)$ respectively in $\mathbf{Hyb}_1^{\mathcal{A}}(1^{\lambda})$ whereas we use $(\mathbf{e}_1^{\top} + \widetilde{\mathbf{e}}_1, \mathbf{e}_1^{\top}\mathbf{R}_{\mathbf{C}} + \widetilde{\mathbf{e}}_2, \mathbf{e}_1^{\top}\mathbf{R}_{\mathbf{A}}\mathbf{R} + \widetilde{\mathbf{e}}_3)$ in $\mathbf{Hyb}_0^{\mathcal{A}}(1^{\lambda})$. As $\widetilde{\chi} > \lambda^{\omega(1)} 4m^{13/2}\beta\chi$ is a wide enough gaussian distribution, θ is polynomially bounded in both sub-linear and linear-minus-sublinear regimes, these two distributions are statistically indistinguishable. □

Lemma 7.11 (Sublinear-threshold case.)**.** There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\mathcal{A}$, for any $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^{\lambda})] - \Pr[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^{\lambda})]| \leq \mathsf{negl}(\lambda)$ when threshold is sublinear.

Proof. Let $j \in S \setminus \mathcal{C}$ be an honest party. Then, by definition of targeted coefficients, we have that

$$\begin{aligned}
\mathbf{s}^{\top}\gamma_{\mathcal{C}}\mathsf{sh}_j\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1}) &= \left(\delta_0\mathbf{s}^{\top}\overline{\Psi} + \sum_{i\in\mathcal{C}}\delta_i\mathbf{s}^{\top}\mathsf{sh}_i\right)\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1}) \\
&= \delta_0\mathbf{s}^{\top}\widetilde{\Psi}_{top} + \left(\sum_{i\in\mathcal{C}}\delta_i\mathbf{s}^{\top}\mathsf{sh}_i\right)\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1}) \\
&= \delta_0(\underline{\widetilde{\Psi}} - \lceil q/2\beta \rfloor C(\mu) - \widetilde{\mathbf{e}}_3^{\top}\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1})) + \\
&\sum_{i\in\mathcal{C}}\delta_i\left(\widetilde{\mathbf{c}}_i^{\top} - (\mathbf{c}_1^{\top} - \widetilde{\mathbf{e}}_1)\mathbf{R}_i\right)\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1}) - \\
&\left(\delta_0\gamma_{\mathcal{C}}\widehat{e} + \gamma_{\mathcal{C}}\sum_{i\in\mathcal{C}}\delta_i\widehat{\mathbf{e}}_i\mathbf{H}_{C,\Psi}\mathbf{G}^{-1}(\lceil q/2\beta \rfloor \mathbb{1}_{n+1,n+1})\right)
\end{aligned}$$

where $\widehat{e}$ is error in $\underline{\widetilde{\Psi}}$ and $\widehat{\mathbf{e}}_i$ are errors in partial decryptions for each party $i \in \mathcal{C}$. Thus, as long as e' smudges the error above, both distributions are statistically close. □

Lemma 7.12 (Linear-minus-sublinear-case.)**.** There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\mathcal{A}$, for any $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^{\lambda})] - \Pr[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^{\lambda})]| \leq \mathsf{negl}(\lambda)$ when threshold is linear-minus-sublinear.

Proof. This is similar to proof of Lemma 7.11 as in $\mathsf{Enc\text{-}Sim}$, $\theta = \prod_{i\in S\setminus\mathcal{C}}\delta_{T_i,i}$ where $T_i = \mathcal{C}\cup\{i\}$. Therefore, for any $j \in S \setminus \mathcal{C}$, $\delta_{T_j,j}^{-1}\theta$ ecan be bounded. □

Lemma 7.13. There exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\mathcal{A}$, for any $\lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^{\lambda})] - \Pr[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^{\lambda})]| \leq \mathsf{negl}(\lambda)$ when $\mathbb{A}$ is a DNF formula.

Proof. As DNF formula has $\{0,1\}$ reconstruction coefficients, there are invertible. Moreover as $\mathcal{C}$ corresponds to a maximally invalid share set, coefficient for $j \in S \setminus \mathcal{C}$ must be 1 as this row yields a new dimension in the row span for $\mathbb{A}$. Thus, we can write the equation like in $\mathsf{PartDec\text{-}Sim}$. □

7.4.2 Proof of Theorem 7.6

We define the following hybrids.

$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$. This is the Selective security under simulation game from Definition 7.1 with $b = 0/1$.

Setup Phase. $\mathcal{A}$ sends $(1^\ell, 1^d, N, S, \mathcal{C}, \mathbb{A}, \mu_0, \mu_1)$. Do the following.

1. Sample $(\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{MCOM.Setup}(1^n, 1^m, q, \sigma)$. Set $\mathsf{crs} := (N, \ell, d, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}})$ where $\mathcal{H}_\rho$ and $\mathcal{H}_{\mathsf{sh}}$ are handles for random oracles.
2. For each $i \in S$, sample $\mathbf{R}_i \leftarrow \{\pm 1\}^{4m^5 \times \ell m}$ and set $(\mathsf{pk}_i, \mathsf{sk}_i) = ((i, \mathbf{B}\mathbf{R}_i), \mathbf{R}_i)$.
3. Compute $((\delta_i)_{i\in S}, \gamma_{\mathcal{C}}) \leftarrow \mathsf{GetTargetedCoeffs}(\mathbb{A}, \mathcal{C})$.
4. Sample $\mathbf{A} \leftarrow \mathbb{Z}_q^{n\times m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{R}_{\mathbf{A}} \leftarrow \{\pm 1\}^{4m^5\times m}, \mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5\times m}$, and $\mathbf{R} \leftarrow \{\pm 1\}^{m\times \ell n \log q}$. Sample $\widetilde{\mathbf{e}}_1 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{4m^5}, \widetilde{\mathbf{e}}_2 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{m}$, and $\widetilde{\mathbf{e}}_3 \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\chi}}^{m}$.
5. Compute $\Psi = \begin{bmatrix} \mathbf{A}\mathbf{R} \\ \mathbf{s}^\top \mathbf{A}\mathbf{R} + \gamma_{\mathcal{C}} \mathbf{e}^\top \mathbf{R}_{\mathbf{A}} \mathbf{R} + \widetilde{\mathbf{e}}_3^\top \end{bmatrix} + \mu_b^\top \otimes \mathbf{G} \in \mathbb{Z}_q^{n+1\times \ell m}$. Let $\overline{\Psi} \in \mathbb{Z}_q^{n\times \ell n \log q}$ denote the top n rows of Ψ and $\underline{\Psi}$ be the last row of Ψ.
6. Sample $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$.
7. Let $\rho = \mathsf{poly}(\lambda, |S|\ell m)$ be the length of randomness used by ER. Sample $r_k = \mathcal{H}_\rho(\mathsf{sd}_\rho, k)$ for each $k \in [\rho]$.
8. Compute $(\mathbf{C}_0, \mathbf{Z}_0) = \mathsf{ER.Sample}(1^\lambda, 1^{|S|\ell m}, \sigma_{\mathsf{er}}; (r_1, \ldots, r_\rho))$.
9. Let A be a maximally invalid set for $\mathbb{A}$. For each $j \in A$, sample $\widetilde{\mathsf{sh}}_j = \mathcal{H}_{\mathsf{sh}}(\mathsf{sd}_{\mathsf{sh}}, j)$.
10. Compute $(\overline{\Psi}, \mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) = \mathsf{ProgShare}_2(\mathbb{A}, A, \{\widetilde{\mathsf{sh}}_j\}_{j\in A}, \overline{\Psi})$.
11. Compute matrix $\mathbf{X} = [\mathbf{X}_1 \mid \ldots \mid \mathbf{X}_{|S|}]$ with $\mathbf{X}_j = \mathbf{T}_{i_j} + \mathsf{sh}_j$.

Set $\mathsf{ct} = (\mathbf{s}^\top \mathbf{B} + \gamma_{\mathcal{C}} \mathbf{e}^\top + \widetilde{\mathbf{e}}_1^\top, \mathbf{s}^\top(\mathbf{C}_0 + \mathsf{MCOM.Com}(\mathbf{X})) + \gamma_{\mathcal{C}} \mathbf{e}^\top \mathbf{R}_{\mathbf{C}} + \widetilde{\mathbf{e}}_2^\top, \Psi, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}})$ and $\mathsf{st} := (\widetilde{\mathbf{e}}_1^\top, \widetilde{\mathbf{e}}_2^\top, \widetilde{\mathbf{e}}_3^\top)$. Send $(\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in S}, \{\mathsf{sk}_i\}_{i\in\mathcal{C}}, \mathsf{ct}, \mathsf{st})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$. In this hybrid, we set all components of ct using $\mathbf{c}_1^\top$.

Setup Phase. $\mathcal{A}$ sends $(1^\ell, 1^d, N, S, \mathcal{C}, \mathbb{A}, \mu_0, \mu_1)$. Do the following.

1. Sample $(\mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{MCOM.Setup}(1^n, 1^m, q, \sigma)$. Set $\mathsf{crs} := (N, \ell, d, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathcal{H}_\rho, \mathcal{H}_{\mathsf{sh}})$ where $\mathcal{H}_\rho$ and $\mathcal{H}_{\mathsf{sh}}$ are handles for random oracles.
2. Compute $((\delta_i)_{i\in S}, \gamma_{\mathcal{C}}) \leftarrow \mathsf{GetTargetedCoeffs}(\mathbb{A}, \mathcal{C})$.
3. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}$ and set $\mathbf{c}_1^\top := \mathbf{s}^\top \mathbf{B} + \gamma_{\mathcal{C}} \mathbf{e}^\top$.
4. Sample $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\rho \leftarrow \{0,1\}^\lambda$.
5. Sample $\mathbf{R}_{\mathbf{A}} \leftarrow \{\pm 1\}^{4m^5\times m}$ and set $\mathbf{A} = \mathbf{B}\mathbf{R}_{\mathbf{A}}$. Sample $\mathbf{R} \leftarrow \{\pm 1\}^{m\times \ell m}$ and $\Psi := \begin{bmatrix} \mathbf{B} \\ \mathbf{c}_1^\top \end{bmatrix} \mathbf{R}_{\mathbf{A}} \mathbf{R} + \begin{bmatrix} \mathbf{0} \\ \widetilde{\mathbf{e}}_3^\top \end{bmatrix} + \mu^\top \otimes \mathbf{G}$.

6. Compute $(\overline{\Psi}, \mathsf{sh}_1, \ldots, \mathsf{sh}_{|S|}) = \mathsf{ProgShare}_2(\mathbb{A}, \mathcal{C}, \{\mathsf{sh}_j\}_{j\in\mathcal{C}}, \overline{\Psi})$ after sampling $\mathbf{D}'_j \leftarrow \mathcal{D}^{4m^5\times \ell m}_{\mathbb{Z},\sigma_{\mathsf{er}}}$ and set $\mathsf{sh}_j = \mathbf{B}\mathbf{D}'_j$ for each $j \in \mathcal{C}$.
7. Program values in $(\mathsf{sh}, \mathsf{sd}_{\mathsf{sh}}, j)$-th entries of H as sh_j for $j \in A$ where A is a maximally invalid party set for $\mathbb{A}$ in S.
8. For each $i \in \mathcal{C}$, sample $\mathbf{R}_i \leftarrow \{\pm 1\}^{4m^5\times \ell m}$ and set $(\mathsf{pk}_i, \mathsf{sk}_i) = ((i, \mathbf{B}\mathbf{R}_i), \mathbf{R}_i)$. For each $i \in S \setminus \mathcal{C}$, set $\mathsf{pk}_i = (i, \mathbf{B}\mathbf{D}_i - \mathsf{sh}_i)$ where $\mathbf{D}_i \leftarrow \mathcal{D}^{4m^5\times \ell m}_{\mathbb{Z},\sigma_{\mathsf{er}}}$.
9. Compute matrix $\mathbf{X} = [\mathbf{X}_1 \mid \ldots \mid \mathbf{X}_{|S|}]$ with $\mathbf{X}_j = \mathbf{T}_{i_j} + \mathsf{sh}_j$.
10. Compute $\mathbf{C} = \mathsf{MCOM.Com}(\mathbf{X}), \mathbf{Z}_{\mathbf{X}} = \mathsf{MCOM.Open}(\mathbf{X}), \mathbf{V} = \mathsf{MCOM.Verify}(1^{|S|\ell m})$.
11. Sample $\mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5\times m}$, $\mathbf{C}_0 = \mathbf{B}\mathbf{R}_{\mathbf{C}} - \mathbf{C}, \mathbf{Z}_0 = [\mathbf{Z}_{0,1} \mid \ldots \mid \mathbf{Z}_{0,|S|}] \in \mathbb{Z}_q^{m\times \ell m|S|}$ as

$$\mathbf{Z}_{0,j} = \begin{cases} \mathbf{D}'_j - \mathbf{R}_{\mathbf{C}}\mathbf{V}_j - \mathbf{Z}_{\mathbf{X},j} & \text{if } j \in S \setminus C \\ \mathbf{D}'_j + \mathbf{R}_j - \mathbf{R}_{\mathbf{C}}\mathbf{V}_j - \mathbf{Z}_{\mathbf{X},j} & \text{if } j \in C \end{cases}$$

12. Sample $r' \leftarrow \mathsf{ER.Explain}(1^\lambda, 1^{T'}, \mathbf{C}_0, \mathbf{Z}_0, \sigma_{\mathsf{er}})$ and set $\mathsf{H}[(\rho, \mathsf{sd}_\rho, k)] = r'_k$ for each $k \in [\rho]$.

Set $\mathsf{ct} = (\mathbf{c}_1^\top + \widetilde{\mathbf{e}}_1^\top, \mathbf{c}_1^\top\mathbf{R}_{\mathbf{C}} + \widetilde{\mathbf{e}}_2^\top, \Psi, \mathsf{sd}_\rho, \mathsf{sd}_{\mathsf{sh}})$ and $\mathsf{st} := (\widetilde{\mathbf{e}}_1^\top, \widetilde{\mathbf{e}}_2^\top, \widetilde{\mathbf{e}}_3^\top)$. Send $(\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in S}, \{\mathsf{sk}_i\}_{i\in\mathcal{C}}, \mathsf{ct}, \mathsf{st})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

$\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ except that we set $\mathbf{c}_1^\top \leftarrow \mathbb{Z}_q^{1\times 4m^5}$.

$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ except that we set $\Psi \leftarrow \mathbb{Z}_q^{n+1\times \ell m}$. Note that this hybrid is independent of b.

Indistinguishability of Hybrids. We will now show the indistinguishability of above hybrids using the following lemmas.

Lemma 7.14. If MCOM satisfies mode indistinguishability, ER satisfies explainability and randomness properties, for any PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that $\forall\ \lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)]| \le \mathsf{negl}(\lambda)$.

Proof. This is similar to proof of DPE scheme from Section 6. Specifically, hybrids 0 to 12 from that proof. The main difference is that we are no longer sharing a random $\mathbf{p}$ but a structured $\overline{\Psi}$. However this is not an issue as $\mathcal{C}$ is a maximally invalid party set and by programmability property, we have that shares of $\mathcal{C}$ are uniformly random. □

Lemma 7.15. Assuming hardness of $2m^2$-decomposed LWE assumption (Assumption 3.5), for any PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that $\forall\ \lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)]| \le \mathsf{negl}(\lambda)$.

Proof. This is similar to proof of Lemma 6.20 except we use $\gamma_{\mathcal{C}}$ instead of p. □

Lemma 7.16. There exists a negligible function $\mathsf{negl}(\cdot)$ such that $\forall\ \lambda \in \mathbb{N}$, $|\Pr[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)]| \le \mathsf{negl}(\lambda)$.

Proof. This follows from generalized leftover hash lemma (Lemma 3.6). Specifically, since $\mathbf{B}$ is statistically close to random,

$$\left\{\left(\begin{bmatrix}\mathbf{B}\\ \mathbf{c}_1^\top\end{bmatrix}, \begin{bmatrix}\mathbf{B}\\ \mathbf{c}_1^\top\end{bmatrix}\mathbf{R}_{\mathbf{A}}\right)\right\} \approx_s \left\{\left(\begin{bmatrix}\mathbf{B}\\ \mathbf{c}_1^\top\end{bmatrix}, \begin{bmatrix}\mathbf{A}\\ \mathbf{b}^\top\end{bmatrix}\right)\right\}$$

After that another leftover hash lemma application lets us set $\Psi \leftarrow \mathbb{Z}_q^{n+1\times \ell m}$. □

8 Threshold Homomorphic Encryption from LWE

In this section, we construct a threshold homomorphic encryption scheme from LWE. More generally, the construction applies to access structures that admit scaled reconstruction, but throughout this section we focus on threshold access structures. Our security notion is stated with respect to a *maximally invalid* corrupted set $\mathcal{C}$.

8.1 Definition

Syntax. A threshold homomorphic encryption scheme (TFHE) for a class of Boolean circuits $\mathcal{F} = \{\mathcal{F}_d\}_{d\in\mathbb{N}}$ and a class of monotone access structures $\mathbb{S}$ over N parties consists of the following polynomial-time algorithms.

$\mathsf{Setup}(1^\lambda, 1^d, \mathbb{A}) \to (\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N)$**.** The randomized setup algorithm takes as input a security parameter λ, a depth bound d, and an access structure $\mathbb{A} \in \mathbb{S}$, and outputs a public key pk together with secret keys $\mathsf{sk}_1, \ldots, \mathsf{sk}_N$.

$\mathsf{Enc}(\mathsf{pk}, \mu) \to \mathsf{ct}$**.** The randomized encryption algorithm takes as input a public key pk and a message $\mu \in \{0,1\}$, and outputs a ciphertext ct.

$\mathsf{Eval}(\mathsf{pk}, C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell) \to \widetilde{\mathsf{ct}}$**.** The deterministic evaluation algorithm takes as input a public key pk, a circuit $C : \{0,1\}^\ell \to \{0,1\}$ of depth at most d, and ciphertexts $\mathsf{ct}_1, \ldots, \mathsf{ct}_\ell$, and outputs an evaluated ciphertext $\widetilde{\mathsf{ct}}$.

$\mathsf{PartDec}(\mathsf{sk}_i, \widetilde{\mathsf{ct}}) \to \mathsf{dsh}_i$**.** The possibly randomized partial decryption algorithm takes as input the secret key sk_i of party i and an evaluated ciphertext $\widetilde{\mathsf{ct}}$, and outputs a decryption share dsh_i.

$\mathsf{Recon}(\mathbb{A}, \{\mathsf{dsh}_j\}_{j\in S}, \widetilde{\mathsf{ct}}) \to y$**.** The deterministic reconstruction algorithm takes as input an access structure $\mathbb{A}$, decryption shares for a set $S \subseteq [N]$, and an evaluated ciphertext $\widetilde{\mathsf{ct}}$, and outputs $y \in \{0,1\} \cup \{\perp\}$.

Definition 8.1 (Threshold FHE for Monotone Policies)**.** A TFHE scheme $(\mathsf{Setup}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{PartDec}, \mathsf{Recon})$ is a threshold FHE scheme for monotone policies if it satisfies the following properties.

Evaluation Correctness. For every $\lambda \in \mathbb{N}$, every $d = \mathsf{poly}(\lambda)$, every $\mathbb{A} \in \mathbb{S}$, every valid set $S \in \mathbb{A}$, every $\ell = \mathsf{poly}(\lambda)$, every circuit $C : \{0,1\}^\ell \to \{0,1\}$ of depth at most d, and every messages $\mu_1, \ldots, \mu_\ell \in \{0,1\}$, if $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N) \leftarrow \mathsf{Setup}(1^\lambda, 1^d, \mathbb{A})$, each $\mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{pk}, \mu_i)$, $\widetilde{\mathsf{ct}} \leftarrow \mathsf{Eval}(\mathsf{pk}, C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell)$, and each $\mathsf{dsh}_j \leftarrow \mathsf{PartDec}(\mathsf{sk}_j, \widetilde{\mathsf{ct}})$ for $j \in S$, then

$$\Pr[\mathsf{Recon}(\mathbb{A}, \{\mathsf{dsh}_j\}_{j\in S}, \widetilde{\mathsf{ct}}) = C(\mu_1, \ldots, \mu_\ell)] = 1.$$

Compactness. There exist a polynomial $\mathsf{poly}(\cdot,\cdot)$ such that the public key, fresh ciphertexts, evaluated ciphertexts, secret-key shares, and decryption shares all have size at most $o(N) \cdot \mathsf{poly}(\lambda, d, \log N)$.

Security. There exists a stateful polynomial-time simulator $\mathcal{S} = (\mathsf{Setup\text{-}Sim}, \mathsf{Enc\text{-}Sim}, \mathsf{PartDec\text{-}Sim})$ with the following syntax.

- $\mathsf{Setup\text{-}Sim}(1^\lambda, 1^d, \mathbb{A}, \mathcal{C}) \to (\mathsf{pk}, (\mathsf{sk}_i)_{i\in\mathcal{C}})$**.** The randomized setup simulation algorithm takes as input a security parameter λ, a depth bound d, an access structure $\mathbb{A} \in \mathbb{S}$, and a corrupted set $\mathcal{C} \subseteq [N]$. It outputs a public key pk together with the secret keys of the corrupted parties.
- $\mathsf{Enc\text{-}Sim}(\mathsf{pk}, \mu) \to \mathsf{ct}$**.** The randomized encryption simulation algorithm takes as input a public key pk and a message $\mu \in \{0,1\}$, and outputs a ciphertext ct.
- $\mathsf{PartDec\text{-}Sim}(S, \mathsf{pk}, C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell, C(\mu_1, \ldots, \mu_\ell)) \to (\mathsf{dsh}_i)_{i\in S}$**.** The partial decryption simulation algorithm takes as input a set $S \subseteq [N]$, a public key pk, a circuit C, the ciphertexts $(\mathsf{ct}_1, \ldots, \mathsf{ct}_\ell)$, and the output bit $C(\mu_1, \ldots, \mu_\ell)$, and outputs decryption shares for all parties in S.

We require the following two security properties.

Semantic Security: For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\Pr\left[\begin{array}{ll} b' = b & : \begin{array}{l}(1^d, \mathbb{A}, \mathcal{C}) \leftarrow \mathcal{A}(1^\lambda) \text{ s.t. } \mathcal{C} \notin \mathbb{A} \text{ is maximally invalid,} \\ (\mathsf{pk}, (\mathsf{sk}_j)_{j\in\mathcal{C}}) \leftarrow \mathsf{Setup\text{-}Sim}(1^\lambda, 1^d, \mathbb{A}, \mathcal{C}), \\ b \leftarrow \{0,1\},\ \mathsf{ct} \leftarrow \mathsf{Enc\text{-}Sim}(\mathsf{pk}, b), \\ b' \leftarrow \mathcal{A}(\mathsf{pk}, \mathsf{ct}, \{\mathsf{sk}_j\}_{j\in\mathcal{C}}, \mathsf{st}_{\mathcal{S}})\end{array}\end{array}\right] \leq 1/2 + \mathsf{negl}(\lambda),$$

where $\mathsf{st}_{\mathcal{S}}$ includes the simulator state after running $\mathsf{Setup\text{-}Sim}$ and then $\mathsf{Enc\text{-}Sim}$.[6]

Simulation Security. For every stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and every $d = \mathsf{poly}(\lambda)$,

$$\left|\Pr[1 \leftarrow \mathsf{ExpReal}_{\mathcal{A}}(1^\lambda, 1^d)] - \Pr[1 \leftarrow \mathsf{ExpIdeal}_{\mathcal{A}}(1^\lambda, 1^d)]\right| \leq \mathsf{negl}(\lambda),$$

where the real and ideal experiments are defined as follows.

$\mathsf{ExpReal}_{\mathcal{A}}(1^\lambda, 1^d)$**:**

1. Given $(1^\lambda, 1^d)$, the adversary $\mathcal{A}$ sends $(\mathbb{A}, \mathcal{C})$ to the challenger, where $\mathcal{C} \notin \mathbb{A}$ is maximally invalid.
2. The challenger samples $(\mathsf{pk}, (\mathsf{sk}_j)_{j\in[N]}) \leftarrow \mathsf{Setup}(1^\lambda, 1^d, \mathbb{A})$ and sends $(\mathsf{pk}, (\mathsf{sk}_j)_{j\in\mathcal{C}})$ to $\mathcal{A}$.
3. The adversary sends $(\mu_i)_{i\in[\ell]}$ to the challenger and receives $(\mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{pk}, \mu_i))_{i\in[\ell]}$.
4. The adversary adaptively sends a polynomial number of queries of the form (S, C), where $S \subseteq [N]$ and $C : \{0,1\}^\ell \to \{0,1\}$ has depth at most d. For each query, the challenger computes $\widetilde{\mathsf{ct}} \leftarrow \mathsf{Eval}(\mathsf{pk}, C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell)$ and replies with $(\mathsf{dsh}_i \leftarrow \mathsf{PartDec}(\mathsf{sk}_i, \widetilde{\mathsf{ct}}))_{i\in S}$.
5. Finally, the adversary outputs a bit.

$\mathsf{ExpIdeal}_{\mathcal{A}}(1^\lambda, 1^d)$**:**

1. Given $(1^\lambda, 1^d)$, the adversary $\mathcal{A}$ sends $(\mathbb{A}, \mathcal{C})$ to the challenger, where $\mathcal{C} \notin \mathbb{A}$ is maximally invalid.
2. The challenger samples $(\mathsf{pk}, (\mathsf{sk}_j)_{j\in\mathcal{C}}) \leftarrow \mathsf{Setup\text{-}Sim}(1^\lambda, 1^d, \mathbb{A}, \mathcal{C})$ and sends $(\mathsf{pk}, (\mathsf{sk}_j)_{j\in\mathcal{C}})$ to $\mathcal{A}$.
3. The adversary sends $(\mu_i)_{i\in[\ell]}$ to the challenger and receives $(\mathsf{ct}_i \leftarrow \mathsf{Enc\text{-}Sim}(\mathsf{pk}, \mu_i))_{i\in[\ell]}$.
4. The adversary adaptively sends a polynomial number of queries of the form (S, C), where $S \subseteq [N]$ and $C : \{0,1\}^\ell \to \{0,1\}$ has depth at most d. For each query, the challenger replies with
$$(\mathsf{dsh}_i)_{i\in S} \leftarrow \mathsf{PartDec\text{-}Sim}(S, \mathsf{pk}, C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell, C(\mu_1, \ldots, \mu_\ell)).$$
5. Finally, the adversary outputs a bit.

Remark 8.2 (Selective vs. semi-adaptive security.)**.** Our definition assumes corrupted parties $\mathcal{C}$ are declared selectively before $\mathcal{A}$ gets pk. This can be generically boosted to a version of security where $\mathcal{A}$ declares $\mathcal{C}$ after receiving pk using known transformations between selective and semi-adaptive security [GKW16].

Remark 8.3 (TFHE without maximal corruptions)**.** In the above definition, we consider an TFHE scheme where $\mathcal{A}$ declares a maximally invalid party set. This can be avoided by using key homomorphic PRFs as demonstrated by [EY24].

8.2 Construction

Parameters. We provide the parameters used in the construction below.

[6] We slightly refine the definition of [BGG+18] by formulating the semantic-security adversary in the ideal world. This is necessary for the definition to capture the intended notion of security.

- n is chosen according to the lattice parameter convention of Section 6.2, so that $q/B_{\mathsf{LWE}} = 2^{\widetilde{O}(n^\epsilon)}$ for a fixed $\epsilon \in (0, 1/2)$. (LWE security)
- $m = \Omega(n \log q)$ (Generalized leftover hash lemma)
- $\chi = \mathsf{poly}(\lambda)$ (LWE security)
- $\chi', \chi'' = \lambda^{\omega(1)}\chi$ (Smudging lemma)
- Odd prime $q > 4\beta^2(\chi + \chi')m^{O(d)} + 4\beta N\alpha\chi''\mathsf{poly}(m)$ (Correctness)

Construction 8.4. Let $\mathbb{A}$ be a threshold access structure whose secret-sharing scheme admits (α, β)-scaled reconstruction. We construct the threshold homomorphic encryption scheme as follows.

$\mathsf{Setup}(1^\lambda, 1^d, \mathbb{A})$**.** Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^{n-1}$ and let $\mathsf{sk} = \begin{bmatrix} -\mathbf{s} \\ 1 \end{bmatrix} \in \mathbb{Z}_q^n$. Secret-share sk according to $\mathbb{A}$ by sampling $(\mathbf{sh}_1, \ldots, \mathbf{sh}_N) \leftarrow \mathsf{SS.Share}(\mathbb{A}, \mathsf{sk})$. Next sample $\mathbf{B} \leftarrow \mathbb{Z}_q^{(n-1)\times m}$, $\mathbf{e} \leftarrow \widetilde{\mathcal{D}}^m_{m,\mathbb{Z},\chi}$, and $\hat{\mathbf{e}} \leftarrow \widetilde{\mathcal{D}}^m_{m,\mathbb{Z},\chi'}$, let $\mathbf{b}^\top = \mathbf{s}^\top\mathbf{B} + \mathbf{e}^\top + \hat{\mathbf{e}}^\top$, and set

$$\mathsf{pk} = \mathbf{A} = \begin{bmatrix} \mathbf{B} \\ \mathbf{b}^\top \end{bmatrix} \in \mathbb{Z}_q^{n\times m}, \qquad \mathsf{sk}_i = \mathbf{sh}_i \ \ \forall i \in [N].$$

Output $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N)$.

$\mathsf{Enc}(\mathsf{pk}, \mu)$**.** Sample $\mathbf{R} \leftarrow \{0,1\}^{m\times m}$ and output $\mathsf{ct} = \mathbf{AR} + \mu\mathbf{G}$.

$\mathsf{Eval}(\mathsf{pk}, C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell)$**.** Let $\mathsf{ct} = [\mathsf{ct}_1 \mid \cdots \mid \mathsf{ct}_\ell]$ and $\mathbf{w} = [0, \ldots, 0, \lceil q/2\beta \rceil]$, then compute $\mathbf{H}_{C,\mathsf{ct}}$ from Lemma 7.3, and output $\widetilde{\mathsf{ct}} = \mathsf{ct}\mathbf{H}_{C,\mathsf{ct}}\mathbf{G}^{-1}(\mathbf{w}^\top)$.

$\mathsf{PartDec}(\mathsf{sk}_i, \widetilde{\mathsf{ct}})$**.** Sample $e_{\mathsf{sm},i} \leftarrow \widetilde{\mathcal{D}}_{m,\mathbb{Z},\chi''}$, and output $\mathsf{dsh}_i = \mathsf{sk}_i^\top\widetilde{\mathsf{ct}} + e_{\mathsf{sm},i}$.

$\mathsf{Recon}(\mathbb{A}, \{\mathsf{dsh}_j\}_{j\in S'}, \widetilde{\mathsf{ct}})$**.** If $S' \notin \mathbb{A}$, output $\perp$. Otherwise choose a minimally valid set $S \subseteq S'$, compute $((\delta_{S,i})_{i\in S}, \gamma_S) \leftarrow \mathsf{SS.GetCoeffs}(\mathbb{A}, S)$, let $z = \sum_{i\in S}\delta_{S,i}\mathsf{dsh}_i$, and output the unique bit $\mu \in \{0,1\}$ such that

$$\min_{k\in\mathbb{Z}} |z - \mu\cdot\gamma_S\cdot\lceil q/2\beta\rceil - kq| < q/4\beta.$$

Remark 8.5. To present our double-smudging technique cleanly, we describe the construction in the GSW FHE framework. We note, however, that the same construction works in an almost black-box manner over other existing FHE schemes as well.

Evaluation Correctness. Note that Recon outputs $\perp$ whenever $S \notin \mathbb{A}$, so we only consider the case $S \in \mathbb{A}$. Let $((\delta_{S,i})_{i\in S}, \gamma_S) \leftarrow \mathsf{SS.GetCoeffs}(\mathbb{A}, S)$. By correctness of the secret-sharing scheme, $\gamma_S\mathsf{sk} = \sum_{i\in S}\delta_{S,i}\mathsf{sk}_i$. Writing $\tilde{\mu} = C(\mu_1, \ldots, \mu_\ell)$ and $\mathsf{ct}_k = \mathbf{AR}_k + \mu_k\mathbf{G}$ for each $k \in [\ell]$, correctness of the underlying [GSW13] evaluation implies

$$\mathsf{sk}^\top\widetilde{\mathsf{ct}} = \tilde{\mu}\cdot\lceil q/2\beta\rceil + [(\mathbf{e}^\top + \hat{\mathbf{e}}^\top)\mathbf{R}_1 \mid \cdots \mid (\mathbf{e}^\top + \hat{\mathbf{e}}^\top)\mathbf{R}_\ell]\mathbf{H}_{C,\mathsf{ct}}\mathbf{G}^{-1}(\mathbf{w}^\top).$$

Hence

$$\sum_{i\in S}\delta_{S,i}\mathsf{dsh}_i = \gamma_S\tilde{\mu}\cdot\lceil q/2\beta\rceil + \mathsf{err},$$

where $|\mathsf{err}| \le \beta(\chi + \chi')m^{O(d)} + N\alpha\chi''\mathsf{poly}(m) < q/4\beta$ by the choice of parameters. Therefore Recon outputs the correct bit.

Compactness. Under the lattice parameter convention of Section 6.2, for a threshold policy with $\tau = \min(t^2, N - t)$, the public key and fresh ciphertext have size $O_d^\dagger(\tau^6)$, each secret-key share and evaluated ciphertext have size $O_d^\dagger(\tau^3)$, and each partial decryption has size $O_d^\dagger(\tau)$. Thus the secret-key shares, evaluated ciphertexts, and partial decryptions are sublinear in N whenever $\tau = o(N^{1/3})$. Including the public key and fresh ciphertext, all parameters are simultaneously sublinear whenever $\tau = o(N^{1/6})$.

Theorem 8.6 (Security.)**.** Assuming the hardness of the LWE assumption (Assumption 3.4), Construction 8.4 satisfies semantic security and simulation security (Definition 8.1).

Proof. We construct the simulator $\mathcal{S} = (\mathsf{Setup\text{-}Sim}, \mathsf{Enc\text{-}Sim}, \mathsf{PartDec\text{-}Sim})$ as follows.

$\mathsf{Setup\text{-}Sim}(1^\lambda, 1^d, \mathbb{A}, \mathcal{C}) \to (\mathsf{pk}, (\mathsf{sk}_i)_{i\in\mathcal{C}})$**.** Similar to the Setup except that we sample $\mathbf{e}_0 \leftarrow \widetilde{\mathcal{D}}^m_{m,\mathbb{Z},\chi}$, and then let $\mathbf{e} = \theta\mathbf{e}_0$ where θ for threshold access structure is defined as below:

- If the threshold is sublinear then compute $((\delta_{T,i})_{i\in\mathcal{C}\cup\{0\}}, \gamma_\mathcal{C}) = \mathsf{SS.GetTargetedCoeffs}(\mathbb{A}, \mathcal{C}, j)$ where $T = \mathcal{C}\cup\{j\}$ for some arbitrary $j \in [N]\setminus\mathcal{C}$. Then let $\theta = \gamma_\mathcal{C}$.
- If the threshold is linear-minus-sublinear then let $\theta = \prod_{i\in[N]\setminus\mathcal{C}} \delta_{T_i,i}$ where $T_i = \mathcal{C}\cup\{i\}$ and $\delta_{T_i,i}$ is computed by running $\mathsf{SS.GetCoeffs}(\mathbb{A}, T_i)$.

Finally let the simulator state be $\mathsf{st} = (\hat{\mathbf{e}}, (\mathsf{sk}_i)_{i\in\mathcal{C}})$.

$\mathsf{Enc\text{-}Sim}(\mathsf{pk}, \mu) \to \mathsf{ct}$**.** Parse simulator's state as $\mathsf{st} = (\hat{\mathbf{e}}, (\mathsf{sk}_i)_{i\in\mathcal{C}}, (\hat{\mathbf{e}}^\top \cdot \mathbf{R}_i)_{i\in[t-1]})$, sample $\mathbf{R}_t \leftarrow \{0,1\}^{m\times m}$, and then compute $\mathsf{ct} = \mathbf{A}\mathbf{R}_t + \mu\mathbf{G}$ and update the state to be $\mathsf{st} = (\hat{\mathbf{e}}, (\mathsf{sk}_i)_{i\in\mathcal{C}}, (\hat{\mathbf{e}}^\top \cdot \mathbf{R}_i)_{i\in[t]})$.

$\mathsf{PartDec\text{-}Sim}(S, \mathsf{pk}, C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell, C(\mu_1, \ldots, \mu_\ell))$**.** Parse $\mathsf{st} = (\hat{\mathbf{e}}, (\mathsf{sk}_i)_{i\in\mathcal{C}}, (\hat{\mathbf{e}}^\top \cdot \mathbf{R}_i)_{i\in[\ell]})$, let $\tilde{\mu} = C(\mu_1, \ldots, \mu_\ell)$, and compute $\widetilde{\mathsf{ct}} \leftarrow \mathsf{Eval}(\mathsf{pk}, C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell)$. If $i \in S \cap \mathcal{C}$ compute $\mathsf{dsh\text{-}sim}_i \leftarrow \mathsf{PartDec}(\mathsf{sk}_i, \widetilde{\mathsf{ct}})$. For every honest party $i \in S\setminus\mathcal{C}$, let $\Delta = [\hat{\mathbf{e}}^\top\mathbf{R}_1 \mid \cdots \mid \hat{\mathbf{e}}^\top\mathbf{R}_\ell]\mathbf{H}_{C,\mathsf{ct}}\mathbf{G}^{-1}(\mathbf{w}^\top)$, sample fresh $e_{\mathsf{sm},i} \leftarrow \widetilde{\mathcal{D}}_{m,\mathbb{Z},\chi''}$, and compute $\mathsf{dsh\text{-}sim}_i$ in each threshold regime as follows:

- If the threshold is sublinear then compute

$$\mathsf{dsh\text{-}sim}_i \leftarrow \gamma_\mathcal{C}^{-1}\left(\sum_{j\in\mathcal{C}} \delta_{T_i,j}\mathsf{sk}_j^\top\widetilde{\mathsf{ct}} + \delta_{T_i,0}\left(\tilde{\mu}\cdot\lceil q/2\beta\rceil + \Delta\right)\right) + e_{\mathsf{sm},i}.$$

  where $((\delta_{T_i,j})_{j\in\mathcal{C}\cup\{0\}}, \gamma_\mathcal{C}) \leftarrow \mathsf{SS.GetTargetedCoeffs}(\mathbb{A}, \mathcal{C}, i)$ and $T_i = \mathcal{C}\cup\{i\}$.
- If the threshold is linear-minus-sublinear then compute

$$\mathsf{dsh\text{-}sim}_i \leftarrow \delta_{T_i,i}^{-1}\left(-\sum_{j\in\mathcal{C}} \delta_{T_i,j}\mathsf{sk}_j^\top\widetilde{\mathsf{ct}} + \gamma_{T_i}\left(\tilde{\mu}\cdot\lceil q/2\beta\rceil + \Delta\right)\right) + e_{\mathsf{sm},i}.$$

  where $((\delta_{T_i,j})_{j\in T_i}, \gamma_{T_i}) \leftarrow \mathsf{SS.GetCoeffs}(\mathbb{A}, T_i)$ and $T_i = \mathcal{C}\cup\{i\}$.

Finally output $(\mathsf{dsh\text{-}sim}_i)_{i\in S}$.

Semantic security. Fix any security parameter λ, and let $(1^d, \mathbb{A}, \mathcal{C}) \leftarrow \mathcal{A}(1^\lambda)$ be such that $\mathcal{C} \notin \mathbb{A}$ is maximally invalid. For each challenge bit $b \in \{0,1\}$, define the following hybrid distributions.

$\mathbf{Hyb}^{\mathcal{A},\mathsf{Sem}}_{0,b}(1^\lambda)$**.** This is the real simulator-view distribution:

$$\left(\mathsf{pk}, (\mathsf{sk}_j)_{j\in\mathcal{C}}, \mathsf{ct}, \mathsf{st}_\mathcal{S}\right),$$

where $(\mathsf{pk}, (\mathsf{sk}_j)_{j\in\mathcal{C}}) \leftarrow \mathsf{Setup\text{-}Sim}(1^\lambda, 1^d, \mathbb{A}, \mathcal{C})$ and $\mathsf{ct} \leftarrow \mathsf{Enc\text{-}Sim}(\mathsf{pk}, b)$.

$\mathbf{Hyb}^{\mathcal{A},\mathsf{Sem}}_{1,b}(1^\lambda)$**.** Same as $\mathbf{Hyb}^{\mathcal{A},\mathsf{Sem}}_{0,b}(1^\lambda)$ except that in $\mathsf{Setup\text{-}Sim}$ the simulator shares $\mathbf{0}$ instead of sk and gives the adversary the shares corresponding to $\mathcal{C}$.

$\mathbf{Hyb}^{\mathcal{A},\mathsf{Sem}}_{2,b}(1^\lambda)$**.** Same as $\mathbf{Hyb}^{\mathcal{A},\mathsf{Sem}}_{1,b}(1^\lambda)$ except that the public-key row $\mathbf{b}^\top = \mathbf{s}^\top\mathbf{B} + \theta\mathbf{e}_0^\top + \hat{\mathbf{e}}^\top$ is replaced by a uniformly random row $\mathbf{u}^\top$ where $\mathbf{u} \leftarrow \mathbb{Z}_q^{1\times m}$.

$\mathbf{Hyb}_{3,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$**.** Same as $\mathbf{Hyb}_{2,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ except that in the challenge ciphertext the affine part $\mathbf{AR}$ is replaced by a uniformly random matrix $\mathbf{D} \leftarrow \mathbb{Z}_q^{n\times m}$, while the simulator state still reveals the value $\hat{\mathbf{e}}^\top\mathbf{R}$.

$\mathbf{Hyb}_{4,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$**.** Same as $\mathbf{Hyb}_{3,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ except that the challenge ciphertext is replaced by a uniformly random matrix $\mathbf{D}' \leftarrow \mathbb{Z}_q^{n\times m}$.

We now prove indistinguishability of adjacent hybrids.

Lemma 8.7. For every $b \in \{0,1\}$, the distributions $\mathbf{Hyb}_{0,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ and $\mathbf{Hyb}_{1,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ are computationally indistinguishable.

Proof. The only difference between the two hybrids is that in $\mathbf{Hyb}_{0,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ the simulator shares sk, whereas in $\mathbf{Hyb}_{1,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ it shares $\mathbf{0}$. The adversary receives only the shares corresponding to the set $\mathcal{C}$, and by assumption $\mathcal{C} \notin \mathbb{A}$. Hence, by privacy of the underlying secret-sharing scheme, the joint distribution of the corrupted shares is indistinguishable in the two experiments. All remaining values are generated identically. Therefore the two hybrids are computationally indistinguishable. □

Lemma 8.8. For every $b \in \{0,1\}$, the distributions $\mathbf{Hyb}_{1,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ and $\mathbf{Hyb}_{2,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ are computationally indistinguishable.

Proof. In $\mathbf{Hyb}_{1,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$, the adversary's view of the secret-sharing component is independent of sk. Thus the only dependence on the secret vector $\mathbf{s}$ is through the row

$$\mathbf{b}^\top = \mathbf{s}^\top\mathbf{B} + \theta\mathbf{e}_0^\top + \hat{\mathbf{e}}^\top.$$

Since $\mathbf{e}_0 \leftarrow \widetilde{\mathcal{D}}_{m,\mathbb{Z},\chi}^m$ and $\hat{\mathbf{e}}$ is an additive offset independent of $\mathbf{s}$ and $\mathbf{B}$, the pair $(\mathbf{B},\mathbf{b})$ is an LWE sample. Replacing $\mathbf{b}^\top$ by a uniform row is therefore computationally indistinguishable under the LWE assumption. This is exactly the transition from $\mathbf{Hyb}_{1,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ to $\mathbf{Hyb}_{2,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$. □

Lemma 8.9. For every $b \in \{0,1\}$, the distributions $\mathbf{Hyb}_{2,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ and $\mathbf{Hyb}_{3,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ are statistically indistinguishable.

Proof. In $\mathbf{Hyb}_{2,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$, the public key matrix $\mathbf{A}$ is uniform over $\mathbb{Z}_q^{n\times m}$, and the ciphertext has the form $\mathsf{ct} = \mathbf{AR} + b\mathbf{G}$, where $\mathbf{R} \leftarrow \{0,1\}^{m\times m}$. The simulator state reveals the additional value $\hat{\mathbf{e}}^\top\mathbf{R}$. Since $\hat{\mathbf{e}}$ is fixed once the setup randomness is fixed, the generalized leftover hash lemma implies that $(\mathbf{A}, \mathbf{AR}, \hat{\mathbf{e}}^\top\mathbf{R})$ and $(\mathbf{A}, \mathbf{D}, \hat{\mathbf{e}}^\top\mathbf{R})$ are statistically indistinguishable, where $\mathbf{D} \leftarrow \mathbb{Z}_q^{n\times m}$ is uniform. Adding the deterministic term $b\mathbf{G}$ to the second component does not affect the argument. Hence $\mathbf{Hyb}_{2,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ and $\mathbf{Hyb}_{3,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ are statistically indistinguishable. □

Lemma 8.10. For every $b \in \{0,1\}$, the distributions $\mathbf{Hyb}_{3,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ and $\mathbf{Hyb}_{4,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ are identically distributed.

Proof. In $\mathbf{Hyb}_{3,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$, the ciphertext is of the form $\mathbf{D} + b\mathbf{G}$, where $\mathbf{D}$ is uniform over $\mathbb{Z}_q^{n\times m}$. Since adding a fixed matrix to a uniform matrix preserves uniformity, $\mathbf{D} + b\mathbf{G}$ is itself uniformly distributed over $\mathbb{Z}_q^{n\times m}$. Therefore it has exactly the same distribution as the uniform matrix $\mathbf{D}'$ used in $\mathbf{Hyb}_{4,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$. □

We can now conclude semantic security.

Lemma 8.11. For every stateful PPT adversary $\mathcal{A}$, its advantage in the semantic-security experiment is negligible.

Proof. By Lemma 8.7, Lemma 8.8, Lemma 8.9, and Lemma 8.10, for each $b \in \{0,1\}$, the distribution $\mathbf{Hyb}_{0,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ is computationally indistinguishable from $\mathbf{Hyb}_{4,b}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$. But $\mathbf{Hyb}_{4,0}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ and $\mathbf{Hyb}_{4,1}^{\mathcal{A},\mathsf{Sem}}(1^\lambda)$ are identical, since the challenge ciphertext is uniform and independent of b. Hence the adversary's success probability in distinguishing the challenge bit is at most $1/2 + \mathsf{negl}(\lambda)$. □

Simulation Security. Fix any security parameter λ, and let $(1^d, \mathbb{A}, \mathcal{C}) \leftarrow \mathcal{A}(1^\lambda)$ be such that $\mathcal{C} \notin \mathbb{A}$ is maximally invalid. Define the following hybrids.

$\mathbf{Hyb}_0^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$**.** This is the real experiment $\mathsf{ExpReal}_{\mathcal{A}}(1^\lambda, 1^d)$.

$\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_0^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ except that the challenger additionally keeps the following hidden state. During setup, after sampling $\hat{\mathbf{e}}$, it initializes $\mathsf{st} = (\hat{\mathbf{e}}, (\mathsf{sk}_i)_{i\in\mathcal{C}})$. Further, whenever it encrypts μ_t, it samples $\mathbf{R}_t$ exactly as in Enc, returns the ciphertext $\mathsf{ct}_t = \mathbf{A}\mathbf{R}_t + \mu_t\mathbf{G}$, and appends $\hat{\mathbf{e}}^\top\mathbf{R}_t$ to the hidden state. Thus, $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ is externally identical to $\mathbf{Hyb}_0^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$.

$\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ except that setup now uses the simulated public-key row $\mathbf{b}^\top = \mathbf{s}^\top\mathbf{B} + \theta\mathbf{e}_0^\top + \hat{\mathbf{e}}^\top$, where $\mathbf{e}_0 \leftarrow \widetilde{\mathcal{D}}^m_{m,\mathbb{Z},\chi}$ and θ is defined exactly as in $\mathsf{Setup\text{-}Sim}$, and the challenge ciphertexts are generated using $\mathsf{Enc\text{-}Sim}$ instead of Enc.

$\mathbf{Hyb}_3^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ except that for every decryption query (S, C), the shares of all honest parties $i \in S \setminus \mathcal{C}$ are answered using $\mathsf{PartDec\text{-}Sim}$, while the shares of corrupted parties $i \in S \cap \mathcal{C}$ are still answered using $\mathsf{PartDec}(\mathsf{sk}_i, \widetilde{\mathsf{ct}})$. This is exactly the ideal experiment $\mathsf{ExpIdeal}_{\mathcal{A}}(1^\lambda, 1^d)$.

We now prove indistinguishability of adjacent hybrids.

Lemma 8.12. The hybrids $\mathbf{Hyb}_0^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ are identically distributed.

Proof. The challenger in $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ only stores additional hidden state. The public key, the challenge ciphertexts, and all answers returned to the adversary are generated exactly as in $\mathbf{Hyb}_0^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$. Therefore the two hybrids are identically distributed. □

Lemma 8.13. The hybrids $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ are statistically indistinguishable.

Proof. First note that $\mathsf{Enc\text{-}Sim}$ samples $\mathbf{R} \leftarrow \{0,1\}^{m\times m}$ and outputs $\mathsf{ct} = \mathbf{A}\mathbf{R} + \mu\mathbf{G}$, exactly as Enc does. Hence, switching from Enc to $\mathsf{Enc\text{-}Sim}$ does not change the ciphertext distribution at all; it only records the hidden values $\hat{\mathbf{e}}^\top\mathbf{R}$ in the simulator state.

Thus, the only external difference between $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ is the distribution of the public-key row. In $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ the row is

$$\mathbf{b}^\top = \mathbf{s}^\top\mathbf{B} + \mathbf{e}^\top + \hat{\mathbf{e}}^\top,$$

whereas in $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ it is

$$\mathbf{b}^\top = \mathbf{s}^\top\mathbf{B} + \theta\mathbf{e}_0^\top + \hat{\mathbf{e}}^\top.$$

Since χ' is a smudging distribution much wider than χ, and since θ is polynomially bounded in both threshold regimes, the two shifted distributions $\mathbf{e} + \hat{\mathbf{e}}$ and $\theta\mathbf{e}_0 + \hat{\mathbf{e}}$ are statistically indistinguishable by the smudging lemma. Therefore the hybrids $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ are statistically indistinguishable. □

For the remaining transition, fix a decryption query (S, C) and an honest party $i \in S \setminus \mathcal{C}$. Let

$$\tilde{\mu} = C(\mu_1, \ldots, \mu_\ell), \qquad \widetilde{\mathsf{ct}} \leftarrow \mathsf{Eval}(\mathsf{pk}, C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\ell),$$

and define

$$e' = [\mathbf{e}_0^\top\mathbf{R}_1 \mid \cdots \mid \mathbf{e}_0^\top\mathbf{R}_\ell]\mathbf{H}_{C,\mathsf{ct}}\mathbf{G}^{-1}(\mathbf{w}^\top), \qquad \Delta = [\hat{\mathbf{e}}^\top\mathbf{R}_1 \mid \cdots \mid \hat{\mathbf{e}}^\top\mathbf{R}_\ell]\mathbf{H}_{C,\mathsf{ct}}\mathbf{G}^{-1}(\mathbf{w}^\top).$$

Since in $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ the public-key row uses the error term $\theta\mathbf{e}_0 + \hat{\mathbf{e}}$, correctness of the underlying GSW evaluation gives

$$\mathsf{sk}^\top\widetilde{\mathsf{ct}} = \tilde{\mu}\cdot\lceil q/2\beta\rceil + \theta e' + \Delta. \tag{1}$$

Lemma 8.14 (Sublinear threshold case)**.** If the threshold is sublinear, then the hybrids $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ are statistically indistinguishable.

Proof. Fix any honest party $i \in S \setminus \mathcal{C}$, and let $T_i = \mathcal{C} \cup \{i\}$. By the definition of the targeted coefficients returned by $\mathsf{SS.GetTargetedCoeffs}(\mathbb{A}, \mathcal{C}, i)$, we have

$$\delta_{T_i,0}\mathsf{sk} + \sum_{j \in \mathcal{C}} \delta_{T_i,j}\mathsf{sk}_j = \gamma_{\mathcal{C}}\mathsf{sk}_i.$$

Multiplying both sides by $\widetilde{\mathsf{ct}}$ and substituting $\mathsf{sk}^\top \widetilde{\mathsf{ct}}$ using Eq. (1) gives

$$\sum_{j \in \mathcal{C}} \delta_{T_i,j}\mathsf{sk}_j^\top \widetilde{\mathsf{ct}} + \delta_{T_i,0}\left(\tilde{\mu} \cdot \lceil q/2\beta \rceil + \Delta\right) = \gamma_{\mathcal{C}}\mathsf{sk}_i^\top \widetilde{\mathsf{ct}} - \delta_{T_i,0}\theta e'.$$

Dividing by $\gamma_{\mathcal{C}}$ and using $\theta = \gamma_{\mathcal{C}}$, the simulated share produced in $\mathbf{Hyb}_3^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ satisfies

$$\mathsf{dsh}_i = \mathsf{sk}_i^\top \widetilde{\mathsf{ct}} - \delta_{T_i,0}e' + e_{\mathsf{sm},i}.$$

By contrast, in $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ the real share is

$$\mathsf{dsh}_i^{\mathrm{real}} = \mathsf{sk}_i^\top \widetilde{\mathsf{ct}} + e'_{\mathsf{sm},i},$$

where $e'_{\mathsf{sm},i} \leftarrow \widetilde{\mathcal{D}}_{m,\mathbb{Z},\chi''}$ is independent fresh noise. Thus, the simulated share differs from the real one only by the deterministic shift $-\delta_{T_i,0}e'$ together with a fresh sample from the same smudging distribution.

Now $|\delta_{T_i,0}| \le \alpha$ by boundedness of the targeted coefficients, and $|e'| \le \chi m^{O(d)}$ by the standard evaluation-noise bound. Hence the shift has magnitude at most $\alpha\chi m^{O(d)}$, which is statistically hidden by the smudging distribution χ''. Therefore, for each fixed honest party i, the real and simulated share distributions are statistically indistinguishable.

Finally, for each query, the shares of corrupted parties are identical in the two hybrids, and the shares of distinct honest parties use fresh independent noises. Thus the joint distribution of all decryption shares returned for that query is statistically indistinguishable in the two hybrids. Since the adversary makes only polynomially many queries, a union bound implies that the full views in $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ are statistically indistinguishable. □

Lemma 8.15 (Linear-minus-sublinear threshold case)**.** If the threshold is linear-minus-sublinear, then the hybrids $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ are statistically indistinguishable.

Proof. Fix any honest party $i \in S \setminus \mathcal{C}$, and let $T_i = \mathcal{C} \cup \{i\}$. Let

$$((\delta_{T_i,j})_{j \in T_i}, \gamma_{T_i}) \leftarrow \mathsf{SS.GetCoeffs}(\mathbb{A}, T_i).$$

By scaled reconstruction,

$$\gamma_{T_i}\mathsf{sk} - \sum_{j \in \mathcal{C}} \delta_{T_i,j}\mathsf{sk}_j = \delta_{T_i,i}\mathsf{sk}_i.$$

Multiplying both sides by $\widetilde{\mathsf{ct}}$ and substituting $\mathsf{sk}^\top \widetilde{\mathsf{ct}}$ using Eq. (1) yields

$$-\sum_{j \in \mathcal{C}} \delta_{T_i,j}\mathsf{sk}_j^\top \widetilde{\mathsf{ct}} + \gamma_{T_i}\left(\tilde{\mu} \cdot \lceil q/2\beta \rceil + \Delta\right) = \delta_{T_i,i}\mathsf{sk}_i^\top \widetilde{\mathsf{ct}} - \gamma_{T_i}\theta e'.$$

Dividing by $\delta_{T_i,i}$, the simulated share produced in $\mathbf{Hyb}_3^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ satisfies

$$\mathsf{dsh}_i = \mathsf{sk}_i^\top \widetilde{\mathsf{ct}} - \delta_{T_i,i}^{-1}\gamma_{T_i}\theta e' + e_{\mathsf{sm},i}.$$

Again, the real share in $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ is

$$\mathsf{dsh}_i^{\mathrm{real}} = \mathsf{sk}_i^\top \widetilde{\mathsf{ct}} + e'_{\mathsf{sm},i},$$

with fresh independent $e'_{\mathsf{sm},i} \leftarrow \widetilde{\mathcal{D}}_{m,\mathbb{Z},\chi''}$. Thus the two distributions differ only by the deterministic shift

$$-\delta_{T_i,i}^{-1}\gamma_{T_i}\theta e'.$$

By definition of θ,

$$\delta_{T_i,i}^{-1}\gamma_{T_i}\theta = \gamma_{T_i} \cdot \prod_{j\in[N]\setminus(\mathcal{C}\cup\{i\})} \delta_{T_j,j}.$$

In the linear-minus-sublinear threshold regime, the number of missing parties is sublinear, and therefore the above coefficient is polynomially bounded by the properties of the linear-minus-sublinear threshold scaled secret-sharing scheme. Since also $|e'| \le \chi m^{O(d)}$, the total shift is bounded by $\mathsf{poly}(\lambda)\cdot\chi m^{O(d)}$, which is statistically hidden by the smudging distribution χ''. Hence the real and simulated share distributions for party i are statistically indistinguishable.

Exactly as in the proof of Lemma 8.14, this extends to the full vector of decryption shares returned for each query and then, by a union bound, to the adversary's full view over polynomially many adaptive queries. □

We can now conclude simulation security.

Lemma 8.16. For every stateful PPT adversary $\mathcal{A}$, its advantage in the simulation-security experiment is negligible.

Proof. By Lemma 8.12, the hybrids $\mathbf{Hyb}_0^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ are identically distributed. By Lemma 8.13, the hybrids $\mathbf{Hyb}_1^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ are statistically indistinguishable. Finally, if the threshold is sublinear, then Lemma 8.14 shows that the hybrids $\mathbf{Hyb}_2^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ are statistically indistinguishable, whereas if the threshold is linear-minus-sublinear, the same conclusion follows from Lemma 8.15. Since $\mathbf{Hyb}_0^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ is exactly the real experiment and $\mathbf{Hyb}_3^{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ is exactly the ideal experiment, the claim follows. □

□

Corollary 8.17 (Centralized TFHE for Threshold Access Structures)**.** Assuming the hardness of the LWE assumption, there exists a TFHE scheme supporting threshold access structures. For threshold t over N users and $\tau = \min(t^2, N-t)$, the public key and fresh ciphertext have size $O_d^\dagger(\tau^6)$, the secret-key shares and evaluated ciphertexts have size $O_d^\dagger(\tau^3)$, and the partial decryption shares have size $O_d^\dagger(\tau)$. In particular, all parameters are sublinear in N whenever $\tau = o(N^{1/6})$.

Corollary 8.18 (Universal Thresholdizer)**.** Assuming the hardness of the LWE assumption, there exists a universal thresholdizer supporting threshold access structures. For threshold t over N users and $\tau = \min(t^2, N-t)$, the secret keys have size $O_d^\dagger(\tau^3)$ and the partial evaluation values have size $O_d^\dagger(\tau)$. These quantities are sublinear in N whenever $\tau = o(N^{1/3})$, including the public key and fresh ciphertext of the underlying TFHE scheme, all parameters are sublinear whenever $\tau = o(N^{1/6})$.

9 Compact Fuzzy Identity Based Encryption

We now provide the construction of a compact fuzzy identity-based encryption scheme [SW05] secure under the LWE assumption. Throughout this section, we utilize the notation $\mathsf{id} \cap \mathsf{id}'$ for $\mathsf{id}, \mathsf{id}' \in \{0,1\}^\ell$ to denote the set $S = \{i : i \in [\ell], \mathsf{id}_i = \mathsf{id}'_i\}$ where $\mathsf{id}_i, \mathsf{id}'_i$ are the i-th bits of $\mathsf{id}, \mathsf{id}'$ respectively.

9.1 Definition

Syntax. A fuzzy identity-based encryption scheme (FIBE) for ID space $\mathcal{I} = \{\{0,1\}^\ell\}_{\ell\in\mathbb{N}}$ consists of the following polynomial-time algorithms.

$\mathsf{Setup}(1^\lambda, 1^\ell) \to (\mathsf{mpk}, \mathsf{msk})$**.** The randomized setup algorithm takes as input security parameter λ, ID space indicator ℓ, outputs public parameters and master secret key pair $(\mathsf{pp}, \mathsf{msk})$. The following algorithms takes pp implicitly.

$\mathsf{KGen}(\mathsf{msk}, 1^\tau, \mathsf{id}) \to \mathsf{sk}_{\mathsf{id}}$**.** The possibly randomized secret key generation algorithm takes as input master secret key msk, threshold $0 < \tau \le \ell$, ID $\mathsf{id} \in \{0,1\}^\ell$, and outputs the secret key $\mathsf{sk}_{\mathsf{id}}$.

$\mathsf{Enc}(\mathsf{id}, \mu) \to \mathsf{ct}$**.** The randomized encryption algorithm takes as input $\mathsf{id} \in \{0,1\}^\ell$, message $\mu \in \{0,1\}$, and outputs ciphertext ct.

$\mathsf{Dec}(\mathsf{sk}_{\mathsf{id}}, \mathsf{ct}) \to \mu'$**.** The deterministic decryption algorithm takes as input secret key for $\mathsf{id} \in \{0,1\}^\ell$, $\mathsf{sk}_{\mathsf{id}}$, ciphertext ct and outputs $\mu \in \{0,1\} \cup \{\perp\}$.

Definition 9.1 (FIBE)**.** An FIBE scheme $(\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ is said to be an FIBE scheme for ID space $\mathcal{I}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}, \ell = \mathsf{poly}(\lambda)$, $\mathsf{id}, \mathsf{id}' \in \{0,1\}^\lambda, |\mathsf{id} \cap \mathsf{id}'| \ge \tau$, $\mu \in \{0,1\}$,

$$\Pr\left[\begin{array}{lcl} \mu = \mathsf{Dec}(\mathsf{sk}_{\mathsf{id}}, \mathsf{ct}) & : & (\mathsf{pp}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^\ell), \\ & & \mathsf{sk}_{\mathsf{id}} \leftarrow \mathsf{KGen}(\mathsf{msk}, \mathsf{id}), \\ & & \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{id}', \mathsf{ct}) \end{array}\right] = 1$$

Security. For any stateful PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that $\forall\ \lambda \in \mathbb{N}$,

$$\left|\Pr\left[\begin{array}{lcl} b \leftarrow \mathcal{A}^{\mathsf{KGen}(\mathsf{msk},\cdot,\cdot)}(\mathsf{ct}) & : & 1^\ell \leftarrow \mathcal{A}(1^\lambda), \\ & & (\mathsf{pp}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^\ell), \\ & & \mathsf{id}^* \leftarrow \mathcal{A}^{\mathsf{KGen}(\mathsf{msk},\cdot,\cdot)}(\mathsf{pp}), \\ & & b \leftarrow \{0,1\}, \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{id}^*, b) \end{array}\right] - \frac{1}{2}\right| \le \mathsf{negl}(\lambda)$$

A stateful PPT machine $\mathcal{A}$ is said to be admissible if for every τ_q, id'_q queried to $\mathsf{KGen}(\mathsf{msk}, \cdot, \cdot)$ oracle, $|\mathsf{id}'_q \cap \mathsf{id}^*| < \tau_q$.

Definition 9.2 (Selective Security.)**.** An FIBE scheme $(\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ for ID space $\mathcal{I}$ is said to be selectively secure if it satisfies correctness as in Definition 9.1 and the weaker selective security where the PPT $\mathcal{A}$ declares the challenge identity id^* at the beginning of the security game before seeing pp.

9.2 Construction

Parameters. We define the parameters used by our construction below.

- n is chosen according to the lattice parameter convention of Section 6.2, so that $q/B_{\mathsf{LWE}} = 2^{\widetilde{O}(n^\epsilon)}$ for a fixed $\epsilon \in (0, 1/2)$. (LWE security)
- $m = \Omega(n\lceil \log q \rceil)$ (Leftover hash lemma)
- $\chi = \mathsf{poly}(\lambda)$ (LWE security)
- $\sigma = \mathsf{poly}(\lambda, m)$ (Gaussian pre-image sampling)
- Odd prime $p > 3\ell$ (Correctness)
- $q > 2(\beta + p(\beta\sqrt{m}\chi + \ell\alpha m^2\chi\sigma))$ (Correctness)

Construction 9.3. We provide the construction of an FIBE scheme for $\mathcal{I} = \{0,1\}^\ell$ using programmable (α, β)-Scaled LSSS (Definition 4.4, Definition 4.1) as follows.

$\mathsf{Setup}(1^\lambda, 1^\ell)$**.** Sample for each $i, b \in [\ell] \times \{0,1\}$, $(\mathbf{A}_{i,b}, \mathbf{T}_{i,b}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q)$ and $\mathbf{p} \leftarrow \mathbb{Z}_q^n$.

Output $\mathsf{pp} := ((\mathbf{A}_{i,b})_{i\in[\ell],b\in\{0,1\}}, \mathbf{p})$ and $\mathsf{msk} := (\mathbf{T}_{i,b})_{i\in[\ell],b\in\{0,1\}}$.

$\mathsf{KGen}(\mathsf{msk}, 1^\tau, \mathsf{id})$. Parse msk as $(\mathbf{T}_{i,b})_{i\in[\ell],b\in\{0,1\}}$. Do the following.

1. Sample for each $i \in [\tau] \subseteq [\ell], \mathsf{sh}_i \leftarrow \mathbb{Z}_q^n$. Sample $(\mathbf{p}, \mathsf{sh}_1, \ldots, \mathsf{sh}_\ell) \leftarrow \mathsf{ProgShare}_1(1^\ell, 1^\tau, (\mathsf{sh}_i)_{i\in[\tau]})$.
2. For each $i \in [\ell]$, sample $\mathbf{r}_i \leftarrow \mathsf{SamplePre}(\mathbf{A}_{i,\mathsf{id}_i}, \mathbf{T}_{i,\mathsf{id}_i}, \mathsf{sh}_i, \sigma)$.

Output $\mathsf{sk}_{\mathsf{id}} = (1^\tau, \mathsf{id}, (\mathbf{r}_i)_{i\in[\ell]})$.

$\mathsf{Enc}(\mathsf{id}, \mu)$**.** Do the following.

1. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, for each $i \in [\ell]$ $\mathbf{e}_i \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^m$, and $e' \leftarrow \mathcal{D}_{\mathbb{Z},\chi}$.
2. Compute $c' = \mathbf{s}^\top \mathbf{p} + pe' + \mu$ and for each $i \in [\ell]$, $\mathbf{c}_i^\top = \mathbf{s}^\top \mathbf{A}_{i,\mathsf{id}_i} + p\mathbf{e}_i^\top$.

Output $\mathsf{ct} = (\mathsf{id}, (\mathbf{c}_i^\top)_{i\in[\ell]}, c')$.

$\mathsf{Dec}(\mathsf{sk}_{\mathsf{id}'}, \mathsf{ct})$**.** Parse $\mathsf{sk}_{\mathsf{id}}$ as $(1^\tau, \mathsf{id}', (\mathbf{r}_i)_{i\in[\ell]})$ and ct as $(\mathsf{id}, (\mathbf{c}_i^\top)_{i\in[\ell]}, c')$. If $|\mathsf{id}' \cap \mathsf{id}| < \tau$, output $\perp$. Otherwise, do the following.

1. Let $S = \mathsf{id}' \cap \mathsf{id}$. Compute $((\delta_{S,j}\}_{j\in S}, \gamma_S) = \mathsf{GetCoeffs}(1^\ell, 1^\tau, S)$.
2. Compute $y = \gamma_S c' - \left(\sum_{j\in S} \delta_{S,j} \mathbf{c}_j^\top \mathbf{r}_j\right)$ and $y' = y \mod p$.

Output $\mu' := \gamma_S^{-1} y' \mod p$.

Correctness. Note that if $|\mathsf{id}' \cap \mathsf{id}| < \tau$, Dec outputs $\perp$. Hence, we focus only on the case of $S = \mathsf{id}' \cap \mathsf{id}, |S| \geq \tau$. Now, $y = \mathbf{s}^\top(\gamma_S \mathbf{p} - \sum_{j\in S} \delta_{S,j} \mathbf{A}_{j,\mathsf{id}_j} \mathbf{r}_j) + p(\gamma_S e' - \sum_j \delta_{S,j} \mathbf{e}_j^\top \mathbf{r}_j) + \gamma_S \mu$. As $\mathbf{A}_{j,\mathsf{id}_j} \mathbf{r}_j = \mathsf{sh}_j$, we have by correctness of LSSS that $y = \gamma_S \mu + p(\gamma_S e' - \sum_j \delta_{S,j} \mathbf{e}_j^\top \mathbf{r}_j)$. If $q > 2(\beta + p(\beta\sqrt{m}\chi + \ell\alpha m^2 \chi\sigma))$, $y \mod q = y$ and thus $y' = \gamma_S \mu \mod p$. Thus, $\mu' = \mu$ with high probability.

Compactness. Let $\rho = \min(\tau^2, \ell - \tau)$, where τ is the matching threshold. Under the parameter convention of Section 6.2, the public parameters, a secret key, and a ciphertext have size $O^\dagger(\ell\rho^6)$, $O^\dagger(\ell\rho^3)$, and $O^\dagger(\ell\rho^4)$, respectively, where $O^\dagger$ hides factors polynomial in λ and $\log \ell$. Consequently, ciphertexts and secret keys are sub-quadratic in ℓ whenever $\rho = o(\ell^{1/4})$. If the public parameters are also required to be sub-quadratic, it suffices that $\rho = o(\ell^{1/6})$.

Theorem 9.4 (Selective Security.)**.** Assuming the hardness of the LWE assumption (Assumption 3.4) holds, Construction 9.3 satisfies selective security (Definition 9.2).

Proof Sketch. We prove this theorem using the following hybrids.

Hybrid 0. This is the real selective security game from Definition 9.2 with $b = 0/1$.

Hybrid 1. In this hybrid, as id^* is given selectively, we will sample $\mathbf{A}_{i,\mathsf{id}_i^*}$ for each $i \in [\ell]$ uniformly randomly without a trapdoor. To generate secret keys for id, we will sample $\mathbf{r}_j \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^m$ for $j \in \mathsf{id} \cap \mathsf{id}^*$. Note that by admissibility, $|\mathsf{id} \cap \mathsf{id}^*| < \tau$. So, we can sample the remaining shares as $\mathsf{ProgShare}_2(1^\ell, 1^\tau, \mathbf{p}, \{\mathbf{A}_{j,\mathsf{id}_j} \mathbf{r}_j\}_{j\in\mathsf{id}\cap\mathsf{id}^*})$ and generate the pre-images accordingly. The indistinguishability statistically by programmability of LSSS and matrix distribution of lattice trapdoors.

Hybrid 2. In this hybrid, we will set $\mathsf{ct} = (\mathsf{id}, (\mathbf{c}_i^\top \leftarrow \mathbb{Z}_q^{1\times m})_{i\in\ell}, c' \leftarrow \mathbb{Z}_q)$. Indistinguishability follows by LWE security for the matrix $[\mathbf{A}_{1,\mathsf{id}_1^*} \mid \ldots \mid \mathbf{A}_{\ell,\mathsf{id}_\ell^*} \mid \mathbf{p}] \in \mathbb{Z}_q^{n\times(\ell m+1)}$.

We omit full proof as it is similar to [ABV+12]. □

By relying on the decomposed LWE assumption, we can make the ciphertext fully compact. Although using Attribute-Based Encryption for circuits [GVW13, BGG+14], we can also achieve compact ciphertexts, such a generic construction will not have great practical performance over a direct algebraic construction.

Acknowledgements

We thank the anonymous reviewers for pointing out that our final parameters for threshold policies can be improved under the succinct LWE assumption at the additional cost of assuming a CRS. We also thank David Wu for answering questions about [CW26a]. Finally, we acknowledge the concurrent work of David Wu, Jeffrey Champion, and Shota Yamada [CWY26], which prompted us to revisit our lattice parameter analysis.

References

[ABB10] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 553–572, French Riviera, May 30 – June 3, 2010. Springer Berlin Heidelberg, Germany. 15, 70, 72

[ABN+22] Shweta Agrawal, Rajarshi Biswas, Ryo Nishimaki, Keita Xagawa, Xiang Xie, and Shota Yamada. Cryptanalysis of boyen's attribute-based encryption scheme in TCC 2013. *Designs, Codes and Cryptography*, 90(10):2301–2318, 2022. 8, 28

[ABV+12] Shweta Agrawal, Xavier Boyen, Vinod Vaikuntanathan, Panagiotis Voulgaris, and Hoeteck Wee. Functional encryption for threshold functions (or fuzzy ibe) from lattices. In *International Workshop on Public Key Cryptography*, pages 280–297. Springer, 2012. 4, 5, 6, 13, 19, 23, 24, 61

[AGHS13] Shweta Agrawal, Craig Gentry, Shai Halevi, and Amit Sahai. Discrete Gaussian leftover hash lemma over infinite domains. In Kazue Sako and Palash Sarkar, editors, *Advances in Cryptology – ASIACRYPT 2013, Part I*, volume 8269 of *Lecture Notes in Computer Science*, pages 97–116, Bengalore, India, December 1–5, 2013. Springer Berlin Heidelberg, Germany. 70

[AGY26] Abtin Afshar, Rishab Goyal, and Saikumar Yadugiri. Distributed monotone-policy encryption with silent setup from lattices. Cryptology ePrint Archive, Paper 2026/372, 2026. 1

[AJL+12] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 483–501, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany. 1, 16

[AMR25] Damiano Abram, Giulio Malavolta, and Lawrence Roy. Key-homomorphic computations for RAM: Fully succinct randomised encodings and more. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part III*, volume 16002 of *Lecture Notes in Computer Science*, pages 236–268, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. 1, 2, 8, 16

[AR13] Divesh Aggarwal and Oded Regev. A note on discrete gaussian combinations of lattice vectors. *arXiv preprint arXiv:1308.2405*, 2013. 70

[ASY22] Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. *Cryptology ePrint Archive*, 2022. 2, 5

[BC02] Amos Beimel and Benny Chor. Universally ideal secret-sharing schemes. *IEEE Transactions on Information Theory*, 40(3):786–794, 2002. 19

[BÇM21] Marshall Ball, Alper Çakan, and Tal Malkin. Linear threshold secret-sharing with binary reconstruction. In Stefano Tessaro, editor, *ITC 2021: 2nd Conference on Information-Theoretic Cryptography*, volume 199 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 12:1–12:22, Seattle, WA, USA, July 23–26, 2021. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik. 2, 5, 19

[BFM+25] Zvika Brakerski, Offir Friedman, Avichai Marmor, Dolev Mutzari, Yuval Spiizer, and Ni Trieu. Threshold fhe with efficient asynchronous decryption. *Cryptology ePrint Archive*, 2025. 2, 14

[BGG+14] Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit abe and compact garbled circuits. In *Advances in Cryptology–EUROCRYPT 2014: 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings 33*, pages 533–556. Springer, 2014. 13, 61

[BGG+18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part I*, volume 10991 of *Lecture Notes in Computer Science*, pages 565–596, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland. 1, 2, 4, 5, 6, 10, 11, 12, 13, 14, 18, 19, 23, 24, 43, 53

[BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, pages 309–325, Cambridge, MA, USA, January 8–10, 2012. Association for Computing Machinery. 4

[BGW88] Michael BenOr, S Goldwasser, and Avi Wigderson. Completeness theorems for noncryptographic fault-tolerant distributed computations. In *Proceedings of the 20th Annual Symposium on the Theory of Computing (STOC'88)*, pages 1–10, 1988. 19

[BKW25] Chris Brzuska, Michael Klooß, and Ivy K. Y. Woo. Threshold public-key encryption: Definitions, relations, and CPA-to-CCA transforms. Cryptology ePrint Archive, Report 2025/1665, 2025. 13

[BL90] Josh Cohen Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In Shafi Goldwasser, editor, *Advances in Cryptology – CRYPTO'88*, volume 403 of *Lecture Notes in Computer Science*, pages 27–35, Santa Barbara, CA, USA, August 21–25, 1990. Springer, New York, USA. 2, 27, 30

[BLMR13] Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic PRFs and their applications. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 410–428, Santa Barbara, CA, USA, August 18–22, 2013. Springer Berlin Heidelberg, Germany. 5, 6, 13, 19, 23, 24

[BM24] Zvika Brakerski and Stav Medina. Limits on adaptive security for attribute-based encryption. In *Theory of Cryptography Conference*, pages 91–123. Springer, 2024. 3

[BP23] LTAN Brandão and Rene Peralta. Nist first call for multi-party threshold schemes. *doi*, 10:6028, 2023. 1

[BR07] Mihir Bellare and Phillip Rogaway. Robust computational secret sharing and a unified account of classical secret-sharing goals. In *Proceedings of the 14th ACM conference on Computer and communications security*, pages 172–184, 2007. 19

[BS23] Katharina Boudgoust and Peter Scholl. Simple threshold (fully homomorphic) encryption from lwe with polynomial modulus. In *international conference on the theory and application of cryptology and information security*, pages 371–404. Springer, 2023. 2, 5

[BSW07] John Bethencourt, Amit Sahai, and Brent Waters. Ciphertext-policy attribute-based encryption. In *2007 IEEE symposium on security and privacy (SP'07)*, pages 321–334. IEEE, 2007. 19

[BV11] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In Rafail Ostrovsky, editor, *52nd Annual Symposium on Foundations of Computer Science*, pages 97–106, Palm Springs, CA, USA, October 22–25, 2011. IEEE Computer Society Press. 4

[BZ14] Dan Boneh and Mark Zhandry. Multiparty key exchange, efficient traitor tracing, and more from indistinguishability obfuscation. In *34rd Annual International Cryptology Conference, CRYPTO 2014*, pages 480–499. Springer Verlag, 2014. 1

[CCD88] David Chaum, Claude Crépeau, and Ivan Damgard. Multiparty unconditionally secure protocols. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, pages 11–19, 1988. 19

[CCK23] Jung Hee Cheon, Wonhee Cho, and Jiseung Kim. Improved universal thresholdizer from threshold fully homomorphic encryption. *IACR Cryptol. ePrint Arch.*, 2023:545, 2023. 2

[CDM00] Ronald Cramer, Ivan Damgård, and Ueli Maurer. General secure multi-party computation from any linear secret-sharing scheme. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 316–334. Springer, 2000. 19

[CGMA85] Benny Chor, Shafi Goldwasser, Silvio Micali, and Baruch Awerbuch. Verifiable secret sharing and achieving simultaneity in the presence of faults. In *26th Annual Symposium on Foundations of Computer Science (sfcs 1985)*, pages 383–395. IEEE, 1985. 19

[CHW25] Jeffrey Champion, Yao-Ching Hsieh, and David J. Wu. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part III*, volume 16002 of *Lecture Notes in Computer Science*, pages 3–34, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. 9, 17, 70

[CK93] Benny Chor and Eyal Kushilevitz. Secret sharing over infinite domains. *Journal of Cryptology*, 6(2):87–95, 1993. 19

[CKL24] Wonhee Cho, Jiseung Kim, and Changmin Lee. (in) security of threshold fully homomorphic encryption based on shamir secret sharing. *Cryptology ePrint Archive*, 2024. 2

[CL25] Yijia Chang and Songze Li. {Arbitrary-Threshold} fully homomorphic encryption with lower complexity. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 8403–8422, 2025. 2, 14

[CW26a] Jeffrey Champion and David J. Wu. Distributed monotone-policy encryption for DNFs from lattices. Cryptology ePrint Archive, Paper 2026/318, To appear in Eurocrypt, 2026. 14, 62

[CW26b] Jeffrey Champion and David J. Wu. Optimal distributed monotone-policy encryption for DNFs and more from lattices. Cryptology ePrint Archive, Paper 2026/1464, 2026. 14

[CWY26] Jeffrey Champion, David J. Wu, and Shota Yamada. Silent threshold encryption from lattices. Cryptology ePrint Archive, Paper 2026/1716, 2026. 14, 62

[DCX+23] Sourav Das, Philippe Camacho, Zhuolun Xiang, Javier Nieto, Benedikt Bünz, and Ling Ren. Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023: 30th Conference on Computer and Communications Security*, pages 356–370, Copenhagen, Denmark, November 26–30, 2023. ACM Press. 1

[Des87] Yvo Desmedt. Society and group oriented cryptography: A new concept. In *conference on the theory and application of cryptographic techniques*, pages 120–127. Springer, 1987. 1

[DF90] Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO '89*, volume 435 of *Lecture Notes in Computer Science*, pages 307–315. Springer, 1990. 1, 19

[DHMR07] Vanesa Daza, Javier Herranz, Paz Morillo, and Carla Rafols. Cca2-secure threshold broadcast encryption with shorter ciphertexts. In *International Conference on Provable Security*, pages 35–50. Springer, 2007. 1

[DHMR08] Vanesa Daza, Javier Herranz, Paz Morillo, and Carla Ràfols. Ad-hoc threshold broadcast encryption with shorter ciphertexts. *Electronic Notes in Theoretical Computer Science*, 192(2):3–15, 2008. 1

[DJWW26] Lalita Devadas, Abhishek Jain, Brent Waters, and David J. Wu. Succinct witness encryption for batch languages and applications. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025*, pages 130–162, Singapore, 2026. Springer Nature Singapore. 1, 2, 13

[DKW21] Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority abe for dnf s from lwe. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 177–209. Springer, 2021. 8, 19, 28

[DPKM+24] Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 219–248. Springer, 2024. 2

[DRS04] Yevgeniy Dodis, Leonid Reyzin, and Adam Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 523–540, Interlaken, Switzerland, May 2–6, 2004. Springer Berlin Heidelberg, Germany. 15

[DRSNS24] Sabyasachi Dutta, Partha Sarathi Roy, Reihaneh Safavi-Naini, and Willy Susilo. Password protected universal thresholdizer. *Cryptology ePrint Archive*, 2024. 2

[DSDFY94] Alfredo De Santis, Yvo Desmedt, Yair Frankel, and Moti Yung. How to share a function securely. In *Proceedings of the twenty-sixth annual ACM symposium on Theory of computing*, pages 522–533, 1994. 1, 19

[dVPS26] François Colin de Verdière, Alain Passelègue, and Damien Stehlé. On threshold fully homomorphic encryption with synchronized decryptors. *Cryptology ePrint Archive*, 2026. 2, 14

[EY24] Ehsan Ebrahimi and Anshu Yadav. Strongly secure universal thresholdizer. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part III*, volume 15486 of *Lecture Notes in Computer Science*, pages 207–239, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore. 2, 44, 53

[FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. How to enhance the security of public-key encryption at minimum cost. In Hideki Imai and Yuliang Zheng, editors, *PKC'99: 2nd International Workshop on Theory and Practice in Public Key Cryptography*, volume 1560 of *Lecture Notes in Computer Science*, pages 53–68, Kamakura, Japan, March 1–3, 1999. Springer Berlin Heidelberg, Germany. 13

[Fra89] Yair Frankel. A practical protocol for large group oriented networks. In *Workshop on the Theory and Application of of Cryptographic Techniques*, pages 56–61. Springer, 1989. 1

[FWW23] Cody Freitag, Brent Waters, and David J. Wu. How to use (plain) witness encryption: Registered ABE, flexible broadcast, and more. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 498–531, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. 1

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st Annual ACM Symposium on Theory of Computing*, pages 169–178, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press. 3

[GJM+24] Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. hinTS: Threshold signatures with silent setup. In *2024 IEEE Symposium on Security and Privacy*, pages 3034–3052, San Francisco, CA, USA, May 19–23, 2024. IEEE Computer Society Press. 1, 13

[GKPW24] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 352–386, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland. 1, 13

[GKW16] Rishab Goyal, Venkata Koppula, and Brent Waters. Semi-adaptive security and bundling functionalities made generic and easy. In *Theory of Cryptography Conference*, pages 361–388. Springer, 2016. 53

[GPSW06] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *Proceedings of the 13th ACM conference on Computer and communications security*, pages 89–98, 2006. 19

[GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, pages 197–206, Victoria, BC, Canada, May 17–20, 2008. ACM Press. 15, 70

[GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *Advances in Cryptology–CRYPTO 2013: 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I*, pages 75–92. Springer, 2013. i, 4, 10, 44, 54

[GVW13] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Attribute-based encryption for circuits. In *STOC*, 2013. 13, 61

[GVW19] Rishab Goyal, Satyanarayana Vusirikala, and Brent Waters. Collusion resistant broadcast and trace from positional witness encryption. In Dongdai Lin and Kazue Sako, editors, *PKC 2019: 22nd International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 11443 of *Lecture Notes in Computer Science*, pages 3–33, Beijing, China, April 14–17, 2019. Springer, Cham, Switzerland. 1

[GY26a] Rishab Goyal and Saikumar Yadugiri. Adaptively-secure flexible and identity-based broadcast encryption from decomposed LWE. Cryptology ePrint Archive, Paper 2026/862, 2026. 1

[GY26b] Rishab Goyal and Saikumar Yadugiri. Distributed monotone policy encryption with stronger security for DNFs and threshold policies from lattices. Cryptology ePrint Archive, Paper 2026/1609, 2026. 14

[GY26c] Rishab Goyal and Saikumar Yadugiri. Equivocal broadcast encryption: Adaptively-secure optimal distributed broadcast encryption from lattices. In *Advances in Cryptology - **CRYPTO** 2026 - 46th Annual International Cryptology Conference*, 2026. 1

[HASW25] Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. Silent threshold encryption with one-shot adaptive security. *Cryptology ePrint Archive*, 2025. 13

[HLW+25] Zhenkai Hu, Haofei Liang, Xiao Wang, Xiang Xie, Kang Yang, Yu Yu, and Wenhao Zhang. Ajax: Fast threshold fully homomorphic encryption without noise flooding. *Cryptology ePrint Archive*, 2025. 2, 14

[ILL89] Russell Impagliazzo, Leonid A. Levin, and Michael Luby. Pseudo-random generation from one-way functions (extended abstracts). In *21st Annual ACM Symposium on Theory of Computing*, pages 12–24, Seattle, WA, USA, May 15–17, 1989. ACM Press. 15

[KMW23] Dimitris Kolonelos, Giulio Malavolta, and Hoeteck Wee. Distributed broadcast encryption from bilinear groups. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part V*, volume 14442 of *Lecture Notes in Computer Science*, pages 407–441, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore. 1

[LTV11] Adriana Lopez-Alt, Eran Tromer, and Vinod Vaikuntanathan. Cloud-assisted multiparty computation from fully homomorphic encryption. Cryptology ePrint Archive, Report 2011/663, 2011. 2

[LW11] Allison B. Lewko and Brent Waters. Decentralizing attribute-based encryption. In *EUROCRYPT*, pages 568–588, 2011. 2, 8, 18, 27

[Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 738–755, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany. 15

[MBH22] Christian Mouchet, Elliott Bertrand, and Jean-Pierre Hubaux. An efficient threshold access-structure for RLWE-based multiparty homomorphic encryption. Cryptology ePrint Archive, Report 2022/780, 2022. 14

[MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany. 15, 69, 70

[MRV+21] Silvio Micali, Leonid Reyzin, Georgios Vlachos, Riad S. Wahby, and Nickolai Zeldovich. Compact certificates of collective knowledge. In *2021 IEEE Symposium on Security and Privacy*, pages 626–641, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press. 1

[NY90] Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *22nd Annual ACM Symposium on Theory of Computing*, pages 427–437, Baltimore, MD, USA, May 14–16, 1990. ACM Press. 13

[OSW07] Rafail Ostrovsky, Amit Sahai, and Brent Waters. Attribute-based encryption with non-monotonic access structures. In *Proceedings of the 14th ACM conference on Computer and communications security*, pages 195–203, 2007. 30

[Pei10] Chris Peikert. An efficient and parallel Gaussian sampler for lattices. In Tal Rabin, editor, *Advances in Cryptology – CRYPTO 2010*, volume 6223 of *Lecture Notes in Computer Science*, pages 80–97, Santa Barbara, CA, USA, August 15–19, 2010. Springer Berlin Heidelberg, Germany. 70

[PS24] Alain Passelègue and Damien Stehlé. Low communication threshold fully homomorphic encryption. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 297–329. Springer, 2024. 2, 14

[Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th Annual ACM Symposium on Theory of Computing*, pages 84–93, Baltimore, MA, USA, May 22–24, 2005. ACM Press. 2, 3, 5, 16

[RSY21] Leonid Reyzin, Adam Smith, and Sophia Yakoubov. Turning hate into love: compact homomorphic ad hoc threshold encryption for scalable mpc. In *International Symposium on Cyber Security Cryptography and Machine Learning*, pages 361–378. Springer, 2021. 1

[Sha79] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979. 2, 5, 22

[Sho00] Victor Shoup. Practical threshold signatures. In Bart Preneel, editor, *Advances in Cryptology – EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 207–220. Springer, 2000. 6

[STH+23] Yukimasa Sugizaki, Hikaru Tsuchida, Takuya Hayashi, Koji Nuida, Akira Nakashima, Toshiyuki Isshiki, and Kengo Mori. Threshold fully homomorphic encryption over the torus. In *European Symposium on Research in Computer Security*, pages 45–65. Springer, 2023. 2, 14

[SW05] Amit Sahai and Brent Waters. Fuzzy identity-based encryption. In *Advances in Cryptology–EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005. Proceedings 24*, pages 457–473. Springer, 2005. 4, 59

[Wat11] Brent Waters. Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In *International workshop on public key cryptography*, pages 53–70. Springer, 2011. 19

[Wee24] Hoeteck Wee. Circuit ABE with $\mathsf{poly}(\mathsf{depth}, \lambda)$-sized ciphertexts and keys from lattices. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part III*, volume 14922 of *Lecture Notes in Computer Science*, pages 178–209, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland. 10, 36

[Wee25] Hoeteck Wee. Almost optimal KP and CP-ABE for circuits from succinct LWE. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part III*, volume 15603 of *Lecture Notes in Computer Science*, pages 34–62, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. 1, 2, 8, 10, 16, 17

[WQZD10] Qianhong Wu, Bo Qin, Lei Zhang, and Josep Domingo-Ferrer. Ad hoc broadcast encryption (poster presentation). In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *ACM CCS 2010: 17th Conference on Computer and Communications Security*, pages 741–743, Chicago, Illinois, USA, October 4–8, 2010. ACM Press. 1

[WW25] Hoeteck Wee and David J. Wu. Unbounded distributed broadcast encryption and registered ABE from succinct LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part III*, volume 16002 of *Lecture Notes in Computer Science*, pages 204–235, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. 1, 2, 8, 16, 17, 70

[WW26] Brent Waters and David J. Wu. Silent threshold cryptography from pairings: Expressive policies in the plain model. Cryptology ePrint Archive, Report 2025/1547, To appear in Eurocrypt, 2026. 1, 2, 13

A Additional Preliminaries and Proofs

A.1 Gadget Trapdoor Preliminaries

Gadget Trapdoors. Let (n, m, q) be LWE parameters such that $m \geq 3n \log q$. Then, there exists two polynomial time algorithms defined as follows.

$\mathsf{TrapGen}(1^n, 1^m, q) \to (\mathbf{A}, \mathbf{T_A})$**.** The randomized trapdoor generation algorithm takes as input lattice dimension n, number of columns m, modulus q, outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and its trapdoor $\mathbf{T_A}$.

$\mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{y}, \sigma) \to \mathbf{x}$**.** The randomized pre-image sampling algorithm takes as input $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, its trapdoor $\mathbf{T_A}$, a vector $\mathbf{y} \in \mathbb{Z}_q^n$, gaussian width parameter σ, and outputs a vector $\mathbf{x} \in \mathbb{Z}_q^m$.

These algorithms satisfy the following properties.

Matrix Distribution. If $(\mathbf{A}, \mathbf{T_A}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q)$, then $\|\mathbf{T_A}\| = 1$, $\mathbf{AT_A} = \mathbf{G}$, and

$$\{\mathbf{A} : (\mathbf{A}, \mathbf{T_A}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q)\} \approx_s \{\mathbf{A} : \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}\}$$

Sampling Correctness. For all $\sigma > 0$, any $\mathbf{y} \in \mathbb{Z}_q^n$, $\mathbf{Ax} = \mathbf{y}$ where $(\mathbf{A}, \mathbf{T_A}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q)$ and $\mathbf{x} \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{y}, \sigma)$.

Pre-image Distribution. Let $(\mathbf{A}, \mathbf{T_A})$ be such that $\mathbf{AT_A} = \mathbf{G}$. Then, for any $\sigma \geq \|\mathbf{T_A}\| m \log n$, any $\mathbf{y} \in \mathbb{Z}_q^n$,

$$\{\mathbf{x} \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{y}, \sigma)\} \approx_s \{\mathbf{x} : \mathbf{A}_\sigma^{-1}(\mathbf{y})\}$$

Here, $\mathbf{A}_\sigma^{-1}(\mathbf{y})$ denotes the (possibly inefficient) process of sampling a discrete gaussian vector with width parameter σ from the coset lattice $\Lambda_q^{\mathbf{y}}(\mathbf{A}) = \{\mathbf{x} : \mathbf{Ax} = \mathbf{y} \mod q, \mathbf{x} \in \mathbb{Z}^m\}$.

Remark A.1. SamplePre can be naturally extended to sample pre-images for matrices by sampling a pre-image vector for each column independently. We will use this version in our work. Sampling correctness and pre-image distribution properties can be naturally extended to matrices as well.

Lemma A.2 ([MP12])**.** Let n, m, q be lattice parameters such that $m \geq 3n \log q$. For all but $\mathsf{negl}(n)$ fraction of matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\sigma \geq \log m$, any vector $\mathbf{y}$ in the column space of $\mathbf{A}$, $\Pr[\|\mathbf{x}\| > \sqrt{m}\sigma : \mathbf{x} \leftarrow \mathbf{A}_\sigma^{-1}(\mathbf{y})] \leq \mathsf{negl}(m)$.

A.2 Additional Proofs

In this section, we provide additional proofs omitted in the main body.

Proof of Lemma 3.6. We prove this using the following claims.

Claim A.3. For positive integers n, m, and odd prime q, such that $m = \Omega(n \log q)$, for any $\ell = \mathsf{poly}(\lambda)$, $\mathbf{W} \leftarrow \mathbb{Z}_q^{n \times \ell m}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times \ell m}, \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + (\mathsf{vec}(\mathbf{I}_\ell))^\top \otimes \mathbf{G}), \{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{BR})\} \approx_s \{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{A})\}$ where $\mathbf{R} \leftarrow \{\pm 1\}^{\ell^2 m \times L}, \sigma = \omega(\sqrt{\log m}), \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times L}$.

Proof. It suffices to prove that the distribution of $\mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D})\mathbf{R}$ is statistically close to uniformly random distribution given $\mathbf{W}, \mathbf{D}$. It is known that linear combination of independent discrete Gaussian samples over a lattice are statistically close to a discrete Gaussian with appropriately increased width, under standard smoothing conditions [Pei10, AGHS13, AR13]. Thus, $(\mathbf{I}_\ell \otimes \mathbf{D})\mathbf{R}$ is a convolution of discrete Gaussian samples and as such is a sufficiently wide Gaussian. As $\mathbf{W} \leftarrow \mathbb{Z}_q^{n \times m}$ is a universal hash function, this follows from Lemma 3.2, Item 3. □

Claim A.4. For positive integers n, m, and odd prime q, such that $m = \Omega(n \log q)$, for any $\ell = \mathsf{poly}(\lambda)$ $\mathbf{W} \leftarrow \mathbb{Z}_q^{n \times \ell m}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times \ell m}, \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + (\mathsf{vec}(\mathbf{I}_\ell))^\top \otimes \mathbf{G}), \{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{BR}, \mathbf{e}^\top \mathbf{R})\} \approx_s \{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{A}, \mathbf{e}^\top \mathbf{R})\}$ where $\mathbf{R} \leftarrow \{\pm 1\}^{\ell^2 m \times L}, \sigma = \omega(\sqrt{\log m}), \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times L}$, $\mathbf{e}^\top \in \mathbb{Z}_q^{\ell^2 m}$ is any fixed vector.

Proof. This follows by a similar argument as Lemma 3.6 and [ABB10, Lemma 13]. □

Lemma A.5 ([GPV08, MP12])**.** Let n, m, q be lattice parameters such that $m = \Omega(n \log q)$, q is an odd prime. Then, for $\sigma > \log m$, all but negligible fraction of matrices $\mathbf{B} \in \mathbb{Z}_q^{n \times m}$,

$$\{(\mathbf{B}, \mathbf{B}_\sigma^{-1}(\mathbf{y}), \mathbf{y}) : \mathbf{y} \leftarrow \mathbb{Z}_q^n\} \approx_s \{(\mathbf{B}, \mathbf{z}, \mathbf{Bz}) : \mathbf{z} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^m\}$$

This can also be extended to matrices naturally by sampling column-wise.

Lemma A.6 ([CHW25, WW25])**.** Let n, m, q be lattice parameters such that $m = \Omega(n \log q)$, q is an odd prime. Then, for any $\mathbf{y} \in \mathbb{Z}_q^n, \mathbf{z} \in \mathbb{Z}_q^m$, all but negligible fraction of matrices $\mathbf{B} \in \mathbb{Z}_q^{n \times m}$,

$$\{(\mathbf{B}, \mathbf{B}_\sigma^{-1}(\mathbf{y} + \mathbf{Bz}))\} \approx_s \{(\mathbf{B}, \mathbf{B}_\sigma^{-1}(\mathbf{y}) + \mathbf{z})\}$$

where $\sigma > \lambda^{\omega(1)} \sqrt{m} \, \|\mathbf{z}\|$. This can also be extended to matrices naturally by sampling column-wise.

Lemma A.7. Let n, m, q be lattice parameters such that $m = \Omega(n \log q)$, q is an odd prime. Then, for $\sigma > \log m$,

$$\left\{(\mathbf{W}, \mathbf{D}, \mathbf{B}) : \mathbf{W} \leftarrow \mathbb{Z}_q^{n \times 2m^3}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times 2m^3}, \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}\right\} \approx_s$$
$$\left\{(\mathbf{W}, \mathbf{D}, \mathbf{B}) : (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \widetilde{\mathsf{MCOM.Setup}}(1^n, 1^m, q, \sigma)\right\}$$

Proof. Parse $\mathbf{W}$ as $[\mathbf{W}_1 \mid \ldots \mid \mathbf{W}_{2m^2}] \in \mathbb{Z}_q^{n\times 2m^3}$, $\mathbf{D}$ as $[\mathbf{D}_1 \mid \ldots \mid \mathbf{D}_{2m^2}] \in \mathbb{Z}_q^{m\times 2m^3}$.

$$\left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \mathbf{W} \leftarrow \mathbb{Z}_q^{n\times 2m^3}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times 2m^3}, \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}\right\}$$

$$\approx_s \left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \begin{array}{l}(\mathbf{W}_1,\mathbf{T}_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n,1^m,q),\\ \mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2} \leftarrow \mathbb{Z}_q^{n\times(2m^2-1)m},\\ \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times 2m^3},\\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}\end{array}\right\} \quad \text{(By matrix distribution of gadget trapdoors)}$$

$$\approx_s \left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \begin{array}{l}(\mathbf{W}_1,\mathbf{T}_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n,1^m,q),\\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times m},\\ \mathbf{A} \leftarrow \mathbb{Z}_q^{n\times m}, \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1,\mathbf{T}_{\mathbf{W}_1},\mathbf{A},\sigma),\\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m},\\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}\end{array}\right\} \quad \text{(By Lemma A.5)}$$

$$\approx_s \left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \begin{array}{l}(\mathbf{W}_1,\mathbf{T}_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n,1^m,q),\\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times m},\\ (\mathbf{A},\mathbf{T}_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n,1^m,q),\\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1,\mathbf{T}_{\mathbf{W}_1},\mathbf{A},\sigma),\\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m},\\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}\end{array}\right\} \quad \text{(By matrix distribution of gadget trapdoors)}$$

$$\equiv \left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : (\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{T}_{\mathbf{B}}) \leftarrow \widetilde{\mathsf{MCOM.Setup}}(1^n,1^m,q,\sigma)\right\} \quad (\text{where } \mathbf{T}_{\mathbf{B}} = \mathbb{1}_{4m^4,2}\otimes\mathbf{T}_{\mathbf{A}})$$

□

Lemma A.8. Let n, m, q be lattice parameters such that $m = \Omega(n\log q)$, q is an odd prime. Then, for $\sigma > \log m$,

$$\left\{(\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{d},\mathbf{y}) : \begin{array}{l}(\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{T}_{\mathbf{B}}) \leftarrow \widetilde{\mathsf{MCOM.Setup}}(1^n,1^m,q,\sigma),\\ \mathbf{y} \leftarrow \mathbb{Z}_q^n, \mathbf{d} \leftarrow \mathsf{SamplePre}(\mathbf{B},\mathbf{T}_{\mathbf{B}},\mathbf{y},\sigma)\end{array}\right\} \approx_s$$
$$\left\{(\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{d},\mathbf{B}\mathbf{d}) : \begin{array}{l}(\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{T}_{\mathbf{B}}) \leftarrow \widetilde{\mathsf{MCOM.Setup}}(1^n,1^m,q,\sigma),\\ \mathbf{d} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{4m^5}\end{array}\right\}$$

This can also be extended to matrices naturally by sampling column-wise

Proof. Parse $\mathbf{W}$ as $[\mathbf{W}_1 \mid \ldots \mid \mathbf{W}_{2m^2}] \in \mathbb{Z}_q^{n\times 2m^3}$, $\mathbf{D}$ as $[\mathbf{D}_1 \mid \ldots \mid \mathbf{D}_{2m^2}] \in \mathbb{Z}_q^{m\times 2m^3}$. Note that by construction, there exists a matrix $\mathbf{A}, \mathbf{T}_{\mathbf{A}}$ inside $\mathbf{B}, \mathbf{T}_{\mathbf{B}}$. Moreover, $\mathbf{A} = \mathbf{W}_1\mathbf{D}_2$. Parse $\mathbf{d} \in \mathbb{Z}_q^{4m^5}$ as

$\mathbf{d} = [\mathbf{d}_1^\top \mid \ldots \mid \mathbf{d}_{4m^4}^\top]^\top$. Let $\widetilde{\mathbf{y}} = \mathbf{B}(\mathbf{d} - \mathbb{1}_{4m^4,2} \otimes \mathbf{d}_2) \in \mathbb{Z}_q^n$ in each of these hybrids below.

$$
\begin{aligned}
&\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{B}\mathbf{d}) : \begin{array}{l} (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \mathsf{MCOM.}\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma), \\ \mathbf{d} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{4m^5} \end{array} \right\} \\
\equiv &\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{A}\mathbf{d}_2 + \widetilde{\mathbf{y}}) : \begin{array}{l} (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \mathsf{MCOM.}\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma), \\ \mathbf{d} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{4m^5} \end{array} \right\} \quad (\text{as } \widetilde{\mathbf{y}} + \mathbf{A}\mathbf{d}_2 = \mathbf{B}\mathbf{d}) \\
\approx_s &\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{y}' + \widetilde{\mathbf{y}}) : \begin{array}{l} (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \mathsf{MCOM.}\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma), \\ \mathbf{y}' \leftarrow \mathbb{Z}_q^n, \mathbf{d}_1 \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^m, [\mathbf{d}_3^\top \mid \ldots \mid \mathbf{d}_{4m^4}^\top]^\top \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{(4m^4-2)m}, \\ \mathbf{d}_2 \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{y}', \sigma) \end{array} \right\} \quad (\text{By Lemma A.5}) \\
\equiv &\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{y}) : \begin{array}{l} (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \mathsf{MCOM.}\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma), \\ \mathbf{y} \leftarrow \mathbb{Z}_q^n, \mathbf{d}_1 \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^m, [\mathbf{d}_3^\top \mid \ldots \mid \mathbf{d}_{4m^4}^\top]^\top \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{(4m^4-2)m}, \\ \mathbf{d}_2 \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{y} - \widetilde{\mathbf{y}}, \sigma) \end{array} \right\} \\
\approx_s &\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{y}) : \begin{array}{l} (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \mathsf{MCOM.}\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma), \\ \mathbf{y} \leftarrow \mathbb{Z}_q^n, \mathbf{d} \leftarrow \mathsf{SamplePre}(\mathbf{B}, \mathbf{T_B}, \mathbf{y}, \sigma) \end{array} \right\} \quad (\text{By [ABB10]})
\end{aligned}
$$

□

Lemma A.9. Let n, m, q be lattice parameters such that $m = \Omega(n \log q)$, q is an odd prime. Then, for any $\mathbf{y} \in \mathbb{Z}_q^n, \mathbf{z} \in \mathbb{Z}_q^{4m^5}$,

$$
\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}) : \begin{array}{l} (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \mathsf{MCOM.}\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma), \\ \mathbf{d} \leftarrow \mathsf{SamplePre}(\mathbf{B}, \mathbf{T_B}, \mathbf{y} + \mathbf{B}\mathbf{z}, \sigma) \end{array} \right\} \approx_s
\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}) : \begin{array}{l} (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{T_B}) \leftarrow \mathsf{MCOM.}\widetilde{\mathsf{Setup}}(1^n, 1^m, q, \sigma), \\ \mathbf{d} \leftarrow \mathsf{SamplePre}(\mathbf{B}, \mathbf{T_B}, \mathbf{y}, \sigma) + \mathbf{z} \end{array} \right\}
$$

where $\sigma > \lambda^{\omega(1)} \sqrt{m} \, \|\mathbf{z}\|$. This can also be extended to matrices naturally by sampling column-wise.

Proof. This is similar to proof of Lemma A.8 except using Lemma A.6. □